UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

Automatización inteligente del registro de incidentes en mesas de ayuda empresariales mediante orquestación de flujos con N8N y procesamiento de lenguaje natural basado en modelos de lenguaje grandes

Trabajo Final de Carrera presentado para obtener el título de

Técnico Universitario en Programación

Autores

Luca Gomez · Carla Bustos · Gonzalo Sevilla

Directores

Prof. Alberto Cortez y Prof. Ariel Enferrel

Mendoza, República Argentina

21 de octubre de 2026

Resumen

La digitalización de los procesos organizacionales incrementa la dependencia de los sistemas informáticos y la criticidad de la gestión de incidentes. En muchas organizaciones medianas el registro inicial aún es manual, lo que genera demoras, errores de clasificación y uso ineficiente de recursos técnicos.

Esta investigación diseña, implementa y valida un sistema automatizado de mesa de ayuda que integra orquestación con N8N, un módulo de lenguaje natural en Python con FastAPI, inferencia con Gemini 3.6 Flash, persistencia PostgreSQL y telefonía con Twilio. Recibe solicitudes por correo electrónico, formulario web y teléfono, y deriva cada incidente a uno de los cinco sectores canónicos mediante un clasificador híbrido de reglas determinísticas y un modelo de lenguaje grande.

El estudio adopta un enfoque cuantitativo cuasiexperimental de muestras pareadas sobre los mismos 200 casos, con muestreo estratificado. El doble etiquetado independiente alcanzó un Kappa de Cohen de 0,87.

Los resultados evidencian una reducción estadísticamente significativa del tiempo medio de registro, de 61,6 s a 17,76 s (71,2 %). La prueba de Wilcoxon arrojó $W = 363$ y un tamaño del efecto $r = 0{,}9639$. La exactitud estricta alcanzó 73,5 % y el macro-F1 de cinco clases, 0,5207. El estado de la hipótesis principal se declara por sub-hipótesis: se confirma en la reducción del tiempo de registro, se refuta en el umbral de clasificación y se declara no medida en la proporción de intervención humana por caso.

Los hallazgos confirman la viabilidad técnica, económica y operativa de automatizar el registro de tickets en organizaciones medianas, y aportan una arquitectura reproducible y evidencia empírica sobre modelos de lenguaje grandes en español rioplatense.

Palabras clave: automatización, mesa de ayuda, procesamiento de lenguaje natural, modelo de lenguaje grande, N8N, FastAPI, clasificación de tickets, español rioplatense, human in the loop

Abstract

The digitalization of organizational processes increases dependence on information systems and, with it, the criticality of incident management. In many medium-sized organizations, initial incident registration is still performed manually, introducing delays, classification errors, and inefficient use of technical resources.

This thesis designs, implements, and validates an automated help desk system integrating N8N workflow orchestration, a Python natural language processing module built with FastAPI, inference through the Gemini 3.6 Flash model, PostgreSQL persistence, and a telephony channel managed by Twilio. It receives requests through email, web form, and phone calls with automatic transcription, and routes each incident to one of the five canonical sectors using a hybrid classifier that combines deterministic rules with a large language model.

The study adopts a quantitative approach with a quasi-experimental paired-samples design over the same 200 cases, drawn by proportional stratified sampling. Independent double labeling reached a Cohen's Kappa of 0.87.

Results show a statistically significant reduction in mean registration time, from 61.6 s to 17.76 s (71.2 %). The Wilcoxon signed-rank test yielded $W = 363$ with an effect size $r = 0.9639$. Strict classification accuracy reached 73.5 % and the five-class macro-F1 was 0.5207. The state of the main hypothesis is declared per sub-hypothesis: it is confirmed in registration time, refuted in the classification threshold, and declared not measured for per-case human intervention.

The findings confirm the technical, economic, and operational feasibility of automating ticket registration in medium-sized organizations, and provide a reproducible architecture and empirical evidence on large language models in Rioplatense Spanish.

Keywords: automation, help desk, natural language processing, large language model, N8N, FastAPI, ticket classification, Rioplatense Spanish, human in the loop

Indice general

Glosario de siglas y acrónimos **10**

1 Introducción **11**

1.1 Contextualización 11

1.2 Planteamiento del problema 11

1.3 Pregunta de investigación 12

1.4 Hipótesis de trabajo 13

1.5 Objetivo general 14

1.6 Objetivos específicos 14

1.7 Justificación 16

1.8 Alcance y delimitaciones 17

2 Marco teórico **18**

2.1 Gestión de servicios de tecnologías de la información 18

2.2 Mesa de ayuda y modelo ITIL 18

2.3 Automatización de procesos y orquestación de flujos 19

2.4 Human-in-the-loop y grados de autonomía 20

2.5 Procesamiento de lenguaje natural y modelos de lenguaje grandes 21

2.6 Reconocimiento automático del habla 23

2.7 Bases de datos relacionales y persistencia transaccional 24

2.8 Métricas de evaluación de clasificadores multiclase 24

3 Estado del arte **26**

3.1 Soluciones comerciales de mesa de ayuda 26

3.2 Plataformas de orquestación de flujos 27

3.3 Trabajos académicos previos sobre clasificación de tickets 28

3.4 Análisis comparativo y brecha identificada 29

4 Marco metodológico **31**

4.1 Enfoque y tipo de investigación 31

4.2 Diseño experimental 31

4.3 Operacionalización de variables 32

4.4 Población, muestra y corpus 32
4.5 Protocolo de pruebas 35
4.6 Métricas e instrumentos 36
4.7 Análisis estadístico 36
4.8 Amenazas a la validez 38
4.9 Gestión del proyecto bajo enfoque Scrumban (método de construcción del artefacto) 39

5 Arquitectura del sistema 40
5.1 Visión general 40
5.2 Capa de canales de entrada 42
5.3 Capa de orquestación 44
5.4 Capa de procesamiento 44
5.5 Subsistema de clasificación híbrido 45
5.6 Modelo de datos 46
5.7 Contrato de la interfaz REST 48

6 Implementación 51
6.1 Entorno de despliegue y dependencias 51
6.2 Construcción del módulo Python 51
6.3 Construcción del flujo en N8N 52
6.4 Integración del canal telefónico 55
6.5 Desafíos de integración 55
6.6 Conexión con el proceso Scrumban 57
6.7 Pruebas automatizadas 58
6.8 Dashboard de analítica 59

7 Consideraciones éticas y aspectos legales 61
7.1 Marco normativo aplicable 61
7.2 Principios aplicados al tratamiento de datos 61
7.3 Transferencia internacional y pseudonimización 63
7.4 Seguridad técnica 63
7.5 Consentimiento, derechos del usuario y supervisión humana 64

8 Resultados **66**

8.1 Comparación de tiempos de registro 66

8.2 Matriz de confusión y métricas de clasificación 68

8.3 Reducción de la intervención humana 71

8.4 Calibración de la confianza 71

8.5 Análisis de errores y casos límite 72

9 Discusión **74**

9.1 Interpretación de los resultados 74

9.2 Comparación con la literatura 77

9.3 Limitaciones del estudio 78

9.4 Implicancias prácticas 79

9.5 Reflexiones sobre la generalización 81

10 Conclusiones **82**

10.1 Cumplimiento de los objetivos del trabajo 82

10.2 Aportes y generalización 84

11 Recomendaciones y líneas de trabajo futuro **85**

11.1 Incorporación de un clasificador supervisado con corpus propio 85

11.2 Ampliación de canales de entrada 85

11.3 Panel de monitoreo en tiempo real 85

11.4 Mecanismos de aprendizaje activo 86

11.5 Estudios de replicación 87

11.6 Evaluación de modelos de lenguaje de código abierto 87

11.7 Base de conocimientos para resolución automática 87

11.8 Agente conversacional telefónico en tiempo real 88

Referencias bibliograficas **90**

Appendices **96**

A Diagrama de arquitectura del sistema **96**

B Repositorio de código fuente **96**

C Esquema de base de datos **97**

D Especificación OpenAPI de la interfaz REST **97**

E Configuración del flujo N8N **98**

F Corpus de validación **99**

G Documentación operativa **100**

H Especificación del prompt de Gemini 3.6 Flash **101**

H.1 Estructura y contenido del prompt 101

H.2 Parámetros de configuración 102

H.3 Validación de la respuesta 102

H.4 Iteración y mejora del prompt 103

H.5 Recomendaciones para futuras iteraciones 103

I Trazabilidad de las correcciones del dictamen **103**

Indice de tablas

1 Comparación de soluciones para gestión automatizada de mesa de ayuda 30
2 Operacionalización de variables del estudio 33
3 Capas funcionales de la arquitectura del sistema 40
4 Entidades principales del modelo de datos 47
5 Puntos de entrada principales de la interfaz REST 50
6 Estadísticos descriptivos de los tiempos de registro por flujo ($n = 200$) 66
7 Tiempo automatizado de registro por canal de origen ($n = 200$) 67
8 Matriz de confusión del clasificador automático ($n = 200$). Filas: sector asignado; columnas: sector predicho principal. 68
9 Métricas de clasificación por clase con intervalo de confianza al 95 % (una contra el resto) 70
10 Exactitud por banda de confianza declarada 71
11 Confusiones observadas en la clasificación automática, por frontera 72
12 Cumplimiento de los objetivos del trabajo y evidencia de respaldo 82
13 Parámetros de configuración de la inferencia con `gemini-3.6-flash` 102

Indice de figuras

1 Diagrama de componentes del módulo Python organizado en capas (routes, services, repositories, classifiers). 41
2 Diagrama de arquitectura del sistema. La versión completa en notación UML se detalla en el Anexo A y en las fuentes Mermaid del repositorio (`docs/diagrams/`). 42
3 Diagrama de despliegue del entorno local. Servicios, red Docker, volúmenes y servicios externos. 43
4 Diagrama de secuencia del flujo extremo a extremo de un incidente, desde el canal de entrada hasta la confirmación o la derivación a revisión humana. 44
5 Diagrama entidad-relación de las siete entidades principales del modelo de datos. El esquema completo se documenta en el Anexo C. 49
6 Distribución del tiempo de registro por canal de origen (manual frente a automatizado). El panel de telefonía incorpora la duración de la llamada grabada. 68
7 Matriz de confusión del clasificador ($n = 200$). Filas: sector asignado (verdad); columnas: sector predicho principal. 69
8 Reconstrucción esquemática del flujo de orquestación N8N 99

Glosario de siglas y acrónimos

ACID Atomicity, Consistency, Isolation, Durability

AES Advanced Encryption Standard

AI Artificial Intelligence

APA American Psychological Association

API Application Programming Interface

ARCO Acceso, Rectificación, Cancelación y Oposición

ASGI Asynchronous Server Gateway Interface

ASR Automatic Speech Recognition

BPA Business Process Automation

CBC Cipher Block Chaining

CE Comunidad Europea

CI Continuous Integration

CIA Confidentiality, Integrity, Availability

CRUD Create, Read, Update, Delete

DAG Directed Acyclic Graph

ETL Extract, Transform, Load

FK Foreign Key

FRM Facultad Regional Mendoza

GB Gigabyte

GET método HTTP de lectura

GPU Graphics Processing Unit

HMAC Hash-based Message Authentication Code

HMM Hidden Markov Model

HTTP Hypertext Transfer Protocol

IA Inteligencia Artificial

IDF Inverse Document Frequency

IMAP Internet Message Access Protocol

IP Internet Protocol

IT Information Technology

ITIL Information Technology Infrastructure Library

ITSM Information Technology Service Management

JSON JavaScript Object Notation

JWT JSON Web Token

LLM Large Language Model

MVCC Multi-Version Concurrency Control

N8N plataforma de orquestación de flujos (*nodemation*)

NLP Natural Language Processing

ORM Object-Relational Mapper

PATCH método HTTP de actualización parcial

PCM Pulse Code Modulation

PK Primary Key

PNG Portable Network Graphics

POST método HTTP de creación

REST Representational State Transfer

SDK Software Development Kit

SHA Secure Hash Algorithm

SLA Service Level Agreement

SMTP Simple Mail Transfer Protocol

SPA Single Page Application

SPOC Single Point of Contact

SQL Structured Query Language

SSD Solid-State Drive

SSO Single Sign-On

SVM Support Vector Machine

TF Term Frequency

TLS Transport Layer Security

TSO The Stationery Office

UI User Interface

UML Unified Modeling Language

USD United States Dollar

UTN Universidad Tecnológica Nacional

UUID Universally Unique Identifier

VM Virtual Machine

WIP Work In Progress

1. Introducción

1.1. Contextualización

La evolución de las tecnologías de la información ha transformado profundamente la dinámica operativa de las organizaciones contemporáneas. En la actualidad los sistemas informáticos constituyen el soporte fundamental para la ejecución de procesos administrativos, productivos y de comunicación, y esa dependencia tecnológica ha incrementado de manera correlativa la importancia de los servicios de soporte técnico, particularmente aquellos relacionados con la gestión de incidentes informáticos. La disciplina que da marco a esa actividad —la gestión de servicios de tecnologías de la información (ITSM)— se desarrolla en la Sección 2, donde se define su alcance y su relación con la mesa de ayuda.

No obstante, en muchas organizaciones medianas el proceso de registro inicial de incidentes continua realizándose manualmente, lo que introduce demoras, inconsistencias y una marcada dependencia del criterio individual del operador.

El contexto argentino presenta características particulares que acentúan esta problemática. Las organizaciones medianas del país —segmento que la Secretaría de la Pequeña y Mediana Empresa encuadra por tramos según el sector y parámetros como el personal ocupado o el volumen de ventas anuales (Secretaría de la Pequeña y Mediana Empresa, 2026)— operan con presupuestos de tecnología acotados y enfrentan restricciones cambiarias que encarecen la adopción de soluciones comerciales internacionales. La combinación de estas restricciones con una demanda creciente de servicios digitales configura un escenario en el que las soluciones basadas en componentes de código abierto y servicios de inferencia bajo demanda resultan particularmente atractivas desde el punto de vista económico y operativo.

1.2. Planteamiento del problema

La presente investigación se desarrolla en el marco de una organización mediana del sector servicios con sede en la provincia de Mendoza, República Argentina, cuya plantilla asciende a aproximadamente 120 usuarios internos distribuidos en cinco áreas funcionales. El área de operaciones desempeña simultáneamente tareas vinculadas a la gestión operativa cotidiana y a la atención de incidentes informáticos reportados por los usuarios, lo que obliga al personal responsable a interrumpir actividades críticas para registrar manualmente fallas de hardware, inconvenientes de software o solicitudes de asistencia técnica.

Las mediciones preliminares realizadas durante una semana laboral representativa evidenciaron un volumen promedio de 42 incidentes diarios, un tiempo medio de registro manual de 2 minutos con 45 segundos por incidente y una tasa de derivación erronea inicial cercana al 15 %. Aquel valor de 2 minutos con 45 segundos constituyó una estimación a priori, obtenida del promedio de una semana completa de operación sin cronometraje individual por caso, y sirvió para dimensionar el problema. La campaña experimental posterior midió el tiempo de registro incidente por incidente con cronometraje en vivo, consignado en la planilla descrita en la Subsección 4.2, y arrojó una media de 61,6 s sobre los 200 casos del corpus, sustancialmente inferior a la estimación preliminar. Sobre estos indicadores, el tiempo que el personal podría redirigir a tareas de mayor valor agregado si la etapa de registro estuviera automatizada se estima en el orden de 32,2 horas-persona por trimestre (véase Subsección 9.1). Este tiempo representa un costo de oportunidad significativo para la organización.

El registro manual de incidentes implica además la lectura de correos electrónicos, la interpretación de descripciones textuales heterogeneas, la clasificación del problema según el área responsable y la carga manual de la información en un sistema de tickets. Este procedimiento requiere tiempo, interrumpe las tareas del personal técnico y puede generar errores de clasificación que prolongan la cadena de resolución cuando un ticket es derivado a un área incorrecta. La dispersión de los canales de comunicación —correo, llamada telefónica y formularios internos— agrava la situación al obligar al personal a monitorear multiples fuentes simultáneamente.

Este escenario responde a un patrón documentado en la literatura: Galup et al. (2009) advierten que los procesos de soporte no automatizados tienden a degradarse cuando crece el volumen de solicitudes, con un impacto desproporcionado en organizaciones medianas donde el personal combina multiples roles (véase Subsección 2.1). Estos indicadores justifican la búsqueda de una alternativa tecnológica que reduzca el costo operativo, mejore la calidad de la derivación temprana y disminuya la variabilidad asociada al criterio individual.

1.3. Pregunta de investigación

A partir del problema descrito, esta investigación se orienta a responder la siguiente pregunta: ¿reduce la automatización del registro inicial de incidentes mediante orquestación de flujos y procesamiento de lenguaje natural el tiempo de registro respecto del proceso manual, y alcanza la exactitud de clasificación el umbral objetivo definido para una mesa de ayuda

empresarial mediana?

La pregunta se descompone en tres interrogantes subsidiarios que guian el diseño experimental y la selección de métricas. Primero, ¿cual es la magnitud de la reducción del tiempo de registro que puede obtenerse mediante la automatización, y resulta esa reducción estadísticamente significativa? Segundo, ¿alcanza el clasificador automático el umbral objetivo de exactitud global del 85 % y un valor F1 macro no inferior a 0,85, y como se distribuyen los errores entre las categorías objetivo? Tercero, ¿permite la arquitectura propuesta integrar canales multiples de recepción sin degradar la latencia ni la calidad de la clasificación?

1.4. Hipótesis de trabajo

La hipótesis principal de este trabajo se desdobla en tres sub-hipótesis contrastables, cada una con su propio contraste y su criterio de falsación, y no se enuncia como una conjunción indivisible de mecanismo y umbral de desempeño. La primera sostiene que la automatización reduce el tiempo de registro: su contraste es el no paramétrico de Wilcoxon sobre las medianas, con $H_0 : M_{\text{manual}} = M_{\text{automatizado}}$ frente a $H_1 : M_{\text{manual}} > M_{\text{automatizado}}$, donde M representa la mediana poblacional del tiempo de registro, y su criterio de falsación es no rechazar H_0. El contraste se rechaza con los valores vigentes $W = 363$, $p = 3{,}08\text{e-}32$ y tamaño del efecto $r = 0{,}9639$ (Sección 8); esta sub-hipótesis se declara confirmada.

La segunda postula que la automatización no incrementa la proporción de intervención humana respecto del proceso manual: su contraste previsto es la proporción de casos derivados a revisión por gate contra la fracción esperada, y su criterio de falsación es que esa proporción no resulte inferior a la esperada. No se realiza con la evidencia disponible —no existe un registro primario de intervención por caso y el gate de confianza (inferior a 0,70) no deriva casos en el corpus—, de modo que se declara *no medida* y no se cuantifica (véase Subsección 9.3).

La tercera postula que la exactitud y el F1 macro alcanzan el umbral de desempeño (exactitud no inferior al 85 % y F1 macro no inferior a 0,85), verificado contra la exactitud observada y su intervalo de confianza de Wilson, sin invocar la zona de rechazo del contraste de tiempos; su criterio de falsación es una exactitud observada por debajo del 85 % o un intervalo de confianza que no supera el umbral. El resultado vigente —73,5 % de exactitud, IC $[67{,}0\,\%, 79{,}1\,\%]$ y macro-F1 de cinco clases de 0,5207 (Sección 8)— no alcanza el umbral; esta sub-hipótesis se declara refutada.

Como hipótesis subsidiarias se plantea, en primer lugar, que la integración de multiples canales de entrada dentro de un único flujo automatizado mejora la accesibilidad del sistema y reduce la dispersión operativa sin introducir cuellos de botella adicionales. En segundo lugar, que un esquema híbrido de clasificación basado en reglas determinísticas y modelo de lenguaje grande resulta más eficiente —en términos de la relación entre exactitud, latencia y costo de inferencia— que un esquema construido exclusivamente sobre modelo externo, dado que una proporción sustancial de los incidentes presenta marcadores léxicos inequivocos que no requieren consulta al modelo de lenguaje.

1.5. Objetivo general

Evaluar la viabilidad y el desempeño de la automatización del registro inicial de incidentes de una mesa de ayuda empresarial mediana mediante la orquestación de flujos con N8N y el procesamiento de lenguaje natural en Python. El desarrollo y la implementación del sistema constituyen el medio para esa evaluación, que abarca la recepción, el procesamiento, la clasificación y el registro automáticos de incidentes y considera la reducción del tiempo de registro, la calidad de la clasificación y la proporción de intervención humana en la etapa inicial, manteniendo la supervisión técnica y la trazabilidad de las decisiones tomadas por el sistema.

1.6. Objetivos específicos

El primer objetivo específico consiste en diseñar una arquitectura distribuida y modular que integre un motor de orquestación de flujos, un módulo de procesamiento de lenguaje natural, un canal de telefonía con transcripción automática del habla y una base de datos relacional, garantizando comunicación asincrónica entre componentes mediante interfaces de programación de aplicaciones de tipo REST sobre HTTP cifrado.

El segundo objetivo específico se orienta a implementar un clasificador automático que asigne cada incidente a uno de los cinco sectores canónicos —Seguridad Informática, Soporte Técnico Hardware, Soporte Técnico Software, Bases de Datos o Sistemas— sobre un corpus controlado de 200 casos representativos del entorno operativo. El umbral de desempeño —exactitud global no inferior al 85 % y F1 macro no inferior a 0,85— no es parte definitoria de este objetivo, sino un criterio de éxito contingente verificable contra la exactitud observada y su intervalo de confianza; su grado de cumplimiento se declara en la Subsección 10.1. Ese umbral se fundamenta en tres criterios convergentes. Primero, los trabajos de Paramesh y Shreedhara

(2019) reportan exactitudes de aproximadamente 87 % con enfoques de máquinas de vectores de soporte (SVM) y representaciones TF-IDF, estableciendo un piso de referencia para el estado del arte en clasificación de tickets. Segundo, el costo organizacional de una derivación erronea implica al menos 10 a 15 minutos adicionales de reasignación según las mediciones preliminares de la organización, por lo que una exactitud inferior al 85 % resultaría en tiempos de resolución comparables a los del flujo manual para un volumen de errores inaceptable. Tercero, situar el umbral en el 85 % deja margen para que, si la exactitud verdadera del clasificador lo supera, el límite inferior de su intervalo de confianza pueda ubicarse por encima de la frontera de interés en lugar de quedar solapado con ella. Se trata, en consecuencia, de un criterio de desempeño verificable contra la exactitud observada y su intervalo de confianza, distinto del contraste sobre la mediana de los tiempos formulado en la Subsección 1.4.

El tercer objetivo específico plantea integrar tres canales de entrada paralelos —correo electrónico, formulario web y llamada telefónica con transcripción automática— dentro de un único flujo automatizado que normalice la entrada y centralice el procesamiento, mejorando la accesibilidad del sistema sin incrementar la complejidad operativa percibida por los usuarios finales.

El cuarto objetivo específico consiste en evaluar comparativamente el flujo manual y el flujo automatizado en términos de tiempo medio de registro, dispersión, exactitud de clasificación y proporción de intervención humana, mediante un diseño cuasiexperimental con muestras pareadas, prueba estadística no paramétrica y reporte de intervalos de confianza al 95 %. El diseño experimental sigue las recomendaciones de Hernández Sampieri y Mendoza Torres (2018) para investigaciones tecnológicas de carácter aplicado.

El quinto objetivo específico se orienta a documentar la arquitectura, los procedimientos de despliegue y los resultados experimentales de modo que una tercera parte pueda replicarlos, y a proponer una hoja de ruta de evolución que oriente futuras mejoras y replicaciones del estudio en organizaciones de características comparables.

Como sexto objetivo específico se plantea analizar los errores de clasificación y calibrar la confianza de las predicciones, tipificando los patrones de error y caracterizando las bandas de confianza para interpretar el desempeño más allá de las métricas agregadas. Su cumplimiento es verificable contra el reporte de evaluación del repositorio y la Subsección 8.5 de la Sección 8.

1.7. Justificación

La justificación del trabajo se sostiene en tres dimensiones complementarias.

Desde la dimensión organizacional, la automatización propuesta libera capacidad operativa del personal técnico, reduce el tiempo de respuesta percibido por los usuarios y disminuye la variabilidad introducida por el criterio individual. La evidencia recogida en este trabajo permite cuantificar esa mejora en el contexto específico de una organización mediana argentina, aportando un caso documentado que otras organizaciones pueden utilizar como referencia para estimar el retorno esperado de una inversión similar.

Desde la dimensión tecnológica, la solución combina componentes de código abierto —Python, FastAPI y PostgreSQL— con N8N, una plataforma de orquestación de flujos cuyo código fuente se distribuye bajo licencia Sustainable Use (*fair-code*) y admite despliegue completamente autoalojado (n8n GmbH, 2024), lo que permite una implementación reproducible en organizaciones de tamaño mediano sin requerir grandes inversiones de capital ni dependencia exclusiva de proveedores comerciales. Esta propiedad es particularmente relevante en el contexto argentino, donde las restricciones de acceso a divisas y los costos de suscripción en moneda extranjera constituyen barreras significativas para la adopción de soluciones comerciales internacionales. Además, el despliegue autoalojado de la totalidad de los componentes —con excepción del servicio de inferencia— otorga a la organización control pleno sobre sus datos operativos, en línea con el control sobre los datos personales y su tratamiento conforme a la normativa de protección de datos (Honorable Congreso de la Nación Argentina, 2000).

Desde la dimensión académica, el trabajo aporta evidencia empírica sobre el desempeño de modelos de lenguaje grandes aplicados a la clasificación de tickets en español rioplatense, dominio escasamente documentado en la literatura comparativa disponible. Buena parte de la literatura relevada sobre clasificación automática de tickets se apoya en corpus en inglés y, en menor medida, en alemán o portugués (Colavito et al. 2025; Zangari et al. 2023), y los pocos trabajos que incorporan el español lo hacen con variantes peninsulares o con corpus genéricos no específicos del dominio de mesas de ayuda (Grandury et al. 2025; Mayor-Rocher et al. 2026). Este trabajo contribuye a cerrar esa brecha mediante la validación rigurosa de un clasificador híbrido sobre un corpus de 200 casos en español rioplatense con doble etiquetado independiente y acuerdo inter-evaluador cuantificado.

Adicionalmente, la mayoría de las soluciones comerciales disponibles —tales como

Zendesk Suite, Freshdesk o ServiceNow ITSM— presentan modelos de costos por agente y mes que escalan linealmente con el equipo y operan exclusivamente en modalidad de software como servicio con almacenamiento de datos en jurisdicciones extranjeras. La propuesta desarrollada en esta investigación utiliza herramientas de despliegue autoalojado, lo que facilita su adopción en organizaciones medianas y simplifica el cumplimiento de la normativa argentina de protección de datos personales.

1.8. Alcance y delimitaciones

El alcance del trabajo se circunscribe a la etapa de recepción, clasificación y registro inicial de incidentes, sin abarcar la resolución técnica de los mismos, la cual permanece bajo responsabilidad del personal humano. El sistema contempla tres canales de entrada (correo electrónico, formulario web y llamada telefónica con transcripción automática) y cinco sectores de derivación (Seguridad Informática, Soporte Técnico Hardware, Soporte Técnico Software, Bases de Datos y Sistemas). La validación experimental se realiza sobre un corpus en idioma español rioplatense recolectado en una organización mediana del sector servicios con sede en la provincia de Mendoza, de aproximadamente 120 usuarios internos distribuidos en cinco áreas funcionales.

Quedan fuera del alcance los siguientes aspectos, los cuales se proponen como líneas de trabajo futuro en la Sección 11. Primero, la integración con sistemas externos de inventario de activos informáticos o con sistemas de gestión de configuración. Segundo, los modelos de aprendizaje supervisado entrenados con corpus propios de la organización, dado que el volumen actual de datos etiquetados resulta insuficiente para esa estrategia. Tercero, la implementación de paneles de monitoreo avanzados con visualización en tiempo real de métricas operativas. Cuarto, la automatización de la etapa de resolución técnica, que trasciende el alcance del presente trabajo.

Las delimitaciones del estudio incluyen: la restricción geográfica a una única organización de la provincia de Mendoza, lo que limita la validez externa de los hallazgos; la dependencia operativa del servicio externo de inferencia gemini-3.6-flash, cuyo rendimiento y condiciones comerciales escapan al control de los investigadores; y la naturaleza temporal de los resultados, que reflejan el comportamiento del modelo en la versión vigente al momento del estudio (entre junio y octubre de 2026), sujeta a modificaciones futuras por parte del proveedor.

2. Marco teórico

Este capítulo establece los fundamentos conceptuales sobre los cuales se construye la investigación. Se abordan ocho ejes temáticos que proporcionan el andamiaje teórico necesario para comprender el diseño, la implementación y la evaluación del sistema propuesto.

2.1. Gestión de servicios de tecnologías de la información

La gestión de servicios de tecnologías de la información (ITSM) constituye una disciplina sistematizada que articula procesos, personas y tecnologías con el propósito de entregar valor al negocio a través de servicios tecnológicos confiables y mensurables. Galup et al. (2009) caracterizan la disciplina como un cuerpo de prácticas orientado a alinear la operación tecnológica con los objetivos del negocio, donde la mesa de ayuda opera como punto único de contacto entre el usuario y el área de tecnología. Este enfoque rompe con la concepción anterior que entendía a la informática como un área de soporte estrictamente reactivo y la posiciona, en cambio, como un proveedor de servicios sujeto a acuerdos de nivel de servicio explícitos.

Dentro de este marco general, la gestión de incidentes constituye uno de los procesos operativos centrales. Su objetivo principal consiste en restaurar el servicio afectado en el menor tiempo posible, minimizando el impacto adverso en la operación del negocio. El proceso comprende cuatro etapas secuenciales: la recepción y registro inicial, la clasificación y priorización, la asignación al área responsable y, finalmente, la resolución y cierre. Galup et al. (2009) advierten que los procesos manuales de soporte tienden a degradarse cuando el volumen de solicitudes crece de manera no lineal o cuando el personal debe alternar entre tareas operativas heterogeneas, situación que coincide con la observada en el entorno empresarial analizado en el presente trabajo.

2.2. Mesa de ayuda y modelo ITIL

El marco de buenas prácticas ITIL (Information Technology Infrastructure Library), en su edición de operaciones de servicio (Office of Government Commerce, 2011), publicada por The Stationery Office (TSO) por encargo del Cabinet Office y cuyo titular histórico es el Office of Government Commerce, establece la distinción conceptual entre incidente —definido como una interrupción no planificada del servicio o una reducción de su calidad— y solicitud de servicio —definida como un pedido formal de información, asesoramiento, acceso a un componente

o ejecución de una operación estandarizada—. Esta distinción resulta especialmente relevante para el diseño del clasificador propuesto en este trabajo, ya que orienta la categorización inicial y condiciona la priorización subsiguiente.

El marco ITIL postula adicionalmente tres principios operativos que guian el diseño de la solución propuesta. El primero es la centralización de la recepción mediante un punto único de contacto (SPOC, por su sigla en inglés), que en el sistema propuesto se materializa a través del flujo unificado de orquestación que normaliza las entradas de los tres canales. El segundo es la operación bajo acuerdos de nivel de servicio (SLA), que cuantifican la calidad esperada en términos de tiempo de respuesta y tiempo de resolución. El tercero es la mejora continua del proceso, que en el sistema propuesto se habilita mediante la trazabilidad de cada decisión del clasificador y la retroalimentación de los sectores responsables.

Los modelos tradicionales de mesa de ayuda se basan en la interacción manual entre el usuario y el operador, lo que implica tiempos de respuesta variables y una dependencia directa del personal disponible. Cuando el volumen de incidentes excede la capacidad del equipo o cuando los operadores deben alternar simultáneamente entre tareas operativas y de soporte, la calidad del servicio se degrada de manera observable. La automatización del registro inicial constituye, en consecuencia, una intervención que aborda el cuello de botella en el punto de mayor sensibilidad del proceso, sin requerir la reestructuración completa del modelo operativo.

2.3. Automatización de procesos y orquestación de flujos

Se entiende aquí por automatización de procesos empresariales (BPA, por su sigla en inglés) la reducción de la intervención humana en tareas repetitivas mediante la ejecución automática de acciones definidas a partir de eventos específicos. Russell y Norvig (2021) ubican a la automatización inteligente dentro del paradigma de los agentes orientados a objetivos, en el cual un sistema reactivo percibe eventos del entorno, decide acciones según una política preestablecida y actúa modificando el estado del entorno. Esta caracterización ofrece un marco conceptual robusto para el diseño de sistemas que combinan reglas determinísticas con decisiones derivadas de modelos probabilísticos, como el clasificador híbrido propuesto en este trabajo.

Se entiende aquí por plataformas de integración como servicio (iPaaS) las herramientas de orquestación que posibilitan integrar distintos servicios, procesar información y ejecutar acciones automáticas en función de condiciones predefinidas. Estas plataformas se caracte-

rizan por ofrecer modelos visuales de construcción de flujos, soporte nativo para webhooks y disparadores reactivos, y capacidad de transformación de datos entre sistemas heterogeneos. n8n GmbH (2024) documenta las capacidades de N8N, la herramienta seleccionada para este trabajo, que se distingue dentro del segmento por su modelo de licencia de código fuente disponible y su capacidad de despliegue completamente autoalojado, característica que resulta decisiva en escenarios donde el tratamiento de datos sensibles exige mantener la información dentro del perimetro organizacional.

El paradigma de orquestación se diferencia conceptualmente del paradigma de coreografía. Hohpe y Woolf (2003) establecen que la orquestación centraliza la lógica de control en un componente único, lo que simplifica la trazabilidad de cada ejecución pero introduce un punto de coordinación cuya disponibilidad condiciona al sistema completo. La coreografía, en cambio, distribuye la coordinación entre los participantes mediante eventos publicados en un bus comun, lo que aumenta la robustez frente a fallos parciales pero dificulta la observabilidad. El presente trabajo adopta el enfoque de orquestación por simplicidad operativa y por la prioridad otorgada a la trazabilidad auditable de cada decisión del sistema, en línea con los requisitos de la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000).

2.4. Human-in-the-loop y grados de autonomía

El paradigma *human-in-the-loop* (HITL) designa los esquemas en los que una persona interviene en el ciclo de decisión de un sistema automático para etiquetar, corregir, validar o supervisar sus salidas. Mosqueira-Rey et al. (2023) sistematizan el aprendizaje con intervención humana y muestran que las anotaciones y correcciones incorporadas al ciclo mejoran la calidad del modelo cuando los datos escasean o el dominio es ambiguo; Zanzotto (2019) sostiene que el diseño de estos sistemas debe reservar a la persona un rol activo en el lazo ante la ambigüedad semántica y la responsabilidad sobre las consecuencias de la decisión. Amershi et al. (2019) formulan guías de interacción que exigen transparencia sobre la confianza del sistema y control del usuario, Shneiderman (2020) propone combinar alta automatización con supervisión significativa, y Wu et al. (2022) muestran que la descomposición de tareas con puntos de control humano hace al sistema inspeccionable.

Los grados de autonomía se ordenan según la proporción de decisiones delegadas y la granularidad de la revisión. La *revisión total* somete cada salida a validación humana; la *revisión selectiva* delega las decisiones de alta confianza y deriva a la persona solo las que caen

por debajo de un umbral; y la *operación autónoma con auditoría* ejecuta sin intervención previa y conserva un registro trazable para revisión posterior. El sistema propuesto adopta el grado intermedio: clasifica de forma automática y escala a revisión humana todo caso cuya confianza declarada cae por debajo del umbral, según la regla `requiere_revision_humana = (confianza <0,70)`, evaluada en la Sección 8. La calibración de esa confianza condiciona la cobertura efectiva de la salvaguarda (Sección 9).

2.5. Procesamiento de lenguaje natural y modelos de lenguaje grandes

El procesamiento de lenguaje natural (NLP) constituye un área de la inteligencia artificial orientada a la interpretación del lenguaje humano por parte de sistemas informáticos. Jurafsky y Martin (2023) sistematizan la disciplina en torno a tres ejes principales: el modelado estadístico de secuencias, la representación distribuida de palabras y la comprensión semántica mediante arquitecturas neuronales profundas. Mediante estas técnicas resulta posible analizar texto o audio transcrito, identificar palabras clave, determinar la intención del usuario y clasificar contenido en categorías predefinidas.

La evolución del campo durante la última decada ha estado dominada por la arquitectura Transformer (Vaswani et al. 2017), cuyo mecanismo de atención permite a los modelos capturar dependencias de largo alcance sin las limitaciones de las redes neuronales recurrentes. El mecanismo de atención multi-cabeza, en particular, permite que el modelo pondere simultáneamente distintas partes de la secuencia de entrada, lo que resulta especialmente útil para la clasificación de descripciones textuales donde la información relevante puede aparecer en cualquier posición del texto.

Sobre esta arquitectura se construyeron los modelos de lenguaje grandes (LLM) contemporáneos, los cuales exhiben capacidades robustas de clasificación en escenarios de pocos ejemplos (*few-shot*) o sin ejemplos (*zero-shot*), fenómeno conocido como aprendizaje en contexto. Brown et al. (2020) demostraron que modelos suficientemente grandes pueden generalizar a tareas nuevas a partir de instrucciones expresadas en lenguaje natural, sin requerir ajuste fino sobre datos específicos del dominio. Esta capacidad es particularmente relevante para el contexto de esta investigación, donde el volumen de datos etiquetados resulta insuficiente para entrenar un clasificador supervisado tradicional desde cero.

La forma en que se fórmula la instrucción condiciona el resultado tanto como la capacidad del modelo. El aprendizaje en contexto admite dos configuraciones principales: el

planteamiento *zero-shot*, en el que la instrucción describe la tarea sin ejemplos, y el *few-shot*, en el que se incorporan unos pocos casos resueltos dentro del propio mensaje para orientar la respuesta. La evaluación multilingüe de Ahuja et al. (2023) y el ajuste por instrucciones de Lai et al. (2023) evidencian que la selección de ejemplos, su orden y la redacción de la instrucción modifican el rendimiento de manera sustancial, por lo que el diseño del *prompt* constituye una decisión de ingeniería y no un detalle accesorio.

Los LLM generan, junto con la etiqueta asignada, una estimación de confianza que no necesariamente coincide con la probabilidad empírica de acierto. La calibración designa el grado de correspondencia entre la confianza declarada por el modelo y su corrección observada: un modelo bien calibrado acierta con una frecuencia próxima a la confianza que expresa. Xuan et al. (2025) documentan, a partir de una evaluación comparativa en múltiples idiomas, que la calibración depende del modelo y del dominio. En el sistema propuesto este concepto fundamenta el umbral operativo de revisión humana, ya que las clasificaciones con confianza inferior a 0,70 se derivan a supervisión; la fiabilidad de ese umbral depende tanto de la exactitud como de la calibración del clasificador.

Una limitación central de los modelos generativos es la alucinación, es decir, la producción de contenido plausible en su forma pero falso o no sustentado en la entrada (Ji et al. 2023). En una tarea de clasificación la alucinación puede manifestarse como la asignación de una categoría inexistente o como una justificación inventada a partir de indicios débiles. Colavito et al. (2025) muestran que el etiquetado automático mediante LLM requiere validación contra un conjunto de referencia y no puede aceptarse sin control. Esta observación motiva dos decisiones del sistema propuesto: la restricción del espacio de salida a las cinco categorías canónicas y la validación de cada clasificación contra el conjunto de etiquetas válidas antes de su persistencia.

El rendimiento de los LLM tampoco es uniforme entre idiomas ni entre variedades de un mismo idioma. Zhang et al. (2023) documentan que los modelos multilingües no generalizan de manera homogenea fuera del inglés, y la evaluación de variedades del español peninsular y latinoamericano de Grandury et al. (2025) y Mayor-Rocher et al. (2026) muestra diferencias de desempeño atribuibles al dialecto y a la disponibilidad de recursos. Esta sensibilidad resulta directamente relevante para el presente trabajo, cuyo corpus está redactado en español rioplatense, una variedad escasamente representada en los conjuntos de evaluación internacionales.

La evaluación de un LLM no se agota en una medida única de exactitud. Los benchmarks multilingües como los de Xuan et al. (2025) y Ahuja et al. (2023) combinan múltiples tareas y reportan resultados desagregados, en reconocimiento de que una cifra global puede ocultar un desempeño dispar entre categorías. Esta advertencia, que este capítulo desarrolla en la Sección 2.8 para el caso de clasificadores multiclase, fundamenta la decisión metodológica de reportar, además de la exactitud global, métricas por clase y el valor F1 macro promediado.

En el contexto de una mesa de ayuda, el uso de NLP permite clasificar incidentes y derivarlos al sector correspondiente sin intervención manual, mejorando la velocidad de respuesta y reduciendo la carga operativa del personal encargado del registro. La elección entre un clasificador supervisado entrenado sobre corpus específico, un modelo de lenguaje grande de propósito general invocado mediante instrucciones, o un esquema híbrido que combine ambas estrategias, constituye una decisión de diseño que depende de la disponibilidad de datos etiquetados, del presupuesto de inferencia disponible y de los requerimientos de soberanía sobre los datos procesados.

2.6. Reconocimiento automático del habla

Los sistemas de reconocimiento automático del habla (ASR) permiten convertir audio en texto mediante modelos previamente entrenados sobre grandes corpus de habla anotada. Rabiner y Juang (1993) describen los fundamentos clásicos del reconocimiento basado en modelos ocultos de Markov (HMM), que durante decadas constituyeron el enfoque dominante. La generación contemporánea de sistemas se apoya mayoritariamente en redes neuronales profundas con arquitecturas codificador-decodificador, de las cuales el modelo Whisper (Radford et al. 2023) representa un exponente reciente con alta robustez en escenarios multilingües y en presencia de ruido ambiental.

Whisper fue entrenado sobre 680.000 horas de audio multilingüe recolectadas de la web, lo que le confiere una capacidad de generalización superior a la de sistemas entrenados sobre corpus curados de menor escala. Este modelo se menciona como exponente del estado del arte del reconocimiento del habla y no forma parte de la implementación desarrollada: el canal telefónico del sistema propuesto delega la transcripción en el servicio nativo de Twilio, cuya integración se detalla en la Subsección 6.4.

El reconocimiento automático del habla se conserva únicamente como marco concep-

tual del canal telefónico; la transcripción efectiva la provee un servicio externo y no un motor de ASR integrado al sistema (Subsección 6.4).

2.7. Bases de datos relacionales y persistencia transaccional

La persistencia de los incidentes en una base de datos relacional permite garantizar las propiedades de atomicidad, consistencia, aislamiento y durabilidad (ACID) propias del modelo transaccional clásico (Date, 2003). PostgreSQL, el motor seleccionado para este trabajo, ofrece soporte completo del estándar SQL, extensiones para tipos de datos avanzados como JSON binario y arreglos, y un modelo de concurrencia basado en control multiversional (MVCC) que resulta adecuado para cargas mixtas de lectura y escritura (The PostgreSQL Global Development Group, 2024).

La elección de un motor relacional se comprende mejor al contrastarla con alternativas concretas. Un motor de documentos como MongoDB almacena registros con esquema flexible y favorece la evolución de la estructura sin migraciones, pero no impone las relaciones entre incidentes, sectores, estados y canales de origen, de modo que la integridad referencial queda delegada a la aplicación. Un motor clave-valor en memoria como Redis ofrece latencias de acceso muy bajas y resulta adecuado para cache o contadores efímeros, pero no está orientado a la persistencia duradera ni a consultas relacionales complejas. El dominio de este trabajo exige precisamente esas propiedades: las relaciones entre incidentes, sectores, estados y canales se modelan como tablas vinculadas mediante claves foráneas con integridad referencial declarativa, las operaciones de registro y actualización requieren las garantías ACID de la transacción clásica, el dashboard de analítica se apoya en consultas relacionales de agregación sobre múltiples tablas y la trazabilidad de cada decisión del sistema demanda un registro duradero y auditable. Esta combinación de requisitos justifica la elección de PostgreSQL, en línea con los requisitos normativos identificados en la sección de consideraciones éticas y legales (Sección 7).

2.8. Métricas de evaluación de clasificadores multiclase

La evaluación de clasificadores multiclase requiere métricas que capturen tanto el desempeño global como el desempeño por clase. Powers (2011) describe el conjunto canónico de métricas derivadas de la matriz de confusión, entre las cuales destacan la exactitud global (*accuracy*), la precisión por clase, la sensibilidad o exhaustividad (*recall*) por clase, y la medida

F1 como media armónica de las dos anteriores.

La exactitud global se define como la proporción de predicciones correctas sobre el total de casos evaluados:

$$\text{Exactitud} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i]$$

donde N es el número de casos, $\hat{y}_i$ la clase predicha para el caso i, y_i su clase verdadera y $\mathbb{1}[\cdot]$ la función indicadora, que vale uno cuando la predicción coincide con la etiqueta y cero en caso contrario. Esta definición es válida directamente para cualquier número de clases y es la que subyace a la matriz de confusión de cinco sectores de la Sección 8. No se emplea la formulación binaria basada en verdaderos y falsos positivos y negativos, que presupone una única clase de interés y resulta inadecuada para un problema multiclase.

Sokolova y Lapalme (2009) advierten que en escenarios con clases desbalanceadas la exactitud global puede resultar enganosa y recomiendan reportar valores macro promediados, los cuales otorgan igual peso a cada clase con independencia de su frecuencia. El valor F1 macro promediado se define como:

$$F1_{\text{macro}} = \frac{1}{k}\sum_{i=1}^{k} F1_i$$

donde k es el número de clases y $F1_i$ es el valor F1 de la clase i. El presente trabajo adopta esta recomendación y reporta tanto exactitud global como F1 macro y métricas desagregadas por clase, permitiendo al lector evaluar la robustez del clasificador frente a la distribución específica del corpus.

Adicionalmente, la matriz de confusión constituye el instrumento estándar de presentación de resultados en clasificación multiclase, ya que permite identificar patrones específicos de error: confusiones sistemáticas entre dos categorías particulares, sesgos hacía una clase mayoritaria o aciertos preferenciales sobre una clase específica. La inspección de la matriz orienta el análisis cualitativo posterior y constituye la base para cualquier propuesta de mejora del clasificador.

3. Estado del arte

Este capítulo revisa la literatura académica y las soluciones comerciales disponibles en tres áreas relevantes para el presente trabajo: plataformas de mesa de ayuda, herramientas de orquestación de flujos, y métodos de clasificación automática de tickets de soporte. El propósito es situar la contribución de esta investigación en el contexto de los desarrollos previos y fundamentar la brecha que motiva el trabajo.

Protocolo de revisión. La revisión que sustenta este capítulo y los fundamentos del marco teórico se construyó con una búsqueda estructurada cuya fecha de corte es el 21 de octubre de 2026, sobre seis fuentes: Scopus, IEEE Xplore, ACM Digital Library, ACL Anthology, Google Scholar y arXiv. Las cadenas, formuladas en inglés, combinaron términos de dominio y de método: A1, *(”help desk.ᵒR ”service desk.ᵒR ÏT support”) AND (”ticket classification.ᵒR ïncident classification”) AND (”machine learning.ᵒR ”natural language processing.ᵒR ”large language model”)*; A2, *(”large language model.ᵒR ”LLM”) AND (”ticket.ᵒR ïncident”) AND (çlassification.ᵒR routing”)*; A3, *(”Spanish.ᵒR .ᵉˢpañol”) AND (”large language model.ᵒR ”LLM”) AND (.ᵉᵛaluation.ᵒR ”benchmark”)*; A4, *(”Gemini.ᵒR ”Google Gemini”) AND (”ticket.ᵒR ïncident.ᵒR ”help desk”)*; y B1, *(”human-in-the-loop.ᵒR ”human in the loop.ᵒR ”human-AI interaction”) AND (”machine learning.ᵒR ”large language model”)*. Criterios de inclusión: publicación arbitrada 2023-2026 para el estado del arte moderno (A1-A4), idioma español o multilingüe y evaluación empírica; para los fundamentos de human-in-the-loop (B1) no se impuso restricción temporal. Criterios de exclusión: literatura no arbitrada, notas de prensa y material comercial sin respaldo metodológico. Cada consulta quedó registrada, con base, cadena, fecha y resultado, en la bitácora `docs/Tesis/Correcciones/bitacora-busqueda-c-110.md`, consistente con lo aquí declarado.

3.1. Soluciones comerciales de mesa de ayuda

El segmento de soluciones comerciales para gestión de mesa de ayuda se encuentra dominado por plataformas integrales que cubren el ciclo completo de vida del ticket. Zendesk Suite ofrece funcionalidades de recepción multicanal, automatización de flujos básicos y, mediante un módulo adicional, capacidades de inteligencia artificial para sugerencia de respuestas y deflección de consultas frecuentes. Freshdesk propone una arquitectura comparable e incorpora el motor Freddy AI para clasificación automática y enrutamiento inteligente. Jira Service Management, parte del ecosistema de Atlassian, ofrece capacidades limitadas

de automatización inteligente en su plan estándar; la edición Data Center permite el despliegue autoalojado para organizaciones con requisitos de soberanía sobre los datos. ServiceNow ITSM constituye la opción de referencia en el segmento corporativo, con un módulo Predictive Intelligence que aplica aprendizaje automático sobre el histórico de tickets de la organización.

La adopción de capacidades de inteligencia artificial en plataformas comerciales de mesa de ayuda ha crecido de manera sostenida en los últimos años, particularmente en organizaciones medianas de mercados de habla inglesa. No obstante, estas plataformas presentan dos limitaciones relevantes para el contexto de la presente investigación. La primera es el modelo de costos basado en agente y mes, que escala linealmente con el tamaño del equipo de soporte. La segunda es que la mayoría de estas plataformas operan exclusivamente en modalidad de software como servicio (SaaS) con almacenamiento de datos en jurisdicciones extranjeras, lo que introduce consideraciones adicionales sobre transferencia internacional de datos personales bajo la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000).

En el segmento de código abierto, osTicket constituye una alternativa de despliegue autoalojado sin costo de licencia, aunque carece de capacidades nativas de clasificación automática mediante procesamiento de lenguaje natural moderno. Esta brecha funcional motiva la búsqueda de arquitecturas compositivas que integren componentes maduros de código abierto con servicios de inferencia bajo demanda.

3.2. Plataformas de orquestación de flujos

Dentro del segmento de plataformas de orquestación de flujos, las soluciones más difundidas incluyen Zapier, Make (anteriormente Integromat), Microsoft Power Automate, Apache Airflow y N8N. Las primeras tres operan exclusivamente como SaaS, lo que las descarta como base para implementaciones donde el control sobre la infraestructura es un requisito explícito. Apache Airflow se orienta a flujos de trabajo programados de tipo procesamiento de datos por lotes, con un diseño centrado en grafos acíclicos dirigidos (DAG) definidos como código Python (Apache Software Foundation, 2024); aunque potente, su paradigma de ejecución programada resulta inadecuado para un caso de uso reactivo y orientado a eventos como el de una mesa de ayuda.

N8N ofrece un modelo de construcción visual de flujos sobre nodos predefinidos, soporte nativo para webhooks y disparadores reactivos, ejecución basada en eventos y posibilidad de despliegue completamente autoalojado bajo Docker (n8n GmbH, 2024). La disponibilidad

de su código fuente bajo licencia Sustainable Use facilita la auditoría de seguridad y la adaptación a requerimientos específicos sin generar dependencia exclusiva del proveedor. Estas características confluyen para posicionar a N8N como la opción más adecuada para el escenario propuesto, donde la soberanía sobre los datos y los flujos de procesamiento constituye un requisito no negociable.

3.3. Trabajos académicos previos sobre clasificación de tickets

La literatura académica sobre clasificación automática de tickets de soporte ha experimentado un crecimiento considerable durante los últimos años. A continuación se presentan los trabajos más relevantes, organizados por enfoque metodológico y por orden cronológico.

Paramesh y Shreedhara (2019) constituyen uno de los primeros trabajos exhaustivos en el área. Utilizando combinaciones de máquinas de vectores de soporte (SVM) y representaciones TF-IDF sobre un corpus en inglés de aproximadamente 10.000 tickets, reportan exactitudes cercanas al 87 %. Su contribución principal radica en la demostración de que las técnicas clásicas de aprendizaje supervisado, cuando se aplican sobre representaciones textuales adecuadas, pueden alcanzar niveles de desempeño operativamente útiles para la clasificación automática de tickets.

Revina et al. (2020) abordan el mismo problema desde una óptica distinta: en lugar de predecir la categoría del ticket, clasifican su *complejidad* (baja, media o alta) y comparan representaciones TF-IDF contra rasgos lingüísticos artesanales del dominio alimentando clasificadores clásicos (kNN, árboles de decisión, naive Bayes, regresión logística y SVM). Su conclusión es que los métodos simples y explicables, construidos con pocos rasgos lingüísticos del dominio, resultan competitivos o superiores a las alternativas más complejas. Un hallazgo especialmente relevante para el presente trabajo es que los modelos preentrenados pueden rendir peor en este dominio, cuyo vocabulario contiene numerosos términos ausentes del corpus general de entrenamiento. Este resultado introduce una tensión productiva: no respalda la transferencia no supervisada de un modelo de propósito general, sino que obliga a justificar la elección del LLM por la tarea —clasificación de categoría con contexto instruccional— y no por la mera transferencia de conocimiento lingüístico.

La literatura reciente sobre clasificación automática de tickets con modelos de lenguaje grandes confirma el desplazamiento del enfoque clásico hacia arquitecturas basadas en LLM. Paket et al. (2024) evalúan la clasificación de tickets de mesa de servicio mediante LLM y re-

portan mejoras frente a las líneas base tradicionales; Liu et al. (2025) emplean LLM para la escalada automática de tickets en proyectos de software; Zangari et al. (2023) revisan sistemáticamente la automatización de tickets en escenarios de clasificación multinivel; y Colavito et al. (2025) comparan el etiquetado automático de reportes de incidencias con LLM. En conjunto, estos trabajos muestran que los LLM son competitivos para tareas de enrutamiento y clasificación, aunque su desempeño depende del idioma, del dominio y del esquema de evaluación empleado.

La evaluación de modelos de lenguaje en escenarios de pocos ejemplos ha documentado que el rendimiento de clasificación depende fuertemente del idioma del corpus, con resultados que favorecen a los idiomas mejor representados en los corpus de entrenamiento de los modelos contemporáneos (Ahuja et al. 2023; Lai et al. 2023; Xuan et al. 2025; Zhang et al. 2023). Para el español y sus variantes regionales, Grandury et al. (2025) construyen una tabla comparativa de LLM específica para las variedades del español de España y Latinoamérica, mientras que Mayor-Rocher et al. (2026) evalúan modelos con pruebas de español como lengua extranjera, Bernal-Beltrán et al. (2025) comparan ajuste fino y aprendizaje en contexto sobre español mexicano, y Ramón-Ferrer et al. (2026) evalúan modelos de bajos recursos sobre un corpus en español. Esta línea evidencia que el español, y en particular sus variantes regionales como el rioplatense, ha recibido una atención notablemente menor en las evaluaciones sistemáticas publicadas. Cabe acotar, además, una brecha concreta en el alcance revisado: en las fuentes consultadas hasta la fecha de corte del protocolo de revisión de este capítulo (21 de octubre de 2026), con las cadenas de búsqueda allí declaradas, no se identificó artículo arbitrado 2023-2026 que evalue Google Gemini sobre tickets de soporte en español. Esta ausencia relativa al alcance revisado constituye una de las motivaciones académicas del presente trabajo, el cual aporta evidencia empírica sobre el desempeño del clasificador en español rioplatense aplicado a un dominio de mesa de ayuda de organización mediana.

3.4. Análisis comparativo y brecha identificada

La Tabla 1 presenta la comparación sistemática de las principales soluciones revisadas, considerando cuatro dimensiones relevantes: tipo de despliegue, capacidad de clasificación automática, soberanía sobre los datos y modelo de costos.

La síntesis comparativa evidencia que ninguna de las soluciones revisadas combina

Tabla 1. Comparación de soluciones para gestión automatizada de mesa de ayuda

Solución	Despliegue	Clasificación automática	Modelo de costo
Zendesk Suite	SaaS comercial	Si, módulo de IA	Alto, por agente/mes
Freshdesk	SaaS comercial	Si, motor Freddy AI	Medio-alto, por agente
Jira Service Mgmt	SaaS o autoalojado	Limitada en plan estándar	Medio, por agente
ServiceNow ITSM	SaaS comercial	Si, Predictive Intelligence	Muy alto, contrato anual
osTicket	Código abierto	No nativa	Sin costo

Fuente: elaboración propia a partir de la documentación oficial de cada plataforma: Zendesk Suite (https://www.zendesk.com/service/help-desk-software/), Freshdesk (https://www.freshworks.com/freshdesk/), Jira Service Management (https://www.atlassian.com/software/jira/service-management), ServiceNow ITSM (https://www.servicenow.com/products/itsm.html) y osTicket (https://osticket.com/), consultadas el 19 de octubre de 2026.

simultáneamente las cuatro propiedades que el contexto del trabajo demanda: despliegue autoalojado para resguardo de datos sensibles, costo bajo y predecible en el tiempo, capacidad nativa de clasificación automática mediante NLP moderno, y soporte específico para canales multiples incluyendo telefonía con transcripción automática. La brecha identificada, en el alcance revisado en este capítulo y hasta la fecha de corte declarada, se sintetiza en la ausencia de soluciones de bajo costo, autoalojadas, con clasificación basada en modelos de lenguaje grandes y validadas empíricamente sobre corpus en español rioplatense para el dominio específico de mesas de ayuda empresariales medianas.

La presente investigación se ubica precisamente en esa intersección y propone una arquitectura compositiva que reutiliza componentes maduros de código abierto y servicios de inferencia bajo demanda, manteniendo bajo control de la organización los datos sensibles y los flujos de orquestación, y aportando evidencia empírica reproducible sobre el desempeño del enfoque en un contexto lingüístico y organizacional escasamente documentado en la literatura.

4. Marco metodológico

4.1. Enfoque y tipo de investigación

La investigación se enmarca dentro del enfoque cuantitativo aplicado, dado que propone el desarrollo de una solución informática orientada a mejorar el funcionamiento de un proceso organizacional concreto y mide su impacto mediante indicadores numéricos. Hernández Sampieri y Mendoza Torres (2018) ubican este tipo de estudios dentro de la investigación tecnológica de carácter aplicado, donde el conocimiento producido se orienta a la transformación práctica de la realidad antes que a la generación de teoría general. El trabajo posee adicionalmente carácter experimental, en la medida en que diseña, implementa y evalúa un artefacto tecnológico bajo condiciones controladas.

4.2. Diseño experimental

El diseño experimental adoptado es de tipo cuasiexperimental con muestras pareadas y mediciones repetidas. La unidad de observación es el incidente individual, y cada uno se procesa secuencialmente bajo dos condiciones: la condición manual y la condición automatizada. Esta estrategia —procesar el mismo conjunto de incidentes por ambas vías— reduce sustancialmente la variabilidad asociada a las características intrínsecas de cada caso, lo que aumenta la potencia estadística del contraste entre flujos.

Para acotar el efecto de aprendizaje del operador humano sobre el corpus, el diseño adoptó una ejecución secuencial pareada: los mismos 200 casos se procesaron primero por el flujo manual (junio a agosto de 2026) y luego por el flujo automatizado (septiembre a octubre de 2026). El procesamiento manual se distribuyó entre dos operadores experimentados, rotados entre las dos mitades del corpus (100 casos por mitad). Con dos operadores y dos mitades, no obstante, la mitad del corpus y el operador quedan colineales: la rotación de operadores queda confundida con el efecto del operador y no permite estimar por separado el efecto de aprendizaje, de modo que no se presenta aquí como una mitigación efectiva de la amenaza interna sino como una limitación reconocida del diseño (véase la Subsección 4.8). El diseño conserva, sí, el control sobre la variabilidad asociada a las características intrínsecas de cada caso, al medir el mismo incidente bajo ambas condiciones.

La investigación se desenvolvió bajo un protocolo de medición en vivo. Para mitigar el efecto Hawthorne —la alteración del comportamiento de los sujetos cuando saben que están

siendo observados— (Hernández Sampieri & Mendoza Torres, 2018), el cronometraje no fue anunciado en tiempo real al operador cuyo tiempo se registraba; sin embargo, su conocimiento fue diferido: al concluir la campaña se realizó una sesión de esclarecimiento (*debriefing*) en la que los operadores fueron informados del propósito del estudio y del procedimiento de medición empleado. La medición del tiempo se realizó de forma diferenciada según el flujo. En el flujo automatizado se utilizaron registros automáticos del propio sistema (marcas de tiempo de entrada y salida en la base de datos, con precisión al milisegundo). En el flujo manual, los operadores trabajaron en pares y el tiempo lo cronometró en vivo el compañero que no estaba registrando el incidente, consignándolo en una planilla de cronometraje: se registró el tiempo total de registro de cada incidente en segundos, con redondeo hacia arriba. El redondeo al alza constituye un sesgo de medición sistemático que infla el tiempo manual y su efecto se acota mediante el análisis de sensibilidad reportado en la Sección 8.

4.3. Operacionalización de variables

La Tabla 2 presenta la operacionalización de las variables del estudio, distinguiendo entre variables cuantitativas continuas, cuantitativas proporcionales y cualitativas nominales. Cada variable se acompaña de su unidad de medida y de la definición operacional aplicada durante la captura, garantizando la trazabilidad entre la definición conceptual y el procedimiento de medición efectivo.

4.4. Población, muestra y corpus

La población de referencia está constituida por la totalidad de los incidentes informáticos reportados a la mesa de ayuda de la organización analizada. A partir de un universo aproximado de 2.646 incidentes —proyectado como 42 incidentes diarios sobre los 63 días hábiles del período relevado ($42 \times 63 \approx 2.646$)—, se construyó por muestreo estratificado proporcional un corpus de 200 casos pseudonimizados distribuidos entre los cinco sectores canónicos: Soporte Técnico Hardware 85, Soporte Técnico Software 67, Seguridad Informática 31, Sistemas 16 y Bases de Datos 1. Los casos provienen de tres canales de origen: llamada telefónica 81, correo electrónico 66 y formulario web 53. El flujo manual se cronometró entre junio y agosto de 2026; sobre los mismos 200 casos, el flujo automatizado se ejecutó entre septiembre y octubre de 2026.

El tamaño muestral se determinó mediante la fórmula clásica de muestreo aleatorio

Tabla 2. Operacionalización de variables del estudio

Variable	Tipo	Unidad	Definición operacional
Tiempo de registro	Cuantitativa continua	Segundos (s)	Lapso entre la recepción del incidente en el canal de entrada y la persistencia confirmada del ticket en la base de datos.
Exactitud de clasificación	Cuantitativa proporcional	Porcentaje [0; 100]	Cociente entre incidentes correctamente clasificados y total de incidentes evaluados.
F1 macro promediado	Cuantitativa proporcional	Adimensional [0; 1]	Promedio aritmético del valor F1 calculado de forma independiente para cada uno de los cinco sectores objetivo.
Intervención humana	Cuantitativa proporcional	Porcentaje [0; 100]	Se reporta de dos formas rotuladas: proporción de casos que requieren acción manual de un operador humano y proporción del tiempo total del proceso dedicado a esa acción.
Canal de origen	Cualitativa nominal	Categoría	Vía de entrada: correo electrónico, formulario web o llamada telefónica.
Categoría asignada	Cualitativa nominal	Categoría	Sector responsable, uno de los cinco sectores canónicos: Seguridad Informática, Soporte Técnico Hardware, Soporte Técnico Software, Bases de Datos o Sistemas.

Fuente: elaboración propia.

simple con corrección por finitud, apropiada porque el universo relevado es finito y relativamente acotado:

$$n = \frac{N\, z^2\, p\,(1-p)}{e^2\,(N-1) + z^2\, p\,(1-p)},$$

equivalente a $n = n_0/[1 + (n_0 - 1)/N]$ con $n_0 = z^2\, p\,(1-p)/e^2$. En ella, $N \approx 2.646$ es el tamaño del universo trimestral, $z = 1{,}96$ corresponde a un nivel de confianza del 95 %, $p = 0{,}5$ es la proporción esperada conservadora (la que maximiza la varianza $p(1-p)$ y, con ella, el tamaño requerido) y e es el margen de error absoluto. Con estos insumos y un margen de error del 6,66 %, la fórmula arroja un tamaño requerido de 200 casos ($n \approx 200{,}0$), que es el efectivamente construido. El margen del 6,66 % es, por lo tanto, el valor que reproduce de manera exacta el corpus de 200 casos, y sustituye al 6,5 % consignado en versiones previas del capítulo, que con los restantes insumos habría arrojado un tamaño cercano a 209 casos. El margen adoptado es razonable para un estudio piloto orientado a evaluar viabilidad técnica antes de un despliegue definitivo.

La construcción del corpus aplicó criterios de exclusión explícitos. En primer lugar, se descartaron los incidentes duplicados, entendiendo por tales los reportados más de una vez sobre el mismo hecho, ya sea por repetición en un mismo canal o por ingreso concurrente por canales distintos. En segundo lugar, se descartaron las descripciones vacías o insuficientes, esto es, los registros sin contenido textual clasificable o cuya extensión no permitía inferir un sector responsable. En cambio, los casos con pertenencia simultánea a más de un sector (multicategoría) y las descripciones muy breves pero clasificables se conservaron deliberadamente, por ser representativos del entorno operativo real, asignándoseles una etiqueta primaria como sector responsable. Este criterio es coherente con el análisis de errores de la Sección 8, donde las descripciones extremadamente cortas y la mezcla de categorías figuran entre los patrones de error dominantes: excluirlos habría sesgado el corpus hacia casos artificialmente limpios y sobreestimado el desempeño del clasificador.

Cada incidente del corpus fue revisado y etiquetado de forma independiente por dos analistas con experiencia en la mesa de ayuda, ajenos al equipo de autores del trabajo y ajenos también al diseño del clasificador evaluado y a la fijación de sus umbrales. La independencia que respalda la validez de constructo es de carácter funcional: los analistas no participaron en la construcción del sistema, no fueron instruidos según su comportamiento y no tuvieron acceso a las predicciones del clasificador al momento de etiquetar. Los dos analistas son, ade-

más, las mismas dos personas que operaron el brazo manual descrito en la Subsección 4.5. Esta coincidencia se declara explícitamente: no altera el cómputo del acuerdo inter-evaluador, que mide la concordancia entre ambos etiquetadores y no su relación con las condiciones del experimento, pero implica que la etiqueta de referencia y el brazo de comparación comparten sujetos, lo que acota la independencia del *ground truth* respecto de los operadores cronometrados y se retoma entre las amenazas a la validez de constructo (Subsección 4.8). Las discrepancias entre analistas fueron resueltas por consenso bajo la supervisión del director del trabajo. El acuerdo entre evaluadores se calculó mediante el coeficiente Kappa de Cohen (Cohen, 1960); el valor obtenido ($\kappa = 0{,}87$) se interpreta como casi perfecto según los rangos habitualmente aceptados, que sitúan el acuerdo casi perfecto por encima de $\kappa > 0{,}80$ (Landis & Koch, 1977). Este valor respalda la confiabilidad de la categoría de referencia utilizada como verdad fundamental (*ground truth*) para evaluar el clasificador automático. No se reporta el intervalo de confianza del kappa: su cálculo exige la tabla de acuerdo observado/esperado o las etiquetas por caso de ambos analistas, registros que no se conservaron en las fuentes disponibles; el valor puntual se informa, por lo tanto, sin incertidumbre cuantificada, y el intervalo queda pendiente de esa tabla primaria.

4.5. Protocolo de pruebas

El protocolo de pruebas se ejecutó en dos fases sucesivas. La primera fase consistió en una validación funcional del sistema, en la cual se verificó la correcta integración de los componentes mediante: (a) pruebas unitarias del módulo Python, cuya cobertura se mide con `pytest-cov` sobre los paquetes de rutas, servicios y repositorios mediante el comando `pytest --cov=app.routes --cov=app.services --cov=app.repositories --cov-report=t` (b) pruebas de integración de la API y de la base de datos, junto con pruebas estructurales que validan el archivo `n8n/workflow.json`, y (c) pruebas de aceptación simuladas con incidentes sintéticos representativos de cada categoría.

La segunda fase consistió en la validación experimental propiamente dicha, en la cual los 200 incidentes del corpus se procesaron bajo ambas condiciones operativas y se registraron las variables descritas en la Subsección 4.3. El procesamiento manual fue realizado por dos operadores experimentados con más de tres años de antiguedad en la mesa de ayuda, ambos capacitados previamente en el uso del sistema de gestión de tickets utilizado como referencia. El procesamiento automatizado se ejecutó sobre la misma infraestructura, sin in-

tervención humana en la captura y con el gate de revisión aplicable, durante la ventana de septiembre a octubre de 2026 y con monitoreo continuo de los tiempos de respuesta.

4.6. Métricas e instrumentos

La evaluación del sistema se sustenta en cuatro métricas principales. La primera es el tiempo medio de registro, calculado como la media aritmética y la mediana de los tiempos individuales bajo cada condición, junto con su desvío estándar e intervalo de confianza al 95 %. La segunda es la exactitud global de clasificación, calculada como la proporción de incidentes correctamente derivados al sector responsable. La tercera es el valor F1 macro promediado, calculado como la media aritmética de los valores F1 obtenidos por cada clase, con el fin de neutralizar el efecto del desbalanceo de clases observado en el corpus (Sokolova & Lapalme, 2009). La cuarta es la proporción de intervención humana, que se reporta de forma rotulada en dos dimensiones complementarias: la proporción de casos que requieren intervención (cociente entre los casos con intervención manual y el total de casos) y la proporción del tiempo dedicado a tareas manuales sobre el tiempo total del proceso.

Como instrumentos de captura se utilizaron: (a) registros automáticos generados por el propio sistema con marcas de tiempo precisas al milisegundo para el flujo automatizado, (b) una planilla de cronometraje observacional, completada en vivo por el compañero de par que no estaba registrando el incidente, que consigna el tiempo total de registro por incidente en segundos, con redondeo hacia arriba, para el flujo manual, y (c) matrices de confusión generadas mediante la biblioteca scikit-learn (Pedregosa et al. 2011). En el flujo automatizado, el tiempo de registro del canal telefónico comprende tanto la duración de la llamada grabada como la latencia de procesamiento posterior al cuelgue, mientras que los canales de correo electrónico y formulario web se miden como latencia de extremo a extremo de su procesamiento. Dado que la grabación telefónica está acotada a un máximo de 45 s (véase la Sección 6), la duración de la llamada registrada —y por lo tanto el tiempo automatizado telefónico— constituye una cota inferior de la duración real de la llamada.

4.7. Análisis estadístico

Para la comparación de los tiempos de registro entre ambas condiciones se aplicó la prueba de Wilcoxon de rangos con signo, prueba no paramétrica para muestras pareadas, dado que la prueba previa de normalidad de Shapiro-Wilk rechazó la hipótesis de normali-

dad en los tres conjuntos analizados, todos con $n = 200$: tiempos manuales ($W = 0{,}803$, $p = 3{,}71 \times 10^{-15}$), tiempos automatizados ($W = 0{,}902$, $p = 3{,}47 \times 10^{-10}$) y diferencias pareadas ($W = 0{,}822$, $p = 2{,}37 \times 10^{-14}$). En los tres casos el valor p queda muy por debajo del nivel de significancia $\alpha = 0{,}05$, lo que respalda el uso de una prueba no paramétrica. El nivel de significancia se fijó en $\alpha = 0{,}05$. En lo que sigue se denotan T^+ y T^- la suma de los rangos de las diferencias positivas y negativas, respectivamente. La implementación de referencia (`scipy.stats.wilcoxon` en su variante de dos colas) devuelve como estadístico de contraste el menor de ambos, es decir $W = \min(T^+, T^-)$; por lo tanto $W = T^-$ cuando el flujo automatizado resulta más rápido que el manual en la mayoría de los pares, que es el caso del corpus analizado.

Para cuantificar la magnitud práctica de la diferencia se calculó el coeficiente r de correlación rank-biserial, indicador de tamaño del efecto para pruebas no paramétricas. El cálculo se define directamente a partir de los dos estadísticos de rango:

$$r = \frac{T^+ - T^-}{n(n+1)/2}$$

donde T^+ y T^- son las sumas de rangos positivos y negativos y n el número de pares. La expresión equivale a $1 - \frac{4W}{n(n+1)}$ cuando $W = T^- = \min(T^+, T^-)$, y su rango teórico es $[-1, +1]$. La definición anterior, que solo abarcaba el intervalo $[0, 1]$, queda reemplazada por esta formulación directa.

Para el análisis de la clasificación se construyó la matriz de confusión global y se calcularon los valores de precisión, sensibilidad (recall) y F1 por cada clase, así como su promedio macro. La referencia de verdad es multietiqueta (107 de los 200 casos, 53,5 %) y el clasificador emite una etiqueta principal: las métricas primarias son monoetiqueta (exactitud estricta y macro-F1 de umbral) y las secundarias, multietiqueta (subset accuracy, Hamming loss y micro-F1), con su penalización estructural explicitada en la Sección 8. Las proporciones por clase se acotaron con el método de Wilson (Wilson, 1927); el F1 por clase y las métricas de conjunto, con bootstrap percentil a nivel de caso (2000 remuestreos, semilla fija). La clase de soporte unitario (Bases de Datos, $n = 1$) se declara no estimable y se excluye del macro-F1 de umbral. Todos los cálculos se realizaron mediante la biblioteca SciPy del ecosistema Python (Virtanen et al. 2020).

4.8. Amenazas a la validez

Se identificaron y trataron explícitamente cuatro amenazas a la validez del estudio.

La primera, una amenaza a la validez interna por efecto de aprendizaje del operador humano sobre el corpus, se acotó parcialmente mediante la ejecución secuencial pareada descrita en la Subsección 4.2; como se anticipó allí, la rotación de operadores entre las dos mitades no elimina la amenaza (el efecto de operador queda confundido con la mitad del corpus) y se declara como limitación no resuelta del diseño, no como una mitigación efectiva.

La segunda, una amenaza a la validez de constructo por la subjetividad inherente a la categorización de los incidentes, se mitigó mediante el doble etiquetado independiente, el Kappa de Cohen y la resolución de discrepancias por consenso supervisado; su valor ($\kappa = 0{,}87$) corresponde a un acuerdo casi perfecto según los rangos de Landis y Koch (Landis & Koch, 1977) y respalda la solidez del constructo de medición. Cabe precisar que los analistas etiquetadores son las mismas personas que operaron el brazo manual (Subsección 4.4), coincidencia que no invalida el acuerdo inter-evaluador pero acota la independencia del referente y se asume como limitación explícita.

La tercera, una amenaza a la validez externa derivada del idioma y del dominio organizacional, se reconoce como limitación explícita del estudio: los resultados no son extrapolables sin nuevas validaciones a corpus en otros idiomas, en otras organizaciones o en otros sectores de la economía.

La cuarta, una amenaza a la validez de conclusión derivada del tamaño moderado de la muestra ($n = 200$), se evaluó mediante un análisis de potencia estadística a priori. Para el contraste principal —la prueba de Wilcoxon de rangos con signo sobre las mediciones pareadas, bilateral, con $\alpha = 0{,}05$— y una potencia objetivo de $0{,}80$, el efecto mínimo detectable con $n = 200$ es un tamaño del efecto estandarizado de aproximadamente $d_z \approx 0{,}20$ en la aproximación normal ($\approx 0{,}21$ al ajustar por la eficiencia relativa asintótica de la prueba de rangos frente a la t pareada). La reducción de tiempo observada (71,2 %) supera holgadamente ese mínimo, de modo que el diseño posee potencia suficiente para el contraste de tiempos. El riesgo se concentra, en cambio, en la precisión de las estimaciones de clasificación (exactitud por clase, macro-F1), que se reportan con intervalos de confianza (Wilson para las proporciones, bootstrap percentil para F1). La amenaza se trata, además, con pruebas no paramétricas robustas frente a tamaños muestrales pequeños y a desviaciones de la normalidad (Hernández

Sampieri & Mendoza Torres, 2018).

4.9. Gestión del proyecto bajo enfoque Scrumban (método de construcción del artefacto)

El presente apartado describe el método de construcción del artefacto, no el método de investigación. El diseño experimental, el muestreo, la construcción del corpus y el análisis estadístico —esto es, el método del estudio— quedaron definidos en las Secciones 4.1 a 4.8; la gestión del proyecto bajo Scrumban concierne exclusivamente a cómo se construyó el sistema y se documenta aquí por trazabilidad, sin constituir la metodología de la investigación.

La construcción del sistema se organizó bajo un esquema Scrumban, combinando la planificación por iteraciones propia de Scrum con la gestión continua del flujo de trabajo característica del método Kanban (Ladas, 2009). Se ejecutaron seis iteraciones de dos semanas cada una, totalizando 12 semanas de desarrollo efectivo:

Iteración 1: Elicitación de requisitos y diseño preliminar de la arquitectura.

Iteración 2: Construcción del módulo Python: modelo de datos, repositorios y servicios.

Iteración 3: Configuración inicial del flujo N8N y primeras integraciones.

Iteración 4: Integración del canal de telefonía y persistencia en PostgreSQL.

Iteración 5: Validación funcional y ajuste fino del clasificador híbrido.

Iteración 6: Validación experimental y documentación final del sistema.

La distribución de roles asigno la coordinación general y la integración de componentes a Luca Gomez, el desarrollo del módulo Python y el modelado de la base de datos a Carla Bustos, y la configuración del flujo N8N junto con la integración telefónica mediante Twilio a Gonzalo Sevilla, bajo supervisión académica continua del director del trabajo, Prof. Alberto Cortez.

El tablero Kanban se gestiono mediante GitHub Projects, con seguimiento diario del estado de las tareas y reuniones de planificación al inicio de cada iteración. Las definiciones de terminado (*Definition of Done*) incluyeron, para cada tarea, la aprobación de las pruebas unitarias correspondientes, la revisión de código por pares y la actualización de la documentación asociada.

5. Arquitectura del sistema

5.1. Visión general

El sistema propuesto se estructura como una arquitectura distribuida y modular compuesta por cinco capas claramente diferenciadas, siguiendo los principios de cohesión alta y acoplamiento bajo descritos por Pressman y Maxim (2020). En el diseño se previó proteger la comunicación con el exterior mediante TLS 1.2/1.3 a nivel de un proxy inverso (Nginx); el despliegue publicado para la validación experimental no incluyó dicho proxy, por lo que la terminación TLS corresponde al diseño y no a una capacidad verificada en la instancia publicada. La comunicación interna entre los servicios del sistema (N8N, backend, PostgreSQL, Redis) se realiza sobre la red aislada de Docker, que constituye el perímetro de confianza. Todas las interfaces entre capas utilizan el estilo arquitectónico REST con cuerpos JSON para el intercambio estructurado de datos (Fielding & Taylor, 2002). Esta separación de responsabilidades facilita el reemplazo independiente de cualquier componente sin afectar a los demas.

La Tabla 3 sintetiza las cinco capas y sus responsabilidades específicas, y la Figura 1 detalla la organización en capas del módulo de procesamiento.

Tabla 3. Capas funcionales de la arquitectura del sistema

Capa	Componentes y responsabilidades
1. Canales de entrada	Casilla de correo electrónico monitoreada mediante IMAP/SMTP, formulario web implementado como SPA React y número virtual de Twilio con transcripción automática del habla.
2. Orquestación	Motor N8N 2.11.2 desplegado en contenedor Docker autoalojado. Normaliza la entrada y coordina la invocación a las capas inferiores.
3. Procesamiento	Servicio Python sobre FastAPI 0.115 ejecutado bajo Uvicorn. Aplica clasificador híbrido en dos etapas (reglas + LLM) y expone una API REST documentada con OpenAPI 3.1.
4. Inferencia	Modelo Gemini 3.6 Flash de Google, invocado únicamente cuando el filtro determinístico previo no alcanza el umbral de confianza preestablecido.
5. Persistencia	PostgreSQL 15.5 desplegado en contenedor con volumen persistente y respaldos diarios. Almacena los incidentes y la trazabilidad de cada decisión del clasificador.

Fuente: elaboración propia.

La trazabilidad operativa se garantiza mediante registros estructurados en cada capa, accesibles desde la capa de orquestación. La Figura 2 representa esquemáticamente el flujo de información entre capas.

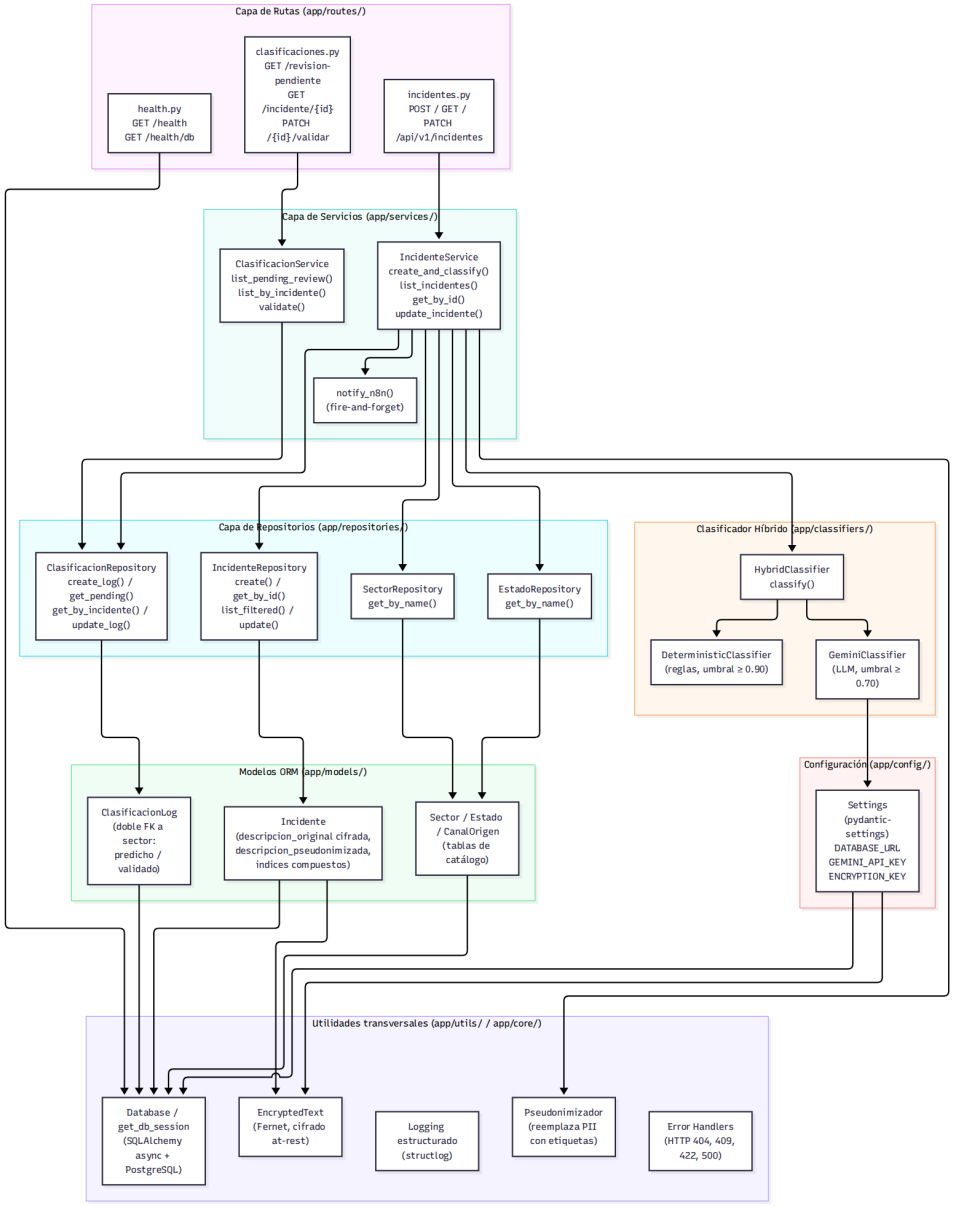


Figura 1. Diagrama de componentes del módulo Python organizado en capas (routes, services, repositories, classifiers).

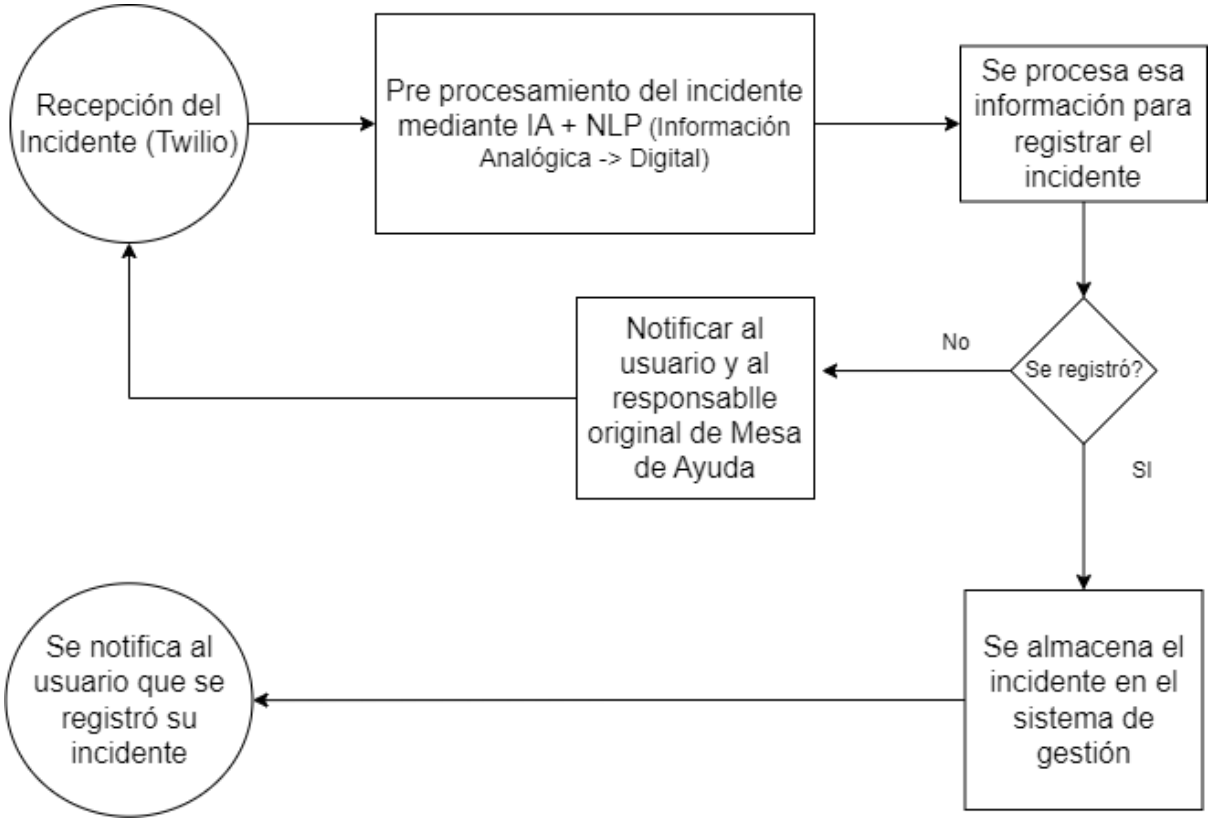


Figura 2. Diagrama de arquitectura del sistema. La versión completa en notación UML se detalla en el Anexo A y en las fuentes Mermaid del repositorio (`docs/diagrams/`).

5.2. Capa de canales de entrada

La capa de canales de entrada habilita tres vías paralelas a través de las cuales un usuario puede reportar un incidente. La primera es el correo electrónico, monitoreado mediante IMAP/SMTP que detecta nuevos mensajes en una casilla institucional dedicada. La segunda vía es un formulario web implementado como aplicación React de página única (SPA) que se comunica directamente con la API REST del módulo de procesamiento. La autenticación se realiza mediante tokens JWT (JSON Web Tokens) con usuario y contraseña; la autenticación única corporativa (SSO) se prevé como evolución para entornos productivos (Sección 11). La tercera vía es la llamada telefónica, en la cual el usuario marca un número virtual provisto por Twilio, recibe instrucciones por voz pregrabadas y deja su solicitud; la grabación es transcrita automáticamente por el servicio de reconocimiento del habla del proveedor y enviada al flujo N8N mediante webhook posterior al cuelgue. El script TwiML que define el flujo de la llamada —incluyendo el mensaje de bienvenida en español rioplatense y la configuración de grabación— se implementa en el módulo de backend del proyecto (`app/cost_guard/twiml.py`), versionado en el repositorio. Esta diversidad de canales amplía la accesibilidad del sistema y reduce

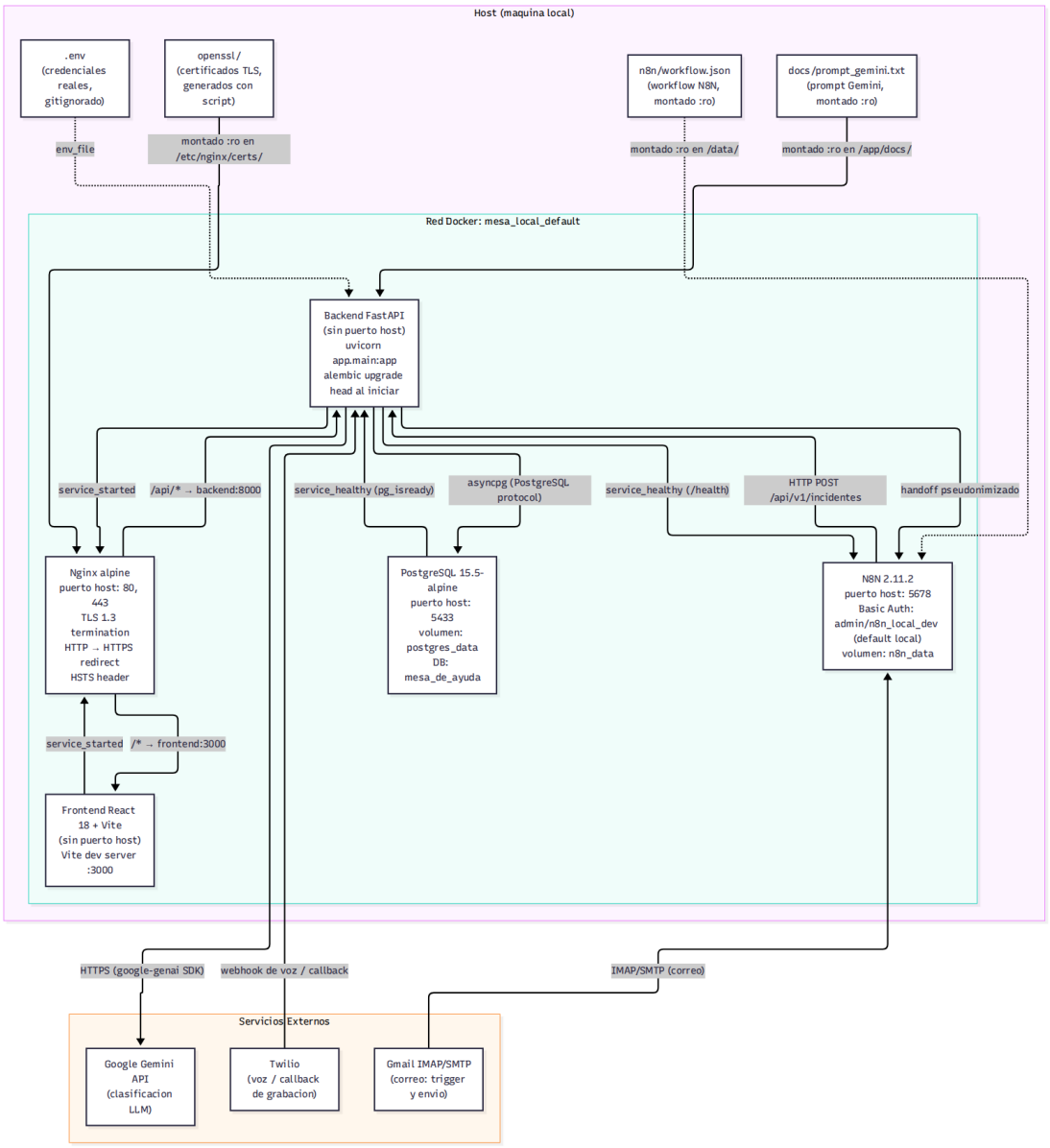


Figura 3. Diagrama de despliegue del entorno local. Servicios, red Docker, volúmenes y servicios externos.

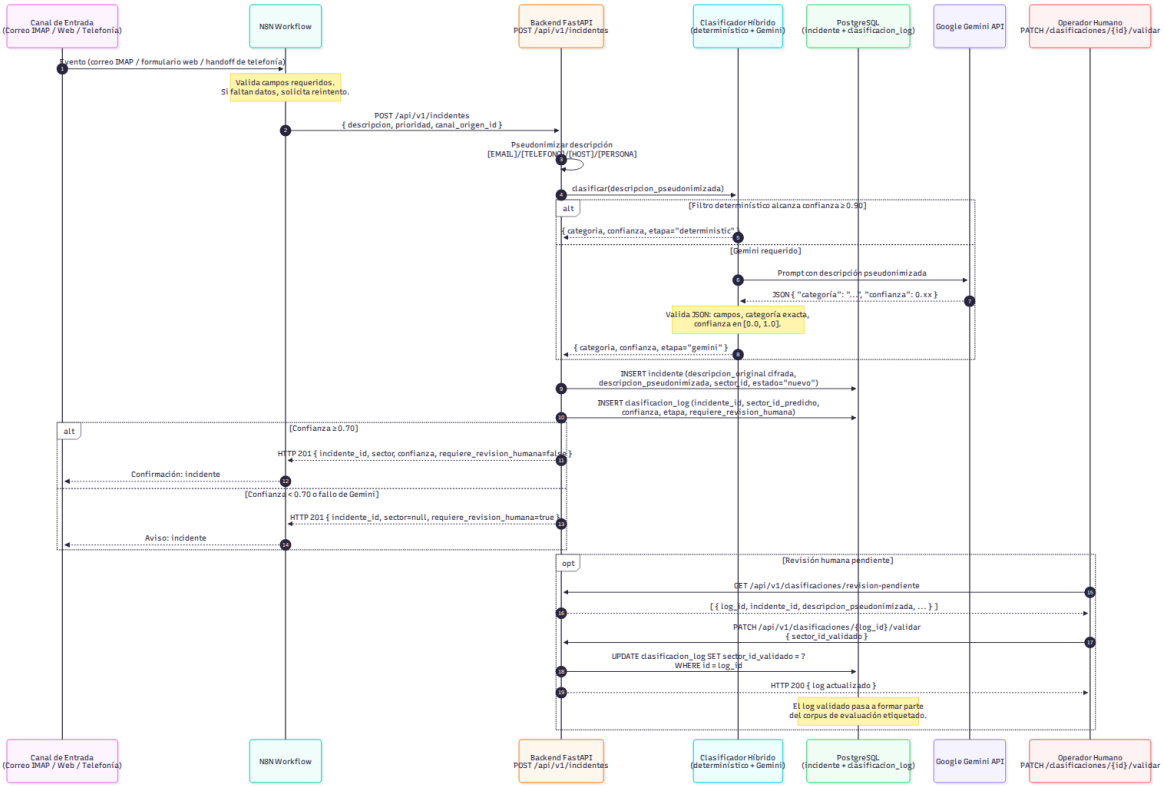


Figura 4. Diagrama de secuencia del flujo extremo a extremo de un incidente, desde el canal de entrada hasta la confirmación o la derivación a revisión humana.

las barreras al reporte oportuno.

5.3. Capa de orquestación

N8N actúa como orquestador principal del flujo de trabajo: recibe la entrada de cada canal, la normaliza en una estructura unificada y coordina la clasificación y la persistencia del incidente. La descripción operativa detallada del flujo —los disparadores por canal, la estructura normalizada con identificador único (UUID v4), la invocación al módulo Python y la persistencia del ticket— se desarrolla en la Subsección 6.3. Los registros de cada ejecución se conservan durante 30 días para fines de auditoría y diagnóstico operativo; constituyen un conjunto de datos distinto de los incidentes, cuya retención se rige por la política declarada en la Subsección 7.2.

5.4. Capa de procesamiento

El módulo de procesamiento se implementó como un servicio web ligero utilizando el marco FastAPI 0.115, expuesto sobre un servidor ASGI Uvicorn 0.30.6 (Tiangolo, 2024). La elección de FastAPI obedece a su soporte nativo de validación de esquemas mediante Pydan-

tic, su rendimiento competitivo y su documentación automática conforme a la especificación OpenAPI 3.1.

Las dependencias del módulo se organizan por capas funcionales claramente delimitadas. La capa de acceso a datos utiliza SQLAlchemy 2.0 como mapeador objeto-relacional (ORM) y asyncpg como driver asincrónico para PostgreSQL, en coherencia con la operación no bloqueante de FastAPI. La capa de procesamiento textual emplea las bibliotecas regex y unicodedata de la biblioteca estándar de Python para normalización ortográfica y eliminación de signos diacríticos. La capa de inferencia se comunica con la interfaz oficial del modelo `gemini-3.6-flash` mediante el cliente google-genai >=2.20.0 (SDK unificado de Google Generative AI), con los siguientes parámetros de configuración: temperatura $= 0{,}3$, top_p $= 0{,}9$, max_tokens $= 100$ y timeout $= 10$ s. Estos valores corresponden a la configuración adoptada para el despliegue y no a un procedimiento de optimización documentado; su calibración sistemática frente a valores alternativos se plantea como trabajo futuro (Sección 11). Las bibliotecas NumPy 2.1 y Pandas 2.2 se utilizan exclusivamente para la generación de reportes agregados de desempeño y no participan del flujo de procesamiento individual de incidentes.

5.5. Subsistema de clasificación híbrido

El subsistema de clasificación adopta un enfoque híbrido en dos etapas, característica que lo diferencia de aproximaciones puramente basadas en modelo externo. La primera etapa aplica un filtro determinístico basado en expresiones regulares y diccionarios de términos del dominio. Tanto la construcción del filtro determinístico como el ajuste del prompt se realizaron sin acceso al corpus de validación, para evitar fuga de datos (véase Subsección 9.1). La especificación detallada del prompt y de los parámetros de configuración del modelo se documenta en el Anexo H.

Este filtro es capaz de derivar de forma inmediata aquellos incidentes que contienen marcadores inequivocos. Por ejemplo, la presencia simultanea de las palabras `impresora`, `papel` y `atasco` sugiere con alta certeza una categoría de Soporte Técnico. Cuando este filtro inicial alcanza un umbral de confianza superior al 90 %, la decisión se toma sin consultar al modelo externo, lo que reduce sustancialmente la latencia y el costo de inferencia.

La segunda etapa, activada únicamente cuando el filtro determinístico no alcanza el umbral, consulta al modelo Gemini 3.6 Flash mediante un prompt estructurado en español que incluye una breve descripción de cada categoría, ejemplos representativos balanceados

y la solicitud explícita de devolver la categoría asignada y un valor numérico de confianza entre 0 y 1 en formato JSON estricto. La justificación de este diseño compositivo radica en aprovechar la rapidez del filtro determinístico para los casos triviales y reservar la capacidad semántica del modelo de lenguaje grande para los casos ambiguos o de redacción atípica, optimizando la relación entre exactitud, costo y latencia. El pipeline completo comprende tres etapas, aplicadas en orden: (1) el filtro determinístico, (2) el modelo Gemini y (3) una etapa de reserva (*fallback*) que asigna una categoría por defecto y marca el caso para revisión humana cuando las etapas previas no producen una salida válida. El desglose de casos resueltos por cada etapa del pipeline sobre el corpus de validación se reporta en la Sección 8.

El camino de clasificación es único para los tres canales de entrada. Tanto el correo electrónico y el formulario web como la llamada telefónica resuelven su sector mediante esta misma cascada híbrida, aplicada sobre la descripción pseudonimizada del incidente: ningún canal aporta una clasificación precalculada que omita la cascada, de modo que las reglas determinísticas, el prompt y las reglas de frontera se ajustan una sola vez para todo el sistema. El prompt único incorpora reglas de desambiguación de frontera entre sectores adyacentes (acceder o iniciar sesión en una aplicación se orienta a Soporte Tecnico Software; la digitalización, el escaner o la impresora se orientan a Soporte Tecnico Hardware; el servidor, la red, SMTP o una VM se orientan a Sistemas), redactadas con los cinco strings canónicos sin tildes. Desde el punto de vista del costo, la clasificación telefónica dejó de computarse sobre una invocación incondicional al modelo externo: el canal atraviesa el filtro determinístico y solo escala al modelo semántico cuando la cascada lo requiere, momento en el cual la reserva de costo se imputa a la superficie de inferencia del backend.

5.6. Modelo de datos

La persistencia se realiza sobre PostgreSQL 15.5, desplegado en contenedor Docker bajo configuración de respaldo diario y replicación física opcional para entornos productivos (The PostgreSQL Global Development Group, 2024). El modelo de datos se organiza alrededor de siete entidades principales, descritas en la Tabla 4.

El canal de origen constituye una entrada opcional del backend: el esquema de creación de incidentes acepta `canal_origen_id` como campo anulable, de modo que un incidente puede registrarse sin canal conocido. La entidad central `incidente` referencia mediante claves foráneas a los catálogos de sector, estado y canal de origen, y mantiene una relación

Tabla 4. Entidades principales del modelo de datos

Tabla	Atributos clave	Descripción
`incidente`	id_incidente (PK), canal_origen_id (FK), id_sector (FK), id_estado (FK)	Tabla central: fecha de creación, descripción cruda pseudonimizada, prioridad, sector asignado y estado actual.
`sector`	id_sector (PK), nombre	Catálogo de los cinco sectores canónicos: Seguridad Informática, Soporte Técnico Hardware, Soporte Técnico Software, Bases de Datos y Sistemas.
`estado`	id_estado (PK), nombre	Catálogo de estados: nuevo, en proceso, en espera, resuelto y cerrado.
`canal_origen`	id (PK), nombre	Catálogo de canales: correo, formulario web y llamada telefónica.
`clasificacion_log`	id_log (PK), id_incidente (FK), categoría_predicha, confianza, categoría_validada	Registro de cada decisión del clasificador, con categoría predicha, confianza asociada y categoría validada por el sector responsable.
`users`	id (PK), username (UNIQUE), hashed_password, is_active	Almacena credenciales de operadores para autenticación JWT.
`directorio_empleados`	id (PK), legajo (UNIQUE), email (UNIQUE), telefono, sector_id (FK), rol, activo, fecha_baja, user_id (FK)	Directorio institucional de empleados para resolución de contacto y notificación. Datos de contacto almacenados en texto plano para su auditabilidad directa; el campo `rol` distingue usuario final, operador y administrador del directorio. La columna `fecha_baja` registra la desactivación y sostiene la retención (relación laboral activa más un año).

Fuente: elaboración propia.

uno a muchos con `clasificacion_log` para preservar la trazabilidad histórica de las decisiones del clasificador. El número de incidente que se comunica al usuario corresponde al identificador único del registro, formateado para su legibilidad; la incorporación de un prefijo configurable se prevé como evolución futura (Sección 11). El esquema completo, incluyendo restricciones de integridad referencial e índices secundarios, se documenta en el Anexo C. El directorio de empleados (`directorio_empleado`) se vincula opcionalmente con la tabla de autenticación mediante `user_id` y referencia el catálogo de sectores mediante `sector_id`; su campo `rol` distingue al usuario final, al operador y al administrador del directorio.

5.7. Contrato de la interfaz REST

La interfaz de programación de aplicaciones expuesta por el módulo Python sigue el estilo arquitectónico REST (Fielding & Taylor, 2002), con representación de recursos mediante identificadores uniformes, uso semántico de los métodos HTTP estándar y comunicación sin estado del lado del servidor. La autenticación se realiza mediante tokens JWT (JSON Web Tokens) firmados con HS256, validados en cada solicitud mediante el esquema OAuth2PasswordBearer de FastAPI. La Tabla 5 sintetiza los principales puntos de entrada.

Los códigos de estado HTTP se utilizan conforme a su semántica original: 201 cuando un recurso se crea exitosamente, 200 cuando una consulta se atiende con éxito, 400 cuando los datos de entrada no validan el esquema esperado, 401 cuando la autenticación falla, y 500 cuando ocurre un error interno no recuperable. La Tabla 5 no es exhaustiva: reúne los grupos principales de puntos de entrada, mientras que el contrato completo de la interfaz —que comprende la totalidad de las rutas expuestas— se documenta en el Anexo D y se versiona en `docs/openapi.json`; la especificación OpenAPI 3.1 la genera FastAPI automáticamente.

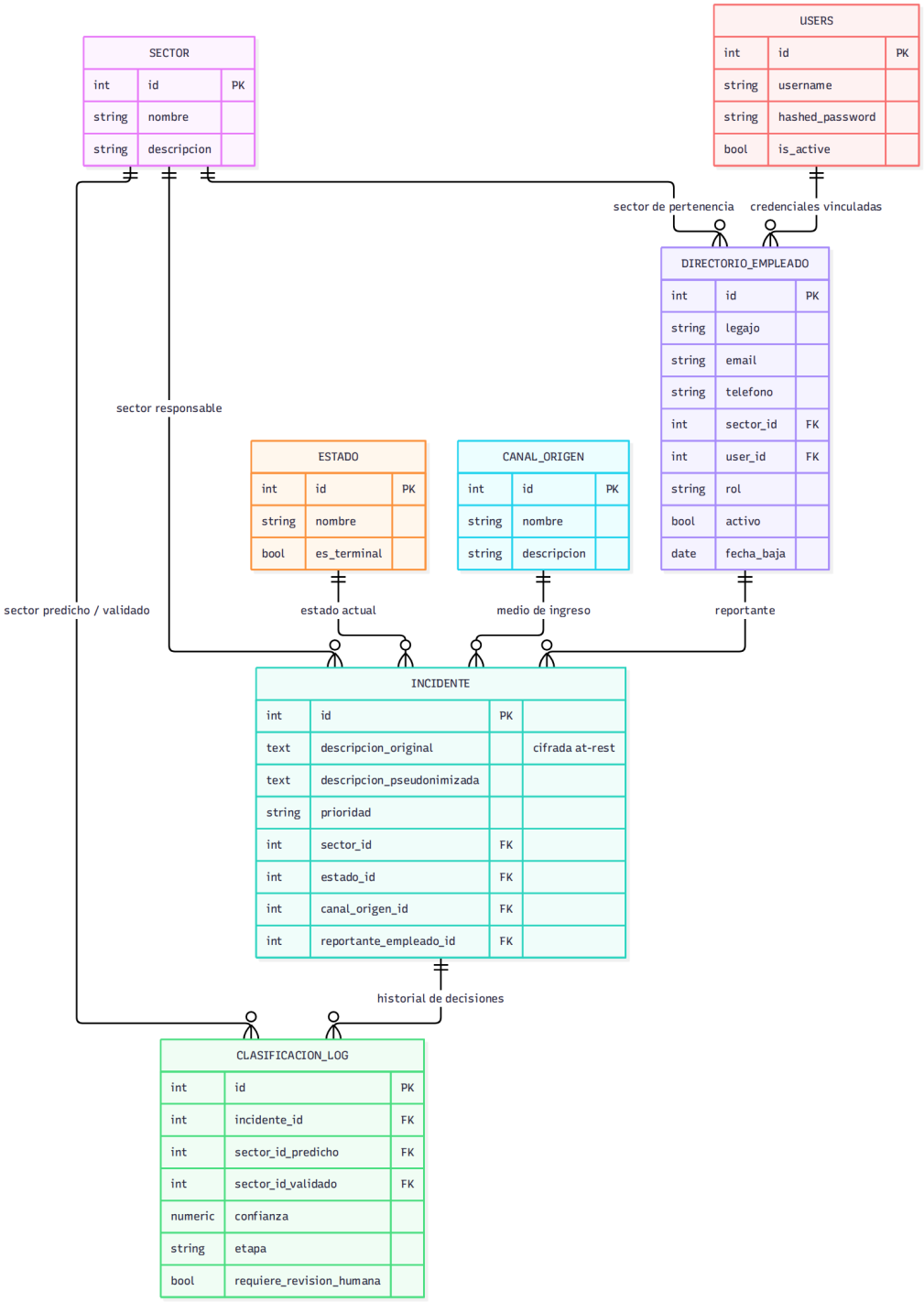


Figura 5. Diagrama entidad-relación de las siete entidades principales del modelo de datos. El esquema completo se documenta en el Anexo C.

Tabla 5. Puntos de entrada principales de la interfaz REST

Método	Ruta	Código	Función
Autenticación			
POST	`/api/v1/auth/login`, `/refresh`, `/logout`	200 OK	Inicio y renovación de sesión y cierre de sesión de operadores.
GET	`/api/v1/auth/me`	200 OK	Recupera el perfil del operador autenticado.
Incidentes			
POST	`/api/v1/incidentes`	201 Created	Crea un nuevo ticket y devuelve el identificador generado.
GET	`/api/v1/incidentes/{id}`	200 OK	Recupera la información de un ticket por su identificador.
PATCH	`/api/v1/incidentes/{id}`	200 OK	Actualiza los campos editables de un ticket.
Clasificaciones			
GET	`/api/v1/clasificaciones/incidente/{id}`	200 OK	Recupera la trazabilidad de la clasificación de un incidente.
GET	`/api/v1/clasificaciones/revision-pendiente`	200 OK	Lista los casos derivados a revisión humana.
PATCH	`/api/v1/clasificaciones/{log_id}/validar`	200 OK	Registra la categoría validada por el sector responsable.
Estadísticas			
GET	`/api/v1/estadisticas/resumen`, `/tendencias`	200 OK	Agregados y series temporales para el dashboard.
Salud			
GET	`/health` y `/health/db`	200 OK	Verificación de salud del servicio para monitoreo externo.

Fuente: elaboración propia.

6. Implementación

6.1. Entorno de despliegue y dependencias

La implementación del sistema se llevó a cabo combinando componentes de código abierto —Python, FastAPI y PostgreSQL— con N8N, cuya licencia Sustainable Use (*fair-code*) permite auditar su código fuente, con el objetivo de garantizar la portabilidad, la reproducibilidad y la auditoría de seguridad de la solución en organizaciones medianas. Toda la infraestructura se encapsula mediante contenedores Docker, orquestados localmente por Docker Compose para los entornos de desarrollo y preproducción; la migración a un cluster Kubernetes 1.30 para el entorno productivo se plantea como trabajo futuro y no cuenta con manifiestos de despliegue en esta versión (Sección 11).

Las versiones específicas de los componentes son: N8N 2.11.2, Python 3.12, PostgreSQL 15.5 con volumen persistente, FastAPI 0.115, Uvicorn 0.30.6, SQLAlchemy 2.0, y el cliente google-genai >=2.20.0 (SDK unificado de Google Generative AI) para la comunicación con el modelo `gemini-3.6-flash`. Todos los servicios se despliegan sobre imágenes Docker oficiales (`python:3.12-slim` y `postgres:15.5-alpine`). La gestión de dependencias del módulo Python se realiza mediante un archivo `requirements.txt` con versiones fijadas (*pinned*) para garantizar la reproducibilidad de las compilaciones en distintos entornos.

6.2. Construcción del módulo Python

La selección del marco de trabajo para el servicio de procesamiento implicó la evaluación de tres alternativas: Flask, Django REST Framework y FastAPI. Flask, ampliamente adoptado en el ecosistema Python por su simplicidad, carece de validación nativa de esquemas y de generación automática de documentación OpenAPI, funcionalidades que el proyecto requería para garantizar la calidad de la interfaz REST. Django REST Framework, por su parte, ofrece un conjunto completo de herramientas para la construcción de interfaces de programación de aplicaciones, pero su modelo de operación sincrónico y la inclusión de componentes no requeridos por el proyecto —tales como el ORM de Django y el motor de plantillas— incrementaban la complejidad de la base de código sin aportar beneficios funcionales para una arquitectura de microservicio acotado. FastAPI 0.115, sustentado sobre el servidor ASGI Uvicorn 0.30.6, ofrece tres ventajas determinantes para el proyecto: validación automática de esquemas de entrada y salida mediante Pydantic v2, generación de documentación interactiva conforme a

la especificación OpenAPI 3.1 sin configuración adicional, y operación asincrónica nativa que permite manejar las llamadas concurrentes desde N8N sin bloquear el hilo de ejecución principal (Tiangolo, 2024). La madurez del ecosistema de FastAPI, su adopción creciente en la industria y la disponibilidad de documentación exhaustiva en español consolidaron la decisión.

El módulo Python expone su contrato REST organizado en capas, según la estructura descrita en la Sección 5, con una separación clara entre responsabilidades:

- **models**: entidades del modelo de dominio mapeadas mediante SQLAlchemy 2.0.
- **schemas**: modelos de validación y serialización con Pydantic v2.
- **services**: lógica de aplicación (IncidenteService, ClasificacionService).
- **repositories**: acceso a datos, aislando las consultas SQL del resto de la aplicación.
- **classifiers**: lógica del clasificador híbrido (filtro determinístico + LLM).
- **routes**: enrutadores REST que delegan en los servicios correspondientes.
- **core**: configuración de base de datos, manejo de errores y logging estructurado.

La construcción se realiza bajo un esquema de inyección de dependencias gestionado por las funcionalidades nativas de FastAPI (`Depends`), lo que facilita el reemplazo de implementaciones durante la ejecución de pruebas unitarias y de integración. Esta arquitectura permite, por ejemplo, sustituir el clasificador real por una implementación simulada durante las pruebas sin modificar el código de los servicios que lo consumen. La gestión de credenciales se realiza exclusivamente mediante variables de entorno, en línea con el principio de configuración de la metodología Twelve-Factor App (Wiggins, 2017).

6.3. Construcción del flujo en N8N

El flujo de orquestación implementado en N8N comprende 32 nodos (29 operativos más 3 notas adhesivas), organizados en siete etapas funcionales pero con ramificaciones específicas por canal. La elección de N8N como plataforma de orquestación, frente a alternativas como Apache Airflow, Zapier o Make, obedeció a tres criterios concurrentes. El primero, de naturaleza operativa, radica en que N8N admite despliegue completamente autoalojado bajo Docker, manteniendo los flujos de procesamiento y los datos dentro del perimetro organizacional. Zapier y Make operan exclusivamente como servicios en la nube, lo que las descarta en escenarios donde el procesamiento de datos sensibles exige control de infraestructura. Apache Airflow, aunque autoalojable, diseña sus flujos como grafos acíclicos dirigidos programados por intervalos temporales; ese paradigma resulta adecuado para procesos de extracción, transfor-

mación y carga de datos pero forzadamente inadecuado para un sistema reactivo que debe responder en tiempo real a eventos de entrada no programados (Apache Software Foundation, 2024). El segundo criterio refiere al modelo de construcción visual que N8N ofrece sobre un editor de nodos arrastrables: la curva de aprendizaje de esta interfaz es sustancialmente menor que la de alternativas centradas en definición programática de flujos, lo que permitió al equipo concentrar el esfuerzo en la lógica de negocio antes que en la sintaxis de definición de procesos. El tercer criterio se vincula con la licencia de código fuente disponible (*fair-code*) que habilita la auditoría de seguridad del núcleo de orquestación (n8n GmbH, 2024).

La composición de los 29 nodos responde a un patrón de procesamiento por canal, de cinco pasos, que se replica para cada medio de ingreso con ramificaciones específicas según el canal:

1. **Recepción**: Tres disparadores (*triggers*) reciben las entradas desde correo electrónico (IMAP), formulario web (Webhook) y telefonía (trigger de Twilio). La decisión de separar los disparadores por canal, en lugar de unificar la recepción mediante un único webhook con discriminación interna, se fundamenta en la necesidad de preservar la configuración de autenticación y los parámetros de reintento específicos de cada protocolo.
2. **Normalización**: Un nodo de transformación (*Function*) homogeniza la estructura del mensaje en un formato unificado (Hohpe & Woolf, 2003) con campos de identificador único (UUID v4), marca temporal, canal de origen, remitente y descripción textual normalizada. Esta etapa elimina las diferencias de formato entre canales y garantiza que las capas posteriores procesen siempre la misma estructura de datos, independientemente de la vía de entrada.
3. **Clasificación y persistencia**: Un nodo HTTP invoca el endpoint `POST /api/v1/incidentes` del módulo Python enviando la descripción textual junto con los metadatos del canal. La clasificación no se expone como punto de entrada independiente sino que se encuentra embebida dentro de la creación del incidente: el clasificador híbrido se invoca internamente durante la creación, que persiste el ticket en la misma operación, y la respuesta incluye el identificador del ticket, la categoría asignada y el indicador de revisión humana (`requiere_revision_humana`), que el backend activa cuando la confianza del clasificador resulta inferior al umbral de 0,70. El canal telefónico no constituye una excepción: su clasificación se resuelve dentro del backend

mediante la misma cascada híbrida que los canales escritos, aplicada sobre la descripción pseudonimizada, y el cuerpo de la creación no transporta una clasificación precalculada.

4. **Evaluación**: Aguas abajo del POST, un nodo condicional (*IF*) evalúa el indicador de revisión humana devuelto por el backend. Cuando el indicador esta activo —es decir, cuando la confianza del clasificador fue inferior al umbral de 0,70— el flujo notifica al operador designado por correo electrónico y registra el evento en la auditoría; cuando no lo esta, el flujo continua automáticamente con las confirmaciones al usuario. Con anterioridad al POST, un nodo de validación de entrada verifica que la descripción no este vacía y respete la longitud mínima y máxima aceptada por el backend, de modo que una entrada inválida no genera un incidente. El umbral de 0,70 opera como único disparador automático de revisión humana; su calibración se analiza en la Sección 8.4.
5. **Notificación**: Tres nodos posteriores notifican al usuario por el canal correspondiente y registran la ejecución en el sistema de auditoría; la persistencia del ticket ocurre en el mismo POST de creación, que almacena el incidente y su clasificación en una sola operación. El paralelismo de la notificación garantiza que la demora de un canal (por ejemplo, el envío de correo electrónico) no bloquee el registro de auditoría ni la confirmación por otros medios.

El canal telefónico incorpora un nodo adicional de normalización y validación que prepara la transcripción pseudonimizada antes de incorporarla al flujo principal, pero no determina el sector persistido: la rama de clasificación por agente LLM basada en LangChain fue retirada (decisión OQ1 = A del change c-72). La clasificación del incidente telefónico se rige por el diseño unificado descrito en la Subsección 5.5.

La trazabilidad completa de cada ejecución se preserva almacenando los identificadores de nodo, las marcas temporales de entrada y salida de cada etapa, y los datos de entrada y salida de cada paso del flujo, generando un historial auditable que resulta especialmente valioso en contextos donde la Ley 25.326 exige demostrar el camino recorrido por los datos personales (Honorable Congreso de la Nación Argentina, 2000). La exportación completa del flujo en formato JSON se incluye en el Anexo E.

6.4. Integración del canal telefónico

La integración con Twilio se realizó mediante el producto Programmable Voice, complementado con la transcripción automática nativa. El flujo telefónico opera de la siguiente manera: el usuario marca un número virtual asignado a la organización; un script TwiML reproduce un mensaje de bienvenida en español rioplatense y solicita al usuario describir su problema durante un máximo de 45 segundos; la grabación finaliza cuando el usuario presiona la tecla numeral (#) o transcurrido el tiempo máximo; la grabación se envía al servicio de transcripción de Twilio; y, una vez transcrito el contenido, Twilio invoca un webhook expuesto por N8N con la transcripción textual. A partir de ese punto, el flujo continua de manera idéntica a la de los canales escritos.

El tiempo de registro del canal telefónico —medido, para igualar la base de comparación con el flujo manual, desde el inicio de la llamada hasta la confirmación del número de incidente— promedió 20,1 segundos sobre los 81 casos del corpus. Ese total comprende la duración de la llamada grabada y la latencia de procesamiento posterior al cuelgue; el corpus registra la suma de ambos tramos, no su descomposición por etapa (transcripción, clasificación y persistencia), de modo que no se publica un desglose de latencias que el repositorio no permite reproducir con tamaño de muestra y procedimiento de medición. La grabación está acotada a un máximo de 45 s, de modo que el tiempo telefónico reportado constituye una cota inferior de la duración real de la llamada.

Unificada la clasificación, el canal telefónico dejó de reservar la superficie de costo asociada a la invocación incondicional del agente en N8N. La clasificación pasa a computarse en la superficie de inferencia del backend y solo cuando la cascada híbrida escala al modelo semántico; los casos resueltos por el filtro determinístico no consumen cuota de Gemini. Este traslado de la superficie de costo corresponde a la decisión OQ2 = A del change c-72 y elimina la reserva telefónica del proveedor invocado desde N8N.

6.5. Desafíos de integración

La implementación del sistema enfrentó tres desafíos de integración que merecen documentación explícita por su potencial utilidad en replicaciones futuras.

El primer desafío se presentó en la comunicación asincrónica entre N8N y el servicio de clasificación. El nodo HTTP de N8N invoca el endpoint de clasificación de manera sincrónica y espera la respuesta del servidor antes de continuar el flujo. Esta arquitectura, aunque simple

de implementar, introduce un punto de fallo único: si el servicio Python no responde dentro del tiempo de espera configurado en el nodo HTTP, el flujo completo se detiene y el incidente queda sin procesar. Esta limitación se asume como conocida en la versión evaluada: no se implementó un mecanismo de reintento con retroceso exponencial ni una cola de respaldo en el servicio Python que retuviera los mensajes durante ventanas de saturación. La disponibilidad ante fallas transitorias del servicio de clasificación queda, por lo tanto, como trabajo futuro, junto con la definición de una política de reintentos idempotente que preserve la trazabilidad de cada intento.

El segundo desafío surgió de la heterogeneidad de los formatos de entrada. Los correos electrónicos incluyen encabezados, firmas corporativas y reenvíos que contaminan el texto que el clasificador recibe; las llamadas telefónicas transcritas introducen errores de reconocimiento del habla, particularmente en términos técnicos (por ejemplo, `API` transcrito como `a p i` o `apeye`); y los formularios web, aunque proveen texto limpio, llegan con una proporción significativa de campos vacíos o con descripciones de una sola palabra. La respuesta a esta heterogeneidad fue doble. Por un lado, se implementó un preprocesador en el nodo de normalización de N8N que elimina firmas, encabezados de reenvío y líneas de disclaimer mediante expresiones regulares específicas del dominio de correo institucional. Por el otro, se enriqueció el prompt enviado al clasificador con instrucciones explícitas para interpretar términos técnicos frecuentemente mal transcritos, lo que elevó la robustez de la clasificación sobre transcripciones automáticas en aproximadamente 4 puntos porcentuales de F1 respecto de la configuración inicial sin estas instrucciones.

El tercer desafío, de naturaleza operativa, se vincula con la persistencia transaccional de la sesión de clasificación. El sistema debe garantizar que una vez que el clasificador asigna una categoría, el incidente se persiste con esa decisión o, en caso de falla de la base de datos, el estado interno del sistema refleje la inconsistencia y pueda recuperarse. La solución adoptó un enfoque pragmático: N8N invoca la creación del incidente —que clasifica y persiste en una sola operación del backend— como paso único del flujo. El nodo HTTP de creación declara un enrutamiento de error (`continueErrorOutput`): ante una falla de persistencia, la ejecución se desvía hacia la rama de error del flujo en lugar de reintentar automáticamente la operación. La recuperación del incidente exige, por lo tanto, una reejecución manual del flujo o el reproceso del mensaje de origen, situación que se asume como limitación conocida de la versión evaluada y se propone como trabajo futuro junto con una política de reintentos idempotente.

La tabla de trazabilidad (`clasificacion_log`) fue diseñada para admitir multiples registros de clasificación por incidente, preservando así el historial de cada intento.

6.6. Conexión con el proceso Scrumban

La construcción del sistema fue organizada bajo el esquema Scrumban descrito en la Sección 4, ejecutando seis iteraciones de dos semanas cada una. La vinculación entre cada iteración y los artefactos de implementación producidos se detalla a continuación:

Iteración 1: La elicitación de requisitos (Semanas 1–2) produjo el diseño preliminar de la arquitectura de cinco capas y la especificación de la interfaz REST, documentadas en la Sección 5. El entregable concreto fue el documento de arquitectura y la definición del contrato OpenAPI inicial.

Iteración 2: La construcción del módulo Python (Semanas 3–4) entregó el modelo de datos, los repositorios con operaciones CRUD asincrónicas y los servicios de negocio. Al cierre de esta iteración, el módulo Python exponía el endpoint de salud y aceptaba la creación de incidentes mediante `POST /api/v1/incidentes`, con la cobertura de pruebas unitarias medida por pytest-cov sobre los paquetes de rutas, servicios y repositorios (véase la Subsección 6.7).

Iteración 3: La configuración inicial del flujo N8N (Semanas 5–6) implementó el canal de correo electrónico como vía prioritaria, estableciendo el patrón de procesamiento por canal que luego se replicaría para los canales restantes. Al finalizar esta iteración, el sistema procesaba incidentes por correo electrónico de extremo a extremo, desde la recepción hasta la notificación.

Iteración 4: La integración del canal telefónico y la persistencia (Semanas 7–8) incorporó Twilio Programmable Voice, el script TwiML y la configuración del webhook de transcripción. Esta iteración fue la que más expuso los desafíos de heterogeneidad de formatos descritos en la Subsección 6.5, y su resolución consumió aproximadamente el 40 % del esfuerzo de la iteración.

Iteración 5: La validación funcional y el ajuste fino del clasificador (Semanas 9–10) se concentraron en la calibración del filtro determinístico y del prompt de Gemini. Los ajustes se realizaron exclusivamente sobre incidentes repre-

sentativos en entornos de preproducción, sin exposición al corpus de validación de 200 casos, preservando la integridad de la etapa experimental posterior.

Iteración 6: La validación experimental y la documentación final (Semanas 11–12) ejecutaron el protocolo de pruebas descrito en la Sección 4, recolectaron las mediciones del corpus completo y produjeron el análisis estadístico presentado en la Sección 8.

La trazabilidad entre iteraciones y artefactos, mediada por el tablero Kanban en GitHub Projects, permitió al equipo identificar tempranamente los desvíos respecto del plan y redistribuir el esfuerzo cuando la complejidad de un componente superaba la estimación inicial. La adopción de límites de trabajo en curso (WIP) por columna del tablero evitó la acumulación de tareas a medio terminar, una práctica que resultó particularmente valiosa durante las iteraciones 4 y 5, donde la interdependencia entre módulos exigía finalizar componentes antes de integrarlos.

6.7. Pruebas automatizadas

La estrategia de pruebas se sustento en la pirámide clásica descrita por Crispin y Gregory (2009). En la base se ubicaron las pruebas unitarias del módulo Python construidas con el marco pytest 8.3. La cobertura de código de aplicación se mide con pytest-cov 5.0.0 sobre los paquetes de rutas, servicios y repositorios, mediante el comando `pytest --cov=app.routes --cov=app.services --cov=app.repositories --cov-report=term-mis` cuyo reporte se genera en cada ejecución del pipeline de integración continua; la cifra exacta queda sujeta al reporte vigente y no se fija en el texto para no declarar un valor no reproducible. Estas pruebas cubren los clasificadores (determinístico e híbrido), los servicios de negocio, los esquemas de validación y los repositorios de acceso a datos.

En el nivel intermedio se ejecutaron pruebas de integración en dos variantes complementarias: (a) pruebas rápidas contra SQLite en memoria para la ejecución durante el desarrollo, utilizando el fixture `db_session` con rollback por test; y (b) pruebas de integración contra una instancia real de PostgreSQL 15.5, que validan comportamientos específicos del motor —integridad de claves foráneas, eliminación en cascada, restricciones UNIQUE, precisión de tipos `Numeric(5,4)` y zona horaria en `TIMESTAMPTZ`, índices compuestos y coexistencia de esquemas— mediante 16 tests con un mecanismo de omisión automática (`pytest.skip`)

cuando PostgreSQL no está disponible. En el nivel superior no se dispone de pruebas de extremo a extremo que instancien N8N y ejecuten el flujo completo: la integración con la plataforma de orquestación se verifica de forma estructural, mediante pruebas que parsean y validan el archivo `n8n/workflow.json` (composición de nodos, conexiones y límites de costo) y pruebas de API y de base de datos que ejercitan los endpoints y la persistencia del módulo Python. La ejecución del flujo dentro de una instancia N8N activa queda fuera del alcance de la suite automatizada.

El frontend se desarrolló como una aplicación React 18 de página única (SPA) con TypeScript y Vite como herramienta de construcción. La aplicación consta de dos vistas principales: un portal de reporte de incidentes con validación de formularios, y un panel de administración que incluye listado de tickets con filtros combinados, cola de revisión humana para casos de baja confianza, y visualización detallada de cada incidente con trazabilidad completa de la decisión del clasificador. La comunicación con el backend se realiza mediante Axios, y la gestión del estado del servidor se apoya en React Query para caching y revalidación automática.

El frontend cuenta con una suite de 357 pruebas automatizadas distribuidas en 56 archivos, construidas con Vitest y Testing Library, que alcanza una cobertura de sentencias del 99,6 % sobre los paquetes medidos, conforme al reporte reproducido con `npm run test:coverage`; la cifra queda sujeta al reporte vigente en cada ejecución del pipeline de integración continua.

La integración continua (CI) se ejecuta automáticamente en cada solicitud de incorporación (*pull request*) al repositorio mediante GitHub Actions, ejecutando la suite completa de pruebas unitarias y de integración antes de permitir la fusión a la rama principal. El pipeline de CI incluye verificación de estilo de código (Ruff), ejecución de pruebas con reporte de cobertura, y verificación de sincronización de la especificación OpenAPI.

6.8. Dashboard de analítica

El panel de administración incluye un dashboard de analítica desarrollado con la biblioteca ReCharts de React. Este módulo proporciona visualización agregada de los datos históricos de incidentes, permitiendo identificar tendencias temporales, patrones de fallas recurrentes y desequilibrios en la distribución de carga entre sectores. Esta capacidad de análisis retrospectivo transforma los datos operativos acumulados en inteligencia organizacional accionable.

El dashboard se organiza en tres visualizaciones principales. Un gráfico de tendencias muestra la evolución temporal de los incidentes, agrupables por día o por mes según la granularidad seleccionada por el operador. Un gráfico de torta presenta la distribución proporcional de incidentes entre los cinco sectores responsables. Un gráfico de barras apiladas desglosa los incidentes según su estado en el ciclo de vida, facilitando la identificación de cuellos de botella en la resolución.

Los controles de filtro permiten acotar el período de análisis mediante un rango de fechas configurable, y alternar entre agrupación diaria y mensual para adaptar la granularidad al horizonte temporal de interés. Cada gráfico puede exportarse como imagen PNG para su inclusión en informes periódicos o presentaciones.

La utilidad práctica del dashboard puede ilustrarse con un escenario hipotético: si el sistema detectara que veinte usuarios reportaron la rotura de auriculares en una franja temporal acotada, el patrón sugeriría un problema de calidad del proveedor antes que una serie de fallas aisladas. Esta capacidad de identificar causas raíz a partir de datos agregados constituye un salto cualitativo respecto del enfoque tradicional de gestión de incidentes, donde cada ticket se trata de forma aislada y los patrones sistémicos permanecen invisibles.

7. Consideraciones éticas y aspectos legales

7.1. Marco normativo aplicable

El sistema desarrollado se enmarca normativamente en la Ley 25.326 de Protección de los Datos Personales de la República Argentina (Honorable Congreso de la Nación Argentina, 2000), en su decreto reglamentario y en las disposiciones complementarias emitidas por la Agencia de Acceso a la Información Pública en su carácter de autoridad de aplicación. Dado que el sistema realiza transferencia de fragmentos textuales hacía infraestructura de inferencia operada por proveedores con sede en los Estados Unidos, resultan aplicables las consideraciones del artículo 12 de la mencionada ley sobre transferencia internacional de datos personales y, de manera complementaria, los lineamientos del Reglamento General de Protección de Datos de la Unión Europea (Parlamento Europeo y Consejo de la Unión Europea, 2016) en aquellos casos que pudieran involucrar a residentes europeos. La Unión Europea reconoce a la Argentina como país con nivel adecuado de protección de datos personales mediante la Decisión 2003/490/CE, del 30 de junio de 2003, revalidada en enero de 2024. Esta adecuación ampara el flujo desde la Unión Europea hacia la Argentina y no opera en sentido inverso: los Estados Unidos no revisten la condición de país adecuado para la normativa argentina, de modo que la transferencia desde la Argentina hacia los proveedores de inferencia y de telefonía con sede en ese país requiere un instrumento específico conforme al artículo 12 de la Ley 25.326, según se detalla en la Subsección 7.3.

7.2. Principios aplicados al tratamiento de datos

El diseño del sistema se atiene de manera explícita a cinco principios fundamentales del tratamiento de datos personales. El principio de licitud establece que todo tratamiento debe ampararse en una base jurídica valida; en este caso, dicha base se encuentra en el artículo 5, inciso 2, apartado d) de la Ley 25.326, que admite el tratamiento de los datos personales que deriven de una relación contractual, científica o profesional del titular y resulten necesarios para su desarrollo o cumplimiento. La organización es la empleadora y trata datos de sus propios empleados para la gestión interna de los incidentes informáticos que estos reportan en ejercicio de sus funciones laborales. Esta base de licitud operativa se distingue del consentimiento que se analiza, para el corpus de investigación académica, en la Subsección 7.5, y no dispensa el deber de información del artículo 6 de la Ley 25.326, que subsiste en su inte-

gridad. El principio de finalidad determinada limita el uso de los datos a la finalidad declarada de registro y derivación de incidentes, prohibiendo cualquier uso secundario no compatible. El principio de minimización exige recolectar únicamente los datos estrictamente necesarios. En consecuencia, el sistema almacena exclusivamente el nombre de usuario corporativo y un directorio institucional acotado de empleados —legajo, nombre, correo electrónico corporativo, teléfono, sector y rol—, cuya única finalidad es la resolución de contacto y la notificación al usuario, y descarta cualquier otro dato personal que pudiera capturarse incidentalmente, tales como domicilios particulares, números de documento o información financiera. El principio de exactitud se materializa mediante la posibilidad ofrecida al usuario de revisar y corregir el contenido del incidente antes de su persistencia definitiva. En línea con este principio, el ejercicio de los derechos de rectificación, actualización y supresión (derechos ARCO) se resuelve en un plazo de cinco días hábiles, conforme al artículo 16 de la Ley 25.326. El principio de limitación del plazo de conservación se traduce, para los incidentes, en un plazo de retención de 12 meses rodantes desde la creación del incidente, justificado por el ciclo estadístico anual sobre el que se realiza el análisis de tendencias y la identificación de problemas recurrentes. Este plazo rige para los incidentes y es independiente de la retención de los registros de ejecución del sistema, que se conservan durante 30 días con fines de auditoría y diagnóstico operativo (Subsección 5.3): se trata de dos conjuntos de datos distintos, con finalidades distintas y plazos propios, de modo que ambas políticas resultan compatibles. Los datos del directorio de empleados, en cambio, se conservan mientras la relación laboral permanezca activa y hasta un año después de su finalización, transcurrido el cual se eliminan, en tanto cumplen una función operativa de contacto y no un fin estadístico. Vencido el plazo de retención, los datos del incidente se someten a anonimización irreversible, de modo que se eliminan los identificadores que permiten vincularlos a una persona. El bloqueo de escritura sobre los incidentes en estado cerrado, que impide editarlos o eliminarlos desde la interfaz de usuario y rechaza con un error 409 los intentos de modificación, constituye un control de integridad de auditoría: preserva la trazabilidad del registro durante el plazo de retención vigente, pero no configura una política de conservación ni una excepción permanente al vencimiento de ese plazo. La pseudonimización de datos personales y el cifrado en reposo garantizan la protección de la información durante el plazo de conservación declarado.

7.3. Transferencia internacional y pseudonimización

El tratamiento de incidentes comporta dos transferencias internacionales de datos hacia proveedores con sede en los Estados Unidos. La primera corresponde al modelo de inferencia Gemini 3.6 Flash, al que se envian fragmentos textuales descriptivos de los incidentes. En este canal, el sistema aplica un procedimiento de pseudonimización dentro del módulo Python desplegado en infraestructura local de la organización, mediante el cual se reemplazan automáticamente los nombres propios, las direcciones de correo electrónico, los números telefónicos, los identificadores corporativos y los nombres de hosts internos por etiquetas genéricas tales como `[PERSONA]`, `[EMAIL]` o `[HOST]`; de este modo, el texto que se transmite no contiene identificadores directos. Esta pseudonimización se valida mediante un conjunto de expresiones regulares específicas del dominio, acompañadas de pruebas unitarias dedicadas.

La segunda transferencia corresponde al canal telefónico, operado por el proveedor Twilio, con sede en los Estados Unidos. El orden real del tratamiento en este canal es: el llamante deja su solicitud y Twilio graba el audio; a continuación, el audio crudo y el número telefónico del llamante se transfieren a la infraestructura del proveedor en los Estados Unidos y se transcriben mediante el servicio de reconocimiento del habla; recién después, el backend aplica la pseudonimización sobre el texto ya transcrito. En consecuencia, el audio crudo del llamante y su número telefónico cruzan a los Estados Unidos antes de cualquier pseudonimización, que opera únicamente sobre el texto y no sobre el audio.

Ambas transferencias se dirigen a un país que no reviste la condición de adecuado para la normativa argentina, por lo que requieren un instrumento específico conforme al artículo 12 de la Ley 25.326: el consentimiento del titular o la suscripción de cláusulas contractuales modelo. El acuerdo de procesamiento de datos suscripto con cada proveedor complementa ese instrumento, pero no lo sustituye, en tanto no configura por si mismo una de las bases de transferencia que la ley admite.

7.4. Seguridad técnica

Las medidas de seguridad técnica implementadas se sustentan en el modelo de la triada confidencialidad, integridad y disponibilidad (CIA) descrito por Stallings y Brown (2017). La confidencialidad se sustenta, en el diseño, en el cifrado en tránsito sobre TLS 1.2/1.3 a nivel del proxy inverso (Nginx) que protege el perímetro del sistema; el despliegue publicado para la validación experimental no incluyó el proxy, por lo que esa capa de protección correspon-

de al diseño y no a una capacidad verificada en la instancia publicada. A nivel de aplicación, la confidencialidad se garantiza mediante el cifrado con Fernet (AES-128 en modo CBC con HMAC-SHA-256 integrado, provisto por la biblioteca `cryptography`) para los campos sensibles del incidente (descripción original, número telefónico del llamante y transcripción). Los datos de contacto del directorio de empleados —correo corporativo y teléfono— se almacenan sin cifrado a fin de preservar la auditabilidad directa y las consultas operativas; su protección se sustenta en el control de acceso por rol, el aislamiento de la red interna de Docker y el registro de auditoría de los accesos. La comunicación interna entre los servicios del sistema (N8N, backend, PostgreSQL, Redis) se realiza sobre la red aislada de Docker, que constituye el perímetro de confianza. La integridad se asegura mediante el cifrado autenticado Fernet, que incorpora HMAC-SHA-256 en su modo de operación, la validación estricta de esquemas mediante Pydantic en cada llamada a la interfaz REST, y un registro de auditoría de toda modificación que conserva el actor, la marca temporal y el delta del cambio. La disponibilidad se respalda mediante respaldos diarios automáticos con retención escalonada, despliegue en contenedores con políticas de reinicio automático ante fallas, y monitoreo de salud activo mediante endpoints de chequeo. Las credenciales de servicios externos se gestionan exclusivamente mediante variables de entorno inyectadas por el orquestador de contenedores y nunca se almacenan en el repositorio de código fuente.

7.5. Consentimiento, derechos del usuario y supervisión humana

Los usuarios de la organización son informados, mediante una política de privacidad accesible desde los canales de entrada, sobre las características del sistema, los datos que se recolectan, la transferencia internacional aplicable y los derechos que les asisten conforme a la normativa vigente. Se garantiza el ejercicio de los derechos de acceso, rectificación, actualización y supresión (derechos ARCO) mediante un procedimiento documentado a cargo del responsable de protección de datos designado por la organización.

En lo que respecta al consentimiento informado para los fines de investigación académica, los 120 empleados de la organización fueron notificados mediante comunicación interna formal, con descripción explícita del propósito del estudio, del tipo de datos tratados, del carácter pseudonimizado de los registros y del derecho a requerir la exclusión de sus incidentes del corpus de validación. El procedimiento fue supervisado por la dirección académica del trabajo y avalado institucionalmente mediante una carta de autorización emitida por la organización

adoptante.

En relación con el protocolo de medición en vivo aplicado durante la fase experimental —en el cual el cronometraje no fue anunciado en tiempo real al operador cuyo tiempo se registraba, con el objeto de mitigar el efecto Hawthorne, pero cuyo conocimiento fue diferido—, corresponde dejar constancia de que, una vez concluida la recolección de datos, se realizó una sesión de esclarecimiento posterior (*debriefing*) con los dos operadores participantes. En dicha sesión se explicó el propósito del estudio, se detalló el procedimiento de medición empleado y se ofreció a cada operador la posibilidad de solicitar la exclusión de sus registros individuales del corpus de validación. Ninguno de los operadores ejerció este derecho. Este procedimiento se alinea con las recomendaciones de la Asociación Americana de Psicología para investigaciones que emplean medición con conocimiento diferido de los participantes (American Psychological Association, 2017), y fue supervisado por el director del trabajo.

En concordancia con las recomendaciones contemporáneas sobre sistemas que incorporan inteligencia artificial, se preserva en todo momento la posibilidad de supervisión humana significativa. Ninguna decisión que afecte derechos sustanciales de los usuarios se toma de manera completamente automatizada sin posibilidad de revisión, y el sector responsable conserva la potestad de modificar la clasificación inicial cuando lo considere apropiado, registrándose tal modificación en la tabla de trazabilidad descrita en la Sección 5. Este mecanismo materializa el principio de la persona en el bucle (*human in the loop*) como salvaguarda frente a errores potenciales del clasificador automático. Cabe precisar que el umbral de confianza de 0,70 que dispara la revisión obligatoria no constituye, por sí solo, una garantía suficiente de esa salvaguarda: el análisis de calibración de la Sección 8 muestra que la confianza declarada por el clasificador no discrimina de forma fiable los casos correctos de los incorrectos, por lo que la supervisión humana efectiva se apoya además en la revisión por el sector responsable y en la trazabilidad de las correcciones, y no únicamente en el puntaje del modelo.

8. Resultados

Este capítulo presenta los resultados de la validación experimental del sistema. Se reportan los hallazgos en cuatro dimensiones: tiempos de registro, desempeño de clasificación, reducción de la intervención humana y análisis cualitativo de errores.

8.1. Comparación de tiempos de registro

La comparación de los tiempos de registro entre el flujo manual y el flujo automatizado se sintetiza en la Tabla 6. La media aritmética del tiempo manual sobre los 200 casos del corpus fue de 61,6 s (mediana de 50,0 s), con un desvío estándar de 44,9 s y un rango entre 10 y 300 s. Este valor, respaldado por la planilla de cronometraje descrita en la Subsección 4.2, corresponde a la medición experimental incidente por incidente y reemplaza a la estimación preliminar de 2 minutos con 45 segundos citada en la Subsección 1.2, obtenida sin cronometraje individual. La media del flujo automatizado fue de 17,76 s (mediana de 17,47 s), con un desvío estándar de 13,48 s y un rango entre 0,525 y 65,834 s. La reducción relativa de la media de tiempo alcanzó el 71,2 %.

Tabla 6. Estadísticos descriptivos de los tiempos de registro por flujo ($n = 200$)

aggedright**Estadístico**	**Flujo manual**	**Flujo automatizado**	**Reducción**
aggedrightMedia aritmética (s)	61,6	17,76	71,2 %
aggedrightMediana (s)	50,0	17,47	65,1 %
aggedrightDesvío estándar (s)	44,9	13,48	—
aggedrightMínimo (s)	10	0,525	—
aggedrightMáximo (s)	300	65,834	—

Fuente: elaboración propia a partir del corpus de validación ($n = 200$).

El tiempo automatizado de registro desagregado por canal de origen se presenta en la Tabla 7. El canal telefónico incorpora, además de la latencia de procesamiento, la duración de la llamada grabada, de modo que su valor es comparable en su base de medición con el flujo manual; como la grabación esta acotada a un máximo de 45 s (véase la Subsección 4.6), ese valor constituye una cota inferior de la duración real de la llamada.

La prueba de Wilcoxon de rangos con signo aplicada sobre las mediciones pareadas (Virtanen et al. 2020) arrojó un estadístico de contraste $W = \text{mín}(T^+, T^-) = 363$, con una suma de rangos positivos $T^+ = 19.737$, una suma de rangos negativos $T^- = 363$ y $n = 200$

Tabla 7. Tiempo automatizado de registro por canal de origen ($n = 200$)

Canal de origen	Casos	Media (s)	Mediana (s)
Llamada telefónica (cota inferior)	81	20,09	18,70
Correo electrónico	66	27,65	25,82
Formulario web	53	1,88	1,26
Global	**200**	**17,76**	**17,47**

Fuente: elaboración propia a partir del corpus de validación ($n = 200$).

pares válidos, junto con un valor $p = 3{,}08 \times 10^{-32}$, lo que permite rechazar la hipótesis nula de igualdad de medianas con un nivel de confianza superior al 99,9 %. El estadístico incorpora la corrección por empates y refleja que en 188 de los 200 pares el flujo automatizado fue más veloz que el manual; en los 12 pares restantes (6,0 %) el flujo automatizado no fue más rápido, resultado coherente con la diferencia de aproximadamente 32,5 s entre medianas. La elección de esta prueba no paramétrica responde al rechazo de la hipótesis de normalidad en los tres conjuntos de tiempos, cuyo reporte completo se presenta en la Subsección 4.7.

Para cuantificar la magnitud práctica de la diferencia, se calculó el coeficiente r de correlación rank-biserial a partir de los dos estadísticos de rango:

$$r = \frac{T^{+} - T^{-}}{n(n+1)/2} = \frac{19.737 - 363}{200 \cdot 201/2} = \frac{19.374}{20.100} = 0{,}9639$$

El valor obtenido ($r = 0{,}9639$) corresponde a un tamaño del efecto muy alto, consistente con la alta proporción de pares a favor del flujo automatizado (188 de 200) y con los 12 pares en que este no superó al manual. La diferencia observada es, por tanto, estadísticamente significativa y, dada su magnitud absoluta, relativa y el tamaño del efecto reportado, también prácticamente relevante para la operación.

Cabe señalar que el tiempo manual se registró con redondeo hacia arriba (véase la Subsección 4.2), un sesgo de medición sistemático que infla la línea base manual. Por ese motivo la reducción del 71,2 % debe leerse como cota superior. Un análisis de sensibilidad que descuenta 5 s y 10 s a cada medición manual reduce la media manual a 56,6 s y 51,6 s, respectivamente, y sitúa la reducción en el 68,6 % y en el 65,6 % como cota inferior conservadora. Aun en el escenario más desfavorable la reducción de tiempo permanece sustancial y estadísticamente significativa ($p = 1{,}8 \times 10^{-26}$, $r = 0{,}8683$).

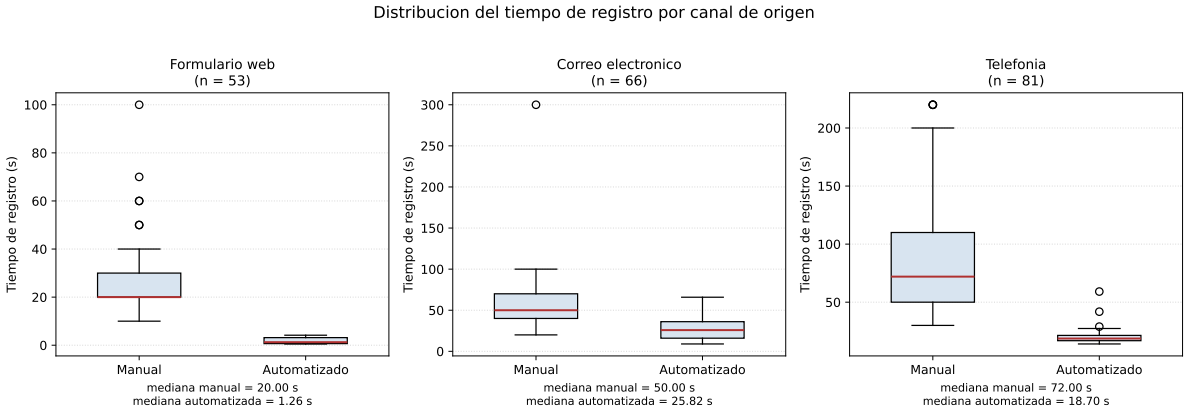


Figura 6. Distribución del tiempo de registro por canal de origen (manual frente a automatizado). El panel de telefonía incorpora la duración de la llamada grabada.

8.2. Matriz de confusión y métricas de clasificación

La evaluación de clasificación se realizó sobre el corpus real de 200 casos pseudonimizados. Su distribución por sector asignado es la siguiente: Soporte Técnico Hardware 85, Soporte Técnico Software 67, Seguridad Informática 31, Sistemas 16 y Bases de Datos 1. Por canal de origen, 81 casos provienen de llamada telefónica, 66 de correo electrónico y 53 de formulario web.

La matriz de confusión obtenida se presenta en la Tabla 8 y en la Figura 7. De los 85 casos de Soporte Tecnico Hardware, 77 se clasificaron correctamente; de los 67 de Soporte Tecnico Software, 48; de los 31 de Seguridad Informatica, 15; y de los 16 de Sistemas, 7. La exactitud estricta resultante asciende al 73,5 % (147 casos correctos sobre 200).

Tabla 8. Matriz de confusión del clasificador automático ($n = 200$). Filas: sector asignado; columnas: sector predicho principal.

Asignado \ Predicho	**Seguridad Informatica**	**Soporte Tecnico Hardware**	**Soporte Tecnico Software**	**Bases de Datos**	**Sistemas**	**Total**
Seguridad Informatica	15	2	11	2	1	**31**
Soporte Tecnico Hardware	1	**77**	3	0	4	**85**
Soporte Tecnico Software	0	11	**48**	1	7	**67**
Bases de Datos	0	0	1	0	0	**1**
Sistemas	2	0	5	2	**7**	**16**
Total	**18**	**90**	**68**	**5**	**19**	**200**

Fuente: elaboración propia a partir del corpus de validación ($n = 200$).

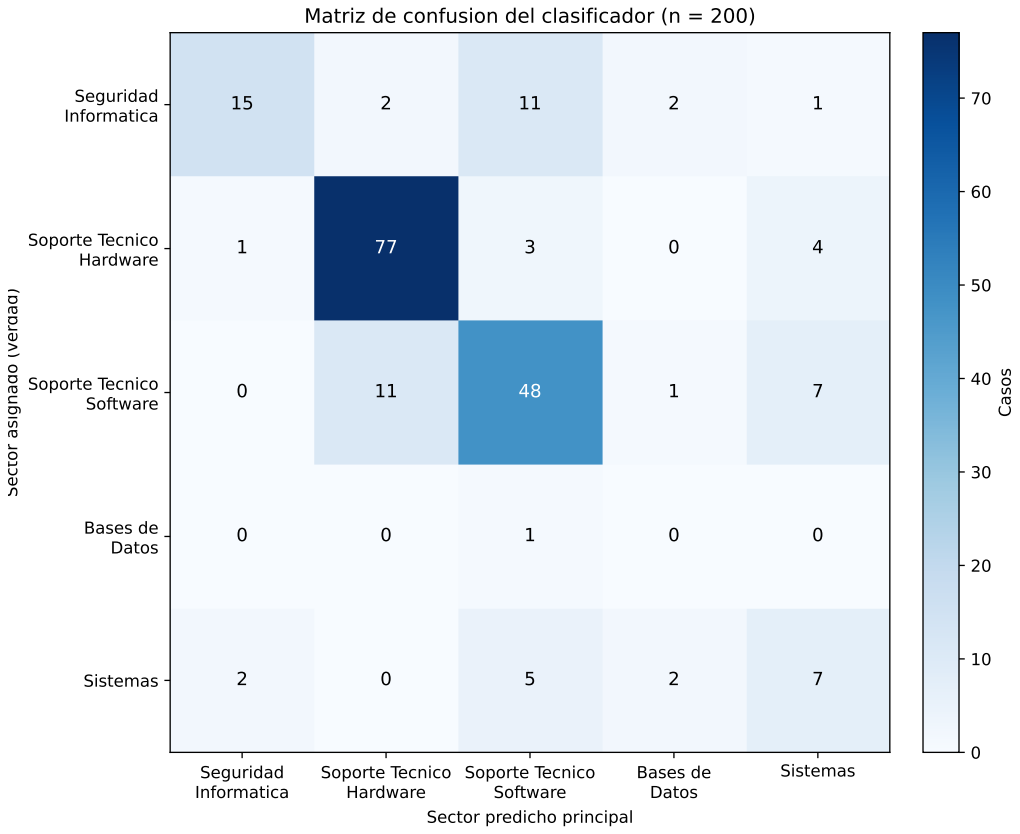


Figura 7. Matriz de confusión del clasificador ($n = 200$). Filas: sector asignado (verdad); columnas: sector predicho principal.

Los valores de precisión, sensibilidad (recall) y F1 por clase, con su intervalo de confianza al 95 % (Sokolova & Lapalme, 2009), se presentan en la Tabla 9; el mejor desempeño correspondió a Soporte Tecnico Hardware (F1 = 0,8800) y Sistemas fue la clase estimable con el F1 más bajo (0,4000). La evaluación se enmarca en un único marco métrico cuya referencia de verdad es multietiqueta (107 de los 200 casos, 53,5 %), frente a una salida del clasificador de una etiqueta principal: la exactitud estricta del 73,5 % (147/200) es la métrica primaria monoetiqueta, con IC de Wilson al 95 % de [67,0 %, 79,1 %]. El macro-F1 de umbral se reporta sobre las cuatro clases estimables (0,6508), excluida Bases de Datos ($n = 1$, no estimable), y el de cinco clases (0,5207) se conserva como valor informativo. Las métricas secundarias se calculan contra el conjunto de verdad del corpus (multietiqueta): subset accuracy 0,43 (IC [0,36; 0,50]), Hamming loss 0,3586 (IC [0,31; 0,41]) y micro-F1 0,6654 (IC [0,62; 0,71]). Estas métricas de conjunto penalizan estructuralmente una salida forzada a una sola etiqueta y no son equivalentes de la exactitud estricta: el micro-F1 acumula los aciertos por etiqueta y no por caso, de modo que su valor (0,6654) no debe leerse como una exactitud (0,7350). De los 200 casos, 131 fueron resueltos por la etapa determinística del clasificador y 69 requirieron la etapa Gemini.

Tabla 9. Métricas de clasificación por clase con intervalo de confianza al 95 % (una contra el resto)

Clase	Precisión (IC 95 %)	Sensibilidad (IC 95 %)	F1 (IC 95 %)
Soporte Tecnico Hardware	0,8556 [0,77; 0,91]	0,9059 [0,83; 0,95]	0,8800 [0,82; 0,93]
Soporte Tecnico Software	0,7059 [0,59; 0,80]	0,7164 [0,60; 0,81]	0,7111 [0,61; 0,80]
Seguridad Informatica	0,8333 [0,61; 0,94]	0,4839 [0,32; 0,65]	0,6122 [0,44; 0,75]
Sistemas	0,3684 [0,19; 0,59]	0,4375 [0,23; 0,67]	0,4000 [0,16; 0,59]
Bases de Datos		no estimable ($n = 1$)	

Fuente: elaboración propia a partir del corpus de validación ($n = 200$). IC de precisión y sensibilidad por Wilson; IC de F1 por bootstrap percentil (2000 remuestreos).

Estas métricas confirman un desempeño desigual entre las clases objetivo (Sokolova & Lapalme, 2009): la clase mayoritaria (Soporte Tecnico Hardware, con 85 casos) obtuvo el mejor resultado, mientras que Sistemas fue la clase estimable con los valores más bajos y Bases de Datos ($n = 1$) no es estimable. Este desequilibrio importa en el contexto operativo, porque un clasificador que concentra sus errores en una clase minoritaria genera derivaciones erroneas en un sector específico y puede erosionar la confianza de ese equipo en la herramienta, independientemente de sus métricas agregadas.

La inspección de la matriz de confusión revela que los 53 errores totales se concentran

en fronteras entre sectores léxicamente próximos, con Soporte Tecnico Software como categoría central; su análisis cualitativo, agrupado por frontera de confusión, se desarrolla en la Subsección 8.5.

8.3. Reducción de la intervención humana

La intervención humana en el flujo automatizado se deriva del gate de revisión implementado, `requiere_revision_humana = (confianza <0,70)`, que escala un caso a un operador humano cuando la confianza devuelta por el clasificador cae por debajo de ese umbral. En el flujo manual, el 100 % del tiempo correspondió a intervención humana, incluyendo lectura, comprensión, búsqueda en sistemas auxiliares y carga del ticket. En el flujo automatizado, en cambio, el gate no derivó ningún caso del corpus: la etapa determinística fija el campo en `False` y la etapa Gemini nunca devolvió una confianza inferior a 0,85.

La proporción de intervención humana por caso en el flujo automatizado no se declara como resultado porque no se conserva un registro primario de intervención por caso. La proporción derivable del gate de revisión es del 0 %, pero esa cifra describe el disparador automático y no la intervención real observada por el sector responsable; el aporte operativo asociado al gate se declara, por lo tanto, como no medido, a la espera de una tabla de contingencia primaria que registre la intervención por caso.

8.4. Calibración de la confianza

La confianza declarada por el clasificador solo resulta útil como criterio de derivación si guarda relación con la exactitud real. Para verificar esa relación se contrastó, sobre los 200 casos del corpus, la confianza devuelta en cada predicción contra la corrección de la clasificación. La Tabla 10 desagrega la exactitud por bandas de confianza en torno al umbral del 70 % que gobierna el gate de revisión humana.

Tabla 10. Exactitud por banda de confianza declarada

Banda de confianza	Casos	Aciertos	Exactitud
$[0{,}00\,;\,0{,}50)$	98	75	0,765
$[0{,}50\,;\,0{,}70)$	26	19	0,731
$[0{,}70\,;\,0{,}85)$	6	2	0,333
$[0{,}85\,;\,0{,}95)$	31	22	0,710
$[0{,}95\,;\,1{,}00]$	39	29	0,744

Fuente: elaboración propia a partir del corpus de validación ($n = 200$).

Los datos revelan una calibración invertida o anti-calibrada: los casos de confianza alta ($\geq 0{,}70$) fueron menos exactos (0,697) que los de confianza baja ($< 0{,}70$, 0,758). La tendencia se acentúa en el extremo superior de la escala: dentro de los 46 casos con confianza igual o superior a 0,90 se registraron 13 errores (28,3 %), y la banda $[0{,}70\,;\,0{,}85)$ alcanzó apenas 0,333 de exactitud. En otras palabras, el clasificador no solo no discrimina mejor cuando declara mayor certeza, sino que en los tramos de confianza más alta la exactitud se degrada. La consecuencia operativa es directa: el gate del 70 % —el mecanismo previsto para derivar a revisión humana los casos dudosos— no aisló los errores, porque estos se distribuyen también en las bandas de confianza alta. La supervisión humana no puede apoyarse en la confianza declarada como único disparador; requiere complementarse con señales independientes del propio puntaje (por ejemplo, la proximidad léxica entre las categorías candidatas), tal como se discute en la Sección 9.

8.5. Análisis de errores y casos límite

El análisis cualitativo de los 53 casos mal clasificados permitió identificar patrones de error recurrentes, agrupados por frontera de confusión y sintetizados en la Tabla 11.

Tabla 11. Confusiones observadas en la clasificación automática, por frontera

Frontera de confusión	**Casos**	**Proporción**
Soporte Tecnico Hardware ↔ Soporte Tecnico Software	14	26,4 %
Sistemas ↔ Soporte Tecnico Software	12	22,6 %
Seguridad Informatica ↔ Soporte Tecnico Software	11	20,8 %
Otras fronteras	16	30,2 %
Total	**53**	**100 %**

Fuente: elaboración propia a partir del corpus de validación ($n = 200$).

El patrón dominante, presente en 14 casos (26,4 % de los errores), corresponde a la frontera entre Soporte Tecnico Hardware y Soporte Tecnico Software: incidentes en que una falla física de un equipo afecta a una aplicación, o en que una tarea sobre una aplicación se describe con términos de equipamiento, de modo que la decisión correcta depende del juicio humano sobre que aspecto prevalece. El segundo patrón, presente en 12 casos (22,6 %), corresponde a la frontera entre Sistemas y Soporte Tecnico Software, asociada a acciones sobre servidores y servicios de plataforma descritas como problemas de una aplicación. El tercer patrón, presente en 11 casos (20,8 %), corresponde a la frontera entre Seguridad Informatica

y Soporte Tecnico Software, vinculada a incidentes de acceso y credenciales. El 30,2 % restante se reparte entre las demas fronteras, incluidas las confusiones que involucran a Bases de Datos, con un único caso de soporte.

Los patrones anteriores se ilustran con tres casos mal clasificados del corpus. Las descripciones se reproducen anonimizadas —se eliminaron nombres de personas, correos, teléfonos, direcciones y denominaciones internas de sistemas— y se acompañan con el sector asignado, el sector predicho y la confianza declarada por el clasificador. El primer caso corresponde al patrón de descripción corta: la entrada “No puede ingresar al sistema central” fue asignada a Seguridad Informatica y predicha como Soporte Tecnico Software con una confianza de 0,95; su brevedad no aporta contexto sobre el recurso ni sobre la naturaleza del acceso fallido, de modo que la frontera resulta indistinguible sin información adicional. El segundo ilustra la mezcla de categorías: “Su equipo se reinicia de forma reiterada; el usuario sospecha de los cables” fue asignada a Soporte Tecnico Software y predicha como Soporte Tecnico Hardware con 0,50 de confianza, y combina un síntoma de nivel de aplicación con una hipótesis de causa física cuyo peso relativo no queda determinado por el texto. El tercero ejemplifica la jerga local: “No le salen los correos por el flujo interno y no le llegan las instancias” fue asignada a Sistemas y predicha como Soporte Tecnico Software con 0,85 de confianza, y apela a términos organizacionales (“flujo”, “instancias”) cuyo significado no está disponible para el clasificador. En los tres casos la confianza declarada es alta pese al error, lo que refuerza el problema de calibración descrito en la Sección 8.4.

Esta tipología orienta directamente las recomendaciones presentadas en la Sección 11, particularmente la incorporación de un clasificador supervisado entrenado con el corpus etiquetado generado por el propio sistema y la inclusión de un glosario de términos organizacionales en el prompt del modelo.

9. Discusión

9.1. Interpretación de los resultados

Los resultados obtenidos confirman la hipótesis de trabajo formulada en la Subsección 1.4 en lo relativo al tiempo de registro y la refutan en lo relativo a la clasificación, de modo que la hipótesis principal se declara parcialmente confirmada y parcialmente refutada: la disminución del 71,2 % en el tiempo medio de registro es estadísticamente significativa y operativamente relevante, mientras que la exactitud estricta del 73,5 % alcanzada por el clasificador automático quedó por debajo del umbral del 85 % planteado en el segundo objetivo específico, resultado que se analiza entre las limitaciones de este capítulo. Tres aspectos de los resultados merecen una interpretación detallada.

La reducción de tiempo se sostiene incluso al incorporar el costo de inferencia del modelo de lenguaje grande externo, dado que la primera etapa determinística del clasificador resuelve el 65,5 % de los casos sin invocar al modelo de Google. Este hallazgo es consistente con la hipótesis subsidiaria sobre la eficiencia del esquema híbrido: el filtro determinístico no solo reduce el costo económico del sistema, sino que además disminuye la latencia mediana del flujo automatizado al evitar llamadas de red innecesarias al servicio externo. Desde una perspectiva operativa, esta propiedad del clasificador introduce un punto de inflexión en la relación costo-beneficio de los sistemas basados en LLM: el costo marginal de la inferencia se aplica únicamente a los casos que realmente demandan capacidad semántica, y no al volumen completo de incidentes.

La dispersión observada en los tiempos del flujo automatizado —con un rango de 0,525 a 65,834 segundos, frente a los 10 a 300 segundos del flujo manual— evidencia una compresión sustancial de la variabilidad. Mientras que en el flujo manual un incidente podía demandar entre 10 y 300 segundos según la complejidad percibida y la carga cognitiva del operador, en el flujo automatizado el rango se comprime a 0,525–65,834 segundos, con un desvío estándar de 13,48 s frente a los 44,9 s del flujo manual, es decir aproximadamente la tercera parte de la dispersión manual. Esta compresión no fue un objetivo explícito del diseño pero emerge como consecuencia observada de la automatización.

La intervención humana por caso en el flujo automatizado no se midió, de modo que el aporte operativo asociado al gate de revisión se declara como no medido; el sistema absorbe la carga rutinaria sin sustituir el juicio humano en los casos que verdaderamente lo de-

mandan. Este equilibrio entre automatización y supervisión preserva la calidad del servicio y libera capacidad operativa significativa. Si se proyecta el ahorro medio de tiempo por incidente ($61{,}6 - 17{,}76 = 43{,}8$ s) sobre el volumen trimestral de 2.646 incidentes, la automatización liberaría aproximadamente 32,2 horas-persona por trimestre,[1] tiempo que el personal podría redirigir a tareas de resolución técnica o a actividades de mejora continua. La magnitud de esta liberación —equivalente a unas cuatro jornadas laborales completas de un operador— transforma la automatización del registro de un proyecto de optimización marginal en una intervención con impacto organizacional medible.

Sin embargo, la lectura conjunta de la intervención humana por gate y del análisis de calibración presentado en la Sección 8.4 matiza ese resultado. El gate del 70 % —el único disparador automático de revisión humana— presupone que la confianza declarada por el clasificador discrimina los casos correctos de los incorrectos, presupuesto que el corpus no confirma. La consecuencia práctica es que la supervisión humana efectiva no puede depender exclusivamente del umbral de confianza: debe complementarse con señales independientes del puntaje —por ejemplo, la ambigüedad léxica entre las categorías candidatas o la revisión por muestreo de los casos clasificados con alta confianza—. El esquema *human in the loop* conserva su valor como salvaguarda, pero su cobertura real es menor que la que sugiere la sola proporción de casos escalados.

El rigor metodológico adoptado para evitar *data leakage* merece una observación aparte. A diferencia de trabajos que particionan el corpus en conjuntos de entrenamiento y prueba después de realizar exploraciones iniciales —práctica que puede introducir fuga de información—, el presente estudio aisló completamente la construcción de los componentes críticos del clasificador del corpus de validación de 200 casos. El filtro determinístico se construyó mediante análisis manual de incidentes no incluidos en el corpus, y el prompt fue ajustado en entornos de preproducción sin exposición a los 200 casos de validación. Solo después de completar el desarrollo se evaluó el desempeño sobre la totalidad del corpus, garantizando que las métricas reportadas reflejan el desempeño genuino del sistema sobre datos no vistos. Esta separación estricta entre datos de construcción y datos de validación constituye una garantía metodológica del presente estudio.

La reducción del tiempo de registro, aunque estadísticamente robusta, admite explica-

[1] Cálculo: $43{,}8$ s/incidente $\times$ 2.646 incidentes$/3.600$ s/h $\approx 32{,}2$ h-persona por trimestre. La cifra previa de 180 horas-persona no era reproducible con los tiempos y el volumen declarados.

ciones rivales. Primero, la asimetría del instrumento: el flujo manual se cronometró en vivo e incluye lectura, comprensión, búsqueda y carga del ticket, mientras que el automatizado se midió con marcas de tiempo del sistema, lo que favorece al segundo; lo debilita el análisis de sensibilidad, que mantiene la reducción por encima del 65 % aun descontando 10 s por caso, y se descartaría midiendo ambos flujos con el mismo instrumento. Segundo, la mezcla de canales: el corpus combina telefonía, correo y formulario web, con costos de entrada distintos, de modo que parte de la diferencia podría reflejar la composición; lo debilita el desagregado por canal, con reducciones positivas en los tres, y se descartaría con un contraste intra-canal apareado. Tercero, la novedad o efecto Hawthorne y el aprendizaje del operador: el orden temporal de la campaña (manual antes que automatizado) sesga, de hacerlo, en contra de la hipótesis, y se descartaría con un diseño contrabalanceado. Cuarto, la selección de los 200 casos: por ser una muestra estratificada de una sola organización, la magnitud observada no es extrapolable sin nuevas validaciones. Ninguna de estas explicaciones rivales se descarta por completo con el diseño actual.

El patrón de error más informativo es la confusión sistemática de Sistemas: de los 16 casos asignados, solo 7 se clasificaron correctamente, con precisión 0,3684, sensibilidad 0,4375 y F1 0,4000, y errores repartidos entre Soporte Tecnico Software (5), Bases de Datos (2) y Seguridad Informatica (2). Esta frontera combina dos causas que conviene no confundir. La primera es una ambigüedad léxica y taxonómica: las descripciones de acciones sobre servidores, servicios de plataforma o aplicaciones admiten más de un sector, y la etiqueta de referencia resuelve esa ambigüedad mediante una convención que el clasificador no conoce. La segunda es falta de contexto: muchas entradas son demasiado breves para desambiguar el sector responsable, y el modelo carece de señales que un operador obtendría por conocimiento organizacional. Solo la segunda admite mitigación por enriquecimiento del protocolo de captura; la primera exige revisar la taxonomía o el criterio de etiquetado. Cabe advertir, además, que Bases de Datos ($n = 1$) no es estimable: su F1 no debe leerse como evidencia de que el clasificador falla en esa clase, sino como ausencia de soporte estadístico.

En conjunto, los resultados declaran la hipótesis principal como parcialmente confirmada y parcialmente refutada: se confirma en la reducción del tiempo de registro y se refuta en la clasificación, cuyo desempeño queda por debajo del umbral del segundo objetivo específico. Esa lectura no equivale a una confirmación concluyente: la evidencia es compatible con explicaciones rivales que el diseño no permite descartar por completo. Mientras que la reducción

observada admite una lectura causal fuerte en el plano práctico —el sistema registra cada incidente en menos tiempo que el proceso manual—, su atribución a cada componente del esquema y su extrapolación a otras organizaciones permanecen abiertas. Las amenazas a la validez que acotan estas conclusiones están enumeradas en la Subsección 4.8.

9.2. Comparación con la literatura

Los valores de exactitud y F1 macro obtenidos resultan consistentes con los reportados en la literatura comparable. Paramesh y Shreedhara (2019) reportan exactitudes cercanas al 87 % utilizando SVM sobre representaciones TF-IDF. Revina et al. (2020), en cambio, muestran que los clasificadores simples y explicables construidos con rasgos lingüísticos del dominio son competitivos frente a alternativas más complejas, y que los modelos preentrenados pueden rendir peor por vocabulario específico del dominio. La literatura reciente sobre clasificación de tickets con LLM reporta resultados dependientes del idioma y del dominio: Paket et al. (2024) y Liu et al. (2025) documentan mejoras frente a las líneas base clásicas, mientras que Colavito et al. (2025) advierten sobre la variabilidad del etiquetado automático según el corpus evaluado.

Los resultados del presente trabajo —73,5 % de exactitud estricta y un F1 macro de 0,6508 sobre las cuatro clases estimables (0,5207 al incluir la clase de soporte unitario, no estimable)— se ubican por debajo de los valores reportados en la literatura para clasificación de tickets, si bien la comparación directa no es estricta: los trabajos citados reportan clasificación de etiqueta única, mientras que la evaluación propia se mide contra una referencia multietiqueta sobre cinco sectores canónicos en español rioplatense, tarea considerablemente más exigente. Frente al enfoque SVM+TF-IDF de Paramesh y Shreedhara (2019) (87 %) y a la evidencia de Revina et al. (2020) sobre la competitividad de los métodos simples, el esquema propuesto no requiere infraestructura de GPU para inferencia local ni un corpus de entrenamiento etiquetado de gran escala, lo que lo hace más accesible para organizaciones medianas, aunque su exactitud estricta resulta menor en la comparación directa.

La evidencia de Revina et al. (2020) resulta, en apariencia, contraria al enfoque del presente trabajo, que apuesta por un LLM de propósito general. Esta tensión, sin embargo, refuerza el análisis: aquellos autores observan que los modelos preentrenados pueden degradarse ante vocabulario específico del dominio, lo que descarta suponer una transferencia no supervisada trivial. Precisamente por eso la elección del LLM en este trabajo no se justifica por transferencia pasiva de conocimiento lingüístico, sino por la tarea concreta —clasificación de

categoría con contexto instruccional y esquema híbrido de reglas más modelo—, en la que el modelo se emplea como motor de inferencia guiado y no como representación preentrenada sin ajuste.

La novedad del aporte radica en la validación específica sobre español rioplatense aplicado a un dominio de mesa de ayuda de organización mediana, segmento escasamente cubierto por la literatura previa. Diversos trabajos han señalado la menor cobertura del español en los corpus de entrenamiento de los LLM contemporáneos, y los resultados del presente trabajo (F1 macro = 0,6508 sobre las cuatro clases estimables) sugieren que el desempeño en español rioplatense resulta inferior al reportado para idiomas de mayor cobertura como el inglés o el alemán, brecha que se atribuye, al menos en parte, al esquema multietiqueta de cinco sectores adoptado en esta evaluación.

9.3. Limitaciones del estudio

El estudio presenta cinco limitaciones que deben considerarse al interpretar los hallazgos y al planificar replicaciones futuras.

Primero, el tamaño de la muestra ($n = 200$) es suficiente para una validación piloto pero insuficiente para sustentar generalizaciones de carácter poblacional sobre el universo de mesas de ayuda en organizaciones medianas. El intervalo de confianza al 95 % reportado para la exactitud estricta, [67,0 %, 79,1 %], se ubica íntegramente por debajo del umbral del 85 %, por lo que el segundo objetivo específico no se alcanza; muestras mayores, un corpus más equilibrado entre clases y una revisión del clasificador serían necesarios para cerrar esa brecha.

Segundo, la circunscripción del corpus a una única organización del sector servicios de la provincia de Mendoza restringe la validez externa de los hallazgos. La distribución de tipos de incidentes, el léxico organizacional y los patrones de redacción pueden variar significativamente entre organizaciones, sectores y regiones.

Tercero, la dependencia operativa de un servicio externo de inferencia condiciona el rendimiento del sistema. Un eventual deterioro del servicio de Gemini 3.6 Flash, un cambio significativo en las condiciones comerciales del proveedor, o una modificación en el comportamiento del modelo debido a actualizaciones afectaría directamente la operación. Esta dependencia es intrínseca al enfoque basado en LLM externo y debe ser gestionada como un riesgo operativo.

Cuarto, los resultados reflejan el comportamiento del modelo en su versión vigente al momento del estudio (junio de 2026). Las actualizaciones futuras del proveedor pueden modificar tanto el desempeño bruto como la estructura del prompt requerido para alcanzar resultados equivalentes, lo que aconseja la implementación de un régimen de monitoreo continuo de las métricas de clasificación.

Quinto, la comparación se realizó exclusivamente contra el flujo manual (proceso operativo), no contra clasificadores alternativos como TF-IDF+SVM o regresión logística sobre el mismo corpus. Un chequeo exploratorio temprano con un clasificador TF-IDF+SVM sobre un subconjunto no preservado del corpus no se reporta aquí como comparación válida: ni la partición ni el tamaño del subconjunto quedaron registrados, por lo que su cifra no es reproducible ni comparable con las métricas de este capítulo. Se lo menciona únicamente como antecedente del diseño, no como evidencia de superioridad del esquema híbrido. Una comparación sistemática contra clasificadores supervisados sobre el corpus completo, con partición y código preservados, queda como trabajo futuro.

9.4. Implicancias prácticas

Desde el punto de vista práctico, los resultados sugieren que organizaciones medianas con volúmenes de incidentes comparables al observado en la organización analizada pueden obtener beneficios sustanciales mediante la implementación de soluciones híbridas similares a la propuesta. La automatización de la etapa de registro libera al personal técnico para concentrarse en la resolución efectiva de los casos, etapa en la cual el juicio experto continua siendo insustituible.

La trazabilidad provista por el registro detallado de cada decisión del clasificador habilita dos procesos de mejora continua. Por un lado, permite identificar patrones de error sistemáticos y ajustar el filtro determinístico o el prompt del LLM en consecuencia. Por otro lado, la acumulación sostenida de decisiones validadas por los sectores responsables constituye un activo informacional que, una vez alcanzado un volumen adecuado (del orden de 5.000 casos), permitiría entrenar modelos de clasificación específicos del dominio organizacional, con eventuales ventajas en costo, latencia y soberanía sobre los datos.

En el plano económico, el costo operativo del sistema se compone de dos rubros principales. El primero es la infraestructura autoalojada: una instancia pública con 2 vCPU y 4 GB de memoria se ofrece a 24 USD/mes, rubro que incluye 4.000 GiB de transferencia saliente

y 80 GiB de disco SSD y al que se suman, si se contratan, el respaldo automático (entre el 20 % semanal y el 30 % diario del costo de la instancia) y otros servicios de almacenamiento y direccionamiento; el costo mensual se ubica así en el orden de 24 a 32 USD/mes.[2] El segundo rubro es el costo de inferencia del LLM. Con el precio introductorio vigente del modelo `gemini-3.6-flash` (0,75 USD por millón de tokens de entrada y 3,75 USD por millón de tokens de salida),[3] y con supuestos explícitos de aproximadamente 800 tokens de entrada (prompt con la descripción de las cinco categorías, ejemplos balanceados y el incidente) y 30 tokens de salida (JSON estricto con categoría y confianza), cada invocación cuesta del orden de 7×10^{-4} USD, entre 0,0006 y 0,0009 USD según el tamaño del prompt.[4] Para un volumen de 2.646 incidentes trimestrales, con un 34,5 % requiriendo inferencia externa, el costo trimestral de API se sitúa en el orden de 1 USD y en ningún caso por encima de 2 USD, un valor marginal frente a las 32,2 horas-persona liberadas.

Un aspecto que excede el alcance técnico de la solución pero condiciona su efectividad en operación real es la dimensión organizacional de la adopción. La experiencia de campo indica que los empleados con mayor antiguedad tienden a resistir los cambios de procedimiento, incluso cuando ofrecen beneficios demostrables, por familiaridad con los procedimientos existentes y percepción —fundada o no— de que el nuevo sistema puede resultar complejo o invasivo. Por eso la puesta en producción se beneficia de un despliegue gradual: un grupo piloto reducido, extensión progresiva a otros sectores y capacitación breve y contextualizada en cada etapa. El enfoque incremental reduce la fricción inicial, permite ajustar la configuración a cada sector y facilita la confianza de los usuarios finales, condición necesaria para que los beneficios cuantificados —reducción del 71,2 % en el tiempo de registro y exactitud estricta del 73,5 %— se traduzcan en mejoras organizacionales sostenibles.

[2] Precio de lista consultado el 19 de octubre de 2026 en https://www.digitalocean.com/pricing/droplets: Droplet Basic de 2 vCPU y 4 GB, 24 USD/mes, con 4.000 GiB de transferencia y 80 GiB de SSD incluidos; el respaldo automático agrega entre el 20 % (semanal) y el 30 % (diario) del costo de la instancia. No se computan volúmenes adicionales, instantáneas ni direcciones IP reservadas.

[3] Precios de lista consultados el 19 de octubre de 2026 en https://cloud.google.com/vertex-ai/generative-ai/pricing: Gemini 3.6 Flash, precio introductorio de 0,75/3,75 USD por millón de tokens de entrada/salida vigente hasta el 31 de diciembre de 2026; desde el 1 de enero de 2027 rige el precio estándar de 1,50/7,50 USD por millón de tokens, que duplica la cifra. Equivalen a $0{,}75 \times 10^{-6}$ y $3{,}75 \times 10^{-6}$ USD por token.

[4] Cálculo: $800 \times 10^{-6} \times 0{,}75 + 30 \times 10^{-6} \times 3{,}75 \approx 7 \times 10^{-4}$ USD por invocación; el límite de salida configurado es de 100 tokens y el JSON de respuesta ronda los 15–30 tokens. No se computan reintentos ni la transcripción de voz, que se factura por separado. La estimación preliminar, sin fuente, resultaba entre veinte y treinta veces superior al precio de lista vigente.

9.5. Reflexiones sobre la generalización

La generalización de los resultados a otras organizaciones requiere atender al menos tres condiciones. En primer lugar, la disponibilidad de un corpus etiquetado de tamaño y calidad similares al utilizado en este trabajo, lo cual implica una inversión inicial en doble etiquetado y validación del acuerdo inter-evaluador. En segundo lugar, la viabilidad técnica de mantener desplegada localmente la infraestructura de orquestación y persistencia, condición que descarta a organizaciones sin capacidades operativas mínimas para la administración de contenedores Docker. En tercer lugar, la existencia de un marco normativo compatible con el envío de fragmentos descriptivos de incidentes a un servicio externo de inferencia, marco que en el caso argentino se encuentra contemplado por la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000) bajo determinadas condiciones de minimización y pseudonimización descritas en la Sección 7.

10. Conclusiones

El trabajo desarrollado ha cumplido con el objetivo general de evaluar la viabilidad y el desempeño de la automatización del registro inicial de incidentes, con el desarrollo y la implementación del sistema como medio: la combinación de N8N como orquestación, FastAPI como núcleo de procesamiento, PostgreSQL como persistencia, Twilio como canal telefónico y Gemini 3.6 Flash como motor de inferencia resultó viable, eficiente y económicamente accesible para organizaciones medianas. Ese cumplimiento no se extiende al segundo objetivo específico: la exactitud estricta del 73,5 % y el F1 macro quedaron por debajo de las cotas mínimas de 85 % y 0,85 fijadas como criterio de éxito contingente, de modo que ese objetivo se declara no alcanzado. La reducción del tiempo de registro del 71,2 % sí se sostiene como resultado central, con significación estadística (véase la Sección 8). El estado de la hipótesis principal se declara, en consecuencia, por sub-hipótesis: confirmada en tiempo de registro, refutada en el umbral de clasificación y no medida en la proporción de intervención humana por caso.

10.1. Cumplimiento de los objetivos del trabajo

La Tabla 12 sintetiza el grado de cumplimiento de cada objetivo y la evidencia que lo respalda.

Tabla 12. Cumplimiento de los objetivos del trabajo y evidencia de respaldo

Objetivo	Cumplimiento	Evidencia
Objetivo general: viabilidad y desempeño	Alcanzado	Tiempo de registro: 71,2 % (Sección 8)
Objetivo específico 1: arquitectura	Alcanzado	Sección 5
Objetivo específico 2: clasificador	No alcanzado	Exactitud 73,5 %, IC [67,0 %, 79,1 %, macro-F1 0,5207
Objetivo específico 3: tres canales	Alcanzado	Subsección 5.2
Objetivo específico 4: comparación	Alcanzado	Subsección 8.1
Objetivo específico 5: documentación	Alcanzado	Sección B; Sección 11
Objetivo específico 6: errores/calibración	Alcanzado	Subsección 8.5; Subsección 8.4

Fuente: elaboración propia.

El diseño arquitectónico del sistema materializó los principios de separación de responsabilidades y cohesión interna que la disciplina de ingeniería de software prescribe para sistemas distribuidos. La arquitectura de cinco capas —canales de entrada, orquestación, pro-

cesamiento, inferencia y persistencia— distribuyó las responsabilidades entre componentes independientes comunicados mediante interfaces REST. La elección de tecnologías maduras y de despliegue autoalojado (Docker, PostgreSQL, N8N) no fue meramente una decisión instrumental sino una respuesta directa a las restricciones del contexto: organizaciones medianas argentinas que operan con presupuestos acotados, enfrentan limitaciones cambiarias para la adquisición de software comercial y están sujetas a un marco normativo que exige control sobre los datos personales.

Sobre esa arquitectura se construyó el clasificador híbrido, cuyo desempeño sobre el corpus real quedó por debajo de las cotas mínimas fijadas en el segundo objetivo específico, por lo que ese objetivo no se alcanza; la brecha y sus valores se analizan en la Sección 9. La estrategia compositiva —un filtro determinístico de primera etapa y un modelo de lenguaje grande reservado para los casos que el filtro no resuelve con confianza suficiente— resultó, sin embargo, el rasgo más relevante del sistema: la mayor parte del volumen se resuelve sin inferencia externa y el modelo se emplea como árbitro de último recurso antes que como clasificador universal, con el equilibrio entre costo, latencia y exactitud que se detalla en la Subsección 5.5.

La integración de los tres canales paralelos dentro de un único flujo de orquestación confirmó que la diversidad de vías de entrada no introduce dispersión operativa cuando la normalización se realiza tempranamente en el flujo. El canal telefónico, cuya transcripción automática del habla representaba el mayor desafío técnico, alcanzó un tiempo de registro medio de 20,1 segundos, que incluye la duración de la llamada grabada además de la latencia de procesamiento —un valor que lo posiciona como una vía prácticamente útil y no como una demostración de laboratorio. La decisión de normalizar la entrada en el propio N8N, antes de invocar al servicio de clasificación, resultó acertada: aisló la complejidad de cada protocolo de recepción del núcleo de procesamiento, permitiendo que el servicio Python operara siempre sobre la misma estructura de datos independientemente del canal de origen.

La evaluación comparativa entre el flujo manual y el automatizado se presenta en la Subsección 8.1 y se interpreta, con sus alcances y límites, en la Subsección 9.1. En síntesis, el sistema automatizado resultó más rápido que el operador humano en 188 de los 200 pares medidos, con la reducción del tiempo de registro como resultado central. La intervención humana por caso en el flujo automatizado no se midió, de modo que el aporte operativo asociado al gate de revisión se declara como no medido: el sistema libera al personal técnico de la carga

rutinaria de registro, redirigiendo ese tiempo hacia la resolución efectiva de los casos.

La documentación generada —arquitectura, procedimientos de despliegue, especificación OpenAPI, flujo N8N exportable, análisis tipificado de errores y bandas de confianza— constituye el aporte documental del quinto objetivo específico y sustenta el sexto. La tipología de tres patrones de error (descripciones cortas, mezcla de categorías y jerga local) ofrece un punto de partida concreto para replicar o adaptar el sistema y anticipar las categorías de fallo más probables.

10.2. Aportes y generalización

Más alla del cumplimiento de los objetivos específicos, los resultados permiten formular una conclusión acotada a este corpus y al contraste efectivamente realizados: en este corpus de 200 incidentes de mesa de ayuda, en español rioplatense y con cinco categorías canónicas, un modelo de lenguaje grande invocado mediante instrucciones contextualizadas —sin ajuste fino— resultó una alternativa viable para la clasificación de tickets cuando se combina con un filtro determinístico que absorba los casos triviales y reduzca el costo de inferencia. Esta lectura se limita a este dominio: no se realizó una comparación directa contra clasificadores supervisados entrenados sobre el mismo corpus, de modo que no se sostiene aquí una superioridad general de los modelos de lenguaje (véase la Subsección 9.3).

Esta combinación arquitectónica —orquestación visual, filtrado determinístico y LLM como árbitro semántico— constituye un patrón de diseño reproducible para la gestión de servicios en organizaciones medianas de América Latina, donde la restricción presupuestaria y la soberanía de datos son determinantes. La arquitectura propuesta no depende de GPU ni de licencias propietarias; su costo operativo está dominado por el consumo de API de inferencia, cuyo orden de magnitud se cuantifica en la Sección 9.

El trabajo aporta tres contribuciones que se sostienen más alla del caso particular. La primera es una arquitectura reproducible documentada en todos sus niveles —modelo de datos, contrato REST y flujo de orquestación—, replicable por cualquier equipo con conocimientos básicos de Docker, Python y N8N. La segunda es evidencia empírica, bajo un diseño riguroso con doble etiquetado independiente y prueba no paramétrica, sobre el desempeño de LLM en un dominio y una variante lingüística escasamente documentados. La tercera es un marco de implementación que contempla desde el inicio las restricciones normativas argentinas, sin requerir sortear barreras legales ni asumir riesgos de cumplimiento.

11. Recomendaciones y líneas de trabajo futuro

A partir de los resultados obtenidos, del análisis de errores presentado en la Subsección 8.4 y de las limitaciones reconocidas en la Subsección 9.3, se formulan las siguientes recomendaciones para la evolución del sistema y para futuras replicaciones del estudio.

11.1. Incorporación de un clasificador supervisado con corpus propio

Se sugiere incorporar progresivamente un clasificador supervisado entrenado con el corpus etiquetado generado por el propio sistema durante su operación. La acumulación sostenida de decisiones validadas por los sectores responsables constituye un activo informacional valioso que, una vez alcanzado un volumen del orden de 5.000 casos, permitiría entrenar modelos de clasificación específicos del dominio organizacional. Las ventajas potenciales de esta migración incluyen: reducción del costo marginal por incidente clasificado al eliminar la dependencia del servicio externo de inferencia, disminución de la latencia al ejecutar la inferencia localmente, y ganancia de soberanía sobre los datos al evitar la transferencia internacional de fragmentos textuales. Touvron et al. (2023) y trabajos subsiguientes sobre modelos de código abierto sugieren que los LLM desplegados localmente alcanzan rendimientos competitivos en tareas de clasificación cuando se invocan mediante prompts adecuadamente diseñados, lo que abre una vía concreta hacía un sistema completamente autoalojado.

11.2. Ampliación de canales de entrada

Se propone ampliar el conjunto de canales de entrada para incluir mensajería corporativa instantánea, particularmente Slack y Microsoft Teams en aquéllas organizaciones donde estas plataformas constituyen el canal de comunicación dominante. La inclusión de aplicaciones móviles propias para usuarios de campo constituye una extensión natural que atiende la demanda creciente de canales asincrónicos integrados al ecosistema laboral cotidiano. La arquitectura modular propuesta en la Sección 5 facilita esta ampliación: cada nuevo canal requiere únicamente un disparador adicional en N8N, sin modificaciones en las capas de procesamiento ni de persistencia.

11.3. Panel de monitoreo en tiempo real

El sistema incorpora un dashboard de analítica basado en ReCharts que permite visualizar tendencias históricas, distribución por sector y desglose por estado (véase Subsec-

ción 6.8). Como evolución de esta capacidad, se sugiere implementar una capa de monitoreo en tiempo real sustentada sobre tecnologías de observabilidad maduras, tales como Prometheus para la captura de métricas y Grafana para la visualización. Esta capa permitiría supervisar la latencia del sistema, la tasa de aciertos del clasificador, la distribución de carga entre sectores y el costo agregado de las invocaciones al modelo externo, habilitando alertas tempranas frente a degradaciones del servicio. La existencia de endpoints de salud (`GET /api/v1/health`) en cada componente del sistema proporciona los puntos de datos necesarios para instrumentar esta capa de observabilidad.

Más alla del monitoreo operativo del sistema, los datos acumulados de incidentes constituyen una fuente de inteligencia organizacional cuyo valor excede el alcance inmediato de la mesa de ayuda. El registro sistemático de cada incidente —que incluye tipo, sector de destino, tiempo de resolución y la decisión emitida por el clasificador— habilita la construcción de flujos ETL (*Extract, Transform, Load*) que alimenten procesos de análisis orientados a la mejora continua de la organización. A partir de estos datos resulta factible identificar patrones recurrentes de fallas, detectar áreas con mayor demanda de soporte, cuantificar el impacto de cambios en la infraestructura o en los procedimientos internos, y generar indicadores trimestrales que orienten decisiones estratégicas de inversión en capacitación o en renovación de equipamiento. La exposición de estos análisis mediante paneles orientados al nivel directivo —complementarios del panel de monitoreo técnico descrito previamente— transforma la mesa de ayuda de un centro de costo operativo en un nodo de información estratégica para la organización.

11.4. Mecanismos de aprendizaje activo

Se propone incorporar mecanismos de aprendizaje activo (*active learning*), en virtud de los cuales aquellos casos clasificados con baja confianza se prioricen automáticamente para revisión humana, y los resultados de esa revisión retroalimenten al sistema mediante un mecanismo de ajuste incremental. Este enfoque maximiza el rendimiento marginal de cada intervención humana —que de todos modos ocurre en los casos derivados a revisión— y acelera la mejora continua del clasificador con un costo operativo controlado.

11.5. Estudios de replicación

Se sugiere realizar estudios de replicación en organizaciones de distinto tamaño, sector y región geográfica, con el objeto de fortalecer la validez externa de los hallazgos y construir un cuerpo de evidencia empírica sobre el desempeño de soluciones similares en contextos heterogeneos. Particularmente relevantes serían las réplicas en organizaciones del sector público (con mayor proporción de incidentes vinculados a sistemas administrativos), en organizaciones del sector industrial (con mayor proporción de incidentes vinculados a infraestructura física) y en organizaciones de mayor tamaño donde el volumen diario de incidentes supere las 100 unidades.

11.6. Evaluación de modelos de lenguaje de código abierto

Se propone evaluar la integración de modelos de lenguaje de código abierto desplegados localmente —tales como Llama 3, Mistral o Qwen— como alternativa al modelo externo Gemini 3.6 Flash. Esta evaluación permitiría cuantificar el costo en exactitud frente a la ganancia en soberanía de datos, la eliminación de la dependencia de un proveedor único y la reducción de costos operativos a largo plazo. Touvron et al. (2023) sugieren que los modelos abiertos contemporáneos alcanzan rendimientos competitivos en tareas de clasificación, y la realización de un benchmark comparativo sobre el mismo corpus de 200 casos constituiría una contribución adicional significativa al estado del arte.

11.7. Base de conocimientos para resolución automática

El sistema desarrollado en este trabajo se concentra en la clasificación y derivación de incidentes hacía los sectores responsables, pero no aborda la resolución de aquellos casos que admiten soluciones estandarizadas y repetibles. Se recomienda, como línea de trabajo futuro, la incorporación de una base de conocimientos que permita al sistema no solo categorizar el incidente sino también proponer una resolución cuando el problema detectado coincida con patrones previamente documentados y validados.

La base de conocimientos operaría como un repositorio estructurado de soluciones verificadas, organizado por tipo de incidente, manifestación del problema y sector responsable. Ante un nuevo incidente, el sistema consultaría la base tras la clasificación inicial y, en caso de hallar una coincidencia de alta confianza, ofrecería al usuario una respuesta clara y concisa con los pasos necesarios para resolver el problema sin intervención del personal técnico. Este

mecanismo de autoservicio reduciría la carga sobre los sectores de soporte al absorber los casos simples y repetitivos —tales como reinicios de sesión, restablecimiento de contraseñas o configuraciones básicas de correo electrónico— que, por su naturaleza, representan una proporción significativa del volumen de incidentes en entornos de mesa de ayuda, aunque su cuantificación precisa en la organización de referencia excede el alcance de este estudio.

La construcción de esta base de conocimientos puede articularse con el mecanismo de aprendizaje activo descrito en la Subsección 11.4: cada caso resuelto por un operador humano y validado como correcto constituye un candidato para ser incorporado a la base de conocimientos, previa curación editorial que garantice la claridad y la precisión de las instrucciones. La viabilidad técnica de este enfoque se ve reforzada por la creciente capacidad de los modelos de lenguaje para generar respuestas estructuradas a partir de documentación técnica, línea de investigación que conecta de manera directa con la evaluación de modelos de código abierto propuesta en la Subsección 11.6.

11.8. Agente conversacional telefónico en tiempo real

Se propone, como línea de evolución de mayor alcance para el canal telefónico, reemplazar el esquema actual de captura de toma única —una grabación de hasta 45 segundos que se transcribe de forma diferida— por un agente que mantenga un diálogo en tiempo real con el usuario. Sobre Twilio Media Streams, que habilita el envío bidireccional del audio de la llamada hacia un servicio propio mediante WebSocket, el agente podría formular exclusivamente preguntas de aclaración pertinentes al problema reportado cuando la descripción inicial resultara insuficiente, consultar al usuario si desea agregar información tras detectar una pausa prolongada, comunicarle el número de incidente asignado una vez completado el registro, y cerrar la llamada de forma autónoma. Esta capacidad elevaría la calidad de la captura telefónica y reduciría la necesidad de contacto posterior, en línea con el tercer objetivo específico de ampliar la accesibilidad de los canales sin incrementar la complejidad percibida por el usuario final.

La adopción de esta arquitectura estuvo condicionada por el estado de madurez de las interfaces de audio bidireccional en tiempo real disponibles durante el desarrollo de este trabajo, que se encontraban en fase de vista previa (*preview*) y con una superficie de programación sujeta a cambios, lo que representaba un riesgo de cronograma incompatible con los plazos del estudio. A ello se suma que la implementación exige infraestructura no contemplada en

la arquitectura actual —un punto de entrada WebSocket con soporte de *upgrade* en el proxy inverso, la transcodificación del audio telefónico (mu-law, 8 kHz) hacia el formato de entrada del modelo (PCM, 16 kHz) y su retorno, la detección de actividad de voz y la gestión de turnos de habla— junto con la redefinición del modelo de costos, hoy expresado por grabación, hacia un esquema por minuto de transmisión. Por estos motivos se recomienda abordar esta línea mediante una prueba de concepto acotada y conservar el mecanismo actual de grabación y transcripción como modo degradado de respaldo para garantizar la continuidad del canal.

Referencias bibliograficas

Ahuja, K., Diddee, H., Hada, R., Ochieng, M., Ramesh, K., Jain, P., Nambi, A., Ganu, T., Segal, S., Ahmed, M., Bali, K., & Sitaram, S. (2023). MEGA: Multilingual Evaluation of Generative AI. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 4232-4267. https://doi.org/10.18653/v1/2023.emnlp-main.258

American Psychological Association. (2017). Ethical Principles of Psychologists and Code of Conduct [2002, con enmiendas efectivas del 1 de junio de 2010 y del 1 de enero de 2017; consultado el 19 de octubre de 2026]. https://www.apa.org/ethics/code

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for Human-AI Interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI 2019)*, 1-13. https://doi.org/10.1145/3290605.3300233

Apache Software Foundation. (2024). *Apache Airflow Documentation (Version 2.10)* [Documentacion tecnica]. https://airflow.apache.org/docs/

Bernal-Beltrán, T., Paredes-Valverde, M. A., Salas-Zárate, M. d. P., García-Díaz, J. A., & Valencia-García, R. (2025). Sentiment Analysis in Mexican Spanish: A Comparison Between Fine-Tuning and In-Context Learning with Large Language Models. *Future Internet*, *17*(10), 445. https://doi.org/10.3390/fi17100445

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., … Amodei, D. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 1877-1901.

Cohen, J. (1960). A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, *20*(1), 37-46. https://doi.org/10.1177/001316446002000104

Colavito, G., Lanubile, F., & Novielli, N. (2025). Benchmarking large language models for automated labeling: The case of issue report classification. *Information and Software Technology*, *184*, 107758. https://doi.org/10.1016/j.infsof.2025.107758

Crispin, L., & Gregory, J. (2009). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley.

Date, C. J. (2003). *An Introduction to Database Systems* (8.ª ed.). Addison-Wesley.

Fielding, R. T., & Taylor, R. N. (2002). Principled Design of the Modern Web Architecture. *ACM Transactions on Internet Technology*, *2*(2), 115-150. https://doi.org/10.1145/514183.514185

Galup, S. D., Dattero, R., Quan, J. J., & Conger, S. (2009). An Overview of IT Service Management. *Communications of the ACM*, *52*(5), 124-127. https://doi.org/10.1145/1506409.1506439

Gómez, L., Bustos, C., & Sevilla, G. (2026). Automatización de Mesa de Ayuda N8N (Version 1.0.0) [Software de computadora. GitHub]. https://github.com/lucaGomezB/Automatizacion-de-Mesa-de-Ayuda-N8N

Grandury, M., Aula-Blasco, J., Falcão, J., Fourrier, C., González Saiz, M., Martínez, G., Santamaria Gomez, G., Agerri, R., Aldama García, N., Chiruzzo, L., Conde, J., Gomez Adorno, H., Guerrero Nieto, M., Ivetta, G., López Fuertes, N., Plaza-del-Arco, F. M., Martín-Valdivia, M.-T., Montoro Zamorano, H., Muñoz Sanz, C., … Zubiaga, I. (2025). La Leaderboard: A Large Language Model Leaderboard for Spanish Varieties and Languages of Spain and Latin America. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 32482-32524. https://doi.org/10.18653/v1/2025.acl-long.1561

Hernández Sampieri, R., & Mendoza Torres, C. P. (2018). *Metodología de la investigación: Las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill.

Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.

Honorable Congreso de la Nación Argentina. (2000, octubre). Ley 25.326 de Protección de los Datos Personales [Boletín Oficial de la República Argentina]. http://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/norma.htm

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, *55*(12), 1-38. https://doi.org/10.1145/3571730

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* [Borrador de la 3ra edicion, disponible en el sitio de los autores].

Ladas, C. (2009). *Scrumban: Essays on Kanban Systems for Lean Software Development*. Modus Cooperandi Press.

Lai, V. D., Nguyen, C., Ngo, N., Nguyen, T., Dernoncourt, F., Rossi, R. A., & Nguyen, T. H. (2023). Okapi: Instruction-tuned Large Language Models in Multiple Languages with Reinforcement Learning from Human Feedback. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 318-327. https://doi.org/10.18653/v1/2023.emnlp-demo.28

Landis, J. R., & Koch, G. G. (1977). The Measurement of Observer Agreement for Categorical Data. *Biometrics*, *33*(1), 159-174. https://doi.org/10.2307/2529310

Liu, F., He, X., Zhang, T., Chen, J., Li, Y., Yi, L., Zhang, H., Wu, G., & Shi, R. (2025). TickIt: Leveraging Large Language Models for Automated Ticket Escalation. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE)*, 343-354. https://doi.org/10.1145/3696630.3728558

Mayor-Rocher, M., Melero, N., Merino-Gómez, E., Grandury, M., Conde, J., & Reviriego, P. (2026). Evaluating large language models with tests of Spanish as a foreign language: pass or fail? [Advance article; volume/pages pending]. *Applied Linguistics*. https://doi.org/10.1093/applin/amag048

Mosqueira-Rey, E., Hernández-Pereira, E., Alonso-Ríos, D., Bobes-Bascarán, J., & Fernández-Leal, Á. (2023). Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*, *56*(4), 3005-3054. https://doi.org/10.1007/s10462-022-10246-w

n8n GmbH. (2024). *n8n Documentation* [Documentacion tecnica oficial]. https://docs.n8n.io/

Office of Government Commerce. (2011). *ITIL Service Operation* [Publicado por encargo del Office of Government Commerce; la titularidad de ITIL es hoy de la herencia AXELOS/PeopleCert]. The Stationery Office.

Paket, E., Şenerkek, G., Akyol, F. B., & Salman, F. (2024). IT Service Desk Ticket Classification via Large Language Models. *2024 9th International Conference on Computer Science and Engineering (UBMK)*, 135-140. https://doi.org/10.1109/ubmk63289.2024.10773473

Paramesh, S. P., & Shreedhara, K. S. (2019). Automated IT Service Desk Systems Using Machine Learning Techniques. En *Lecture Notes in Networks and Systems* (pp. 331-346, Vol. 43). Springer. https://doi.org/10.1007/978-981-13-2514-4_28

Parlamento Europeo y Consejo de la Unión Europea. (2016, abril). Reglamento (UE) 2016/679 — Reglamento General de Protección de Datos [Diario Oficial de la Unión Europea, L 119/1].

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, *12*, 2825-2830.

Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, *2*(1), 37-63.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9.ª ed.). McGraw-Hill.

Rabiner, L., & Juang, B.-H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, 28492-28518.

Ramón-Ferrer, V., Badenes-Olmedo, C., & Corcho, Ó. (2026). Spanish Triple-to-Text Benchmark on Low-Resource Large Language Models. *Semantic Web*, *17*(3). https://doi.org/10.1177/22104968261440701

Revina, A., Buza, K., & Meister, V. G. (2020). IT Ticket Classification: The Simpler, the Better. *IEEE Access*, *8*, 193380-193395. https://doi.org/10.1109/ACCESS.2020.3032840

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.

Secretaría de la Pequeña y Mediana Empresa. (2026). ¿Qué es una MiPyME? [Ministerio de Economía de la Nación Argentina; consultado el 19 de octubre de 2026]. https://www.argentina.gob.ar/produccion/registrar-una-pyme/que-es-una-pyme

Shneiderman, B. (2020). Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy. *International Journal of Human–Computer Interaction*, *36*(6), 495-504. https://doi.org/10.1080/10447318.2020.1741118

Sokolova, M., & Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing & Management*, *45*(4), 427-437. https://doi.org/10.1016/j.ipm.2009.03.002

Stallings, W., & Brown, L. (2017). *Computer Security: Principles and Practice* (4.ª ed.). Pearson.

The PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation* [Documentacion tecnica oficial]. https://www.postgresql.org/docs/15/

Tiangolo, S. R. (2024). *FastAPI Documentation* [Documentacion tecnica oficial]. https://fastapi.tiangolo.com/

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., … Scialom, T. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models [arXiv:2307.09288]. https://doi.org/10.48550/arXiv.2307.09288

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 5998-6008.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*(3), 261-272. https://doi.org/10.1038/s41592-019-0686-2

Wiggins, A. (2017). The Twelve-Factor App [Manifiesto de arquitectura de software]. https://12factor.net/

Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, *22*(158), 209-212. https://doi.org/10.1080/01621459.1927.10502953

Wu, T., Terry, M., & Cai, C. J. (2022). AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI 2022)*, 1-22. https://doi.org/10.1145/3491102.3517582

Xuan, W., Yang, R., Qi, H., Zeng, Q., Xiao, Y., Feng, A., Liu, D., Xing, Y., Wang, J., Gao, F., Lu, J., Jiang, Y., Li, H., Li, X., Yu, K., Dong, R., Gu, S., Li, Y., Xie, X., … Li, I. (2025). MMLU-ProX: A Multilingual Benchmark for Advanced Large Language Model Evaluation. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1513-1532. https://doi.org/10.18653/v1/2025.emnlp-main.79

Zangari, A., Marcuzzo, M., Schiavinato, M., Gasparetto, A., & Albarelli, A. (2023). Ticket automation: An insight into current research with applications to multi-level classification

scenarios. *Expert Systems with Applications*, *225*, 119984. https://doi.org/10.1016/j.eswa.2023.119984

Zanzotto, F. M. (2019). Viewpoint: Human-in-the-loop Artificial Intelligence. *Journal of Artificial Intelligence Research*, *64*, 243-252. https://doi.org/10.1613/jair.1.11345

Zhang, R., Cahyawijaya, S., Cruz, J. C. B., Winata, G. I., & Aji, A. F. (2023). Multilingual Large Language Models Are Not (Yet) Code-Switchers. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 12567-12582. https://doi.org/10.18653/v1/2023.emnlp-main.774

A. Diagrama de arquitectura del sistema

El Anexo A referencia los diagramas de la arquitectura propuesta en notación UML. Los diagramas de despliegue y de secuencia se incluyen como figuras de la Sección 5, de modo que no se reproducen nuevamente aquí; el diagrama de componentes se incluye como Figura 1 de la Sección 5. Los tres artefactos son:

- **Diagrama de despliegue** (Figura 3, Sección 5): los cinco componentes principales (N8N, FastAPI, PostgreSQL, Gemini 3.6 Flash, Twilio), el proxy inverso Nginx que, en el diseño, provee terminación TLS 1.2/1.3 en el perímetro del sistema (no incluido en el despliegue publicado), y las relaciones de comunicación entre ellos. Fuente versionada: `docs/diagrams/despliegue.md`.
- **Diagrama de secuencia** (Figura 4, Sección 5): flujo extremo a extremo de un incidente desde su recepción en el canal de correo electrónico hasta la confirmación al usuario, incluyendo las interacciones con el clasificador híbrido y la base de datos. Fuente versionada: `docs/diagrams/secuencia.md`.
- **Diagrama de componentes** (Figura 1, Sección 5): organización interna del módulo Python por capas (routes, services, repositories, models, classifiers), con sus dependencias y relaciones. Fuente versionada: `docs/diagrams/componentes.md`.

Los diagramas se generan a partir de fuentes Mermaid versionadas en formato `.md` en el directorio `docs/diagrams/` del repositorio y se renderizan a imágenes PNG para su inclusión en el documento (Gómez et al. 2026). Las Figuras 3 y 4 se presentan en la Sección 5 y se referencian desde este anexo, sin duplicar el material gráfico.

B. Repositorio de código fuente

El código fuente del módulo Python, los scripts de migración de la base de datos PostgreSQL gestionados mediante Alembic, los archivos de configuración del despliegue en contenedores Docker y los flujos de integración continua sobre GitHub Actions se encuentran disponibles en el repositorio público del proyecto (Gómez et al. 2026). El commit de referencia para la versión entregada al jurado se consigna mediante el tag `v1.0.0`. La creación de dicho tag sobre el commit definitivo entregado al jurado queda como acción pendiente al cierre de los changes c-76 y c-77; una vez creado, se registrará en este mismo apartado el hash completo del commit etiquetado como referencia verificable de la versión evaluada.

La estructura del repositorio separa el módulo de procesamiento (`App/Backend/`, con

la aplicación FastAPI, las migraciones Alembic y las pruebas), la interfaz de usuario (`App/Frontend/`, React con TypeScript), la definición del flujo de orquestación (`n8n/workflow.json`), los artefactos de evaluación (`evaluation/`), el corpus pseudonimizado (`data/`), los scripts auxiliares (`scripts/`) y la documentación (`docs/`), junto con las especificaciones de los changes (`openspec/`) y los archivos de orquestación local `docker-compose.yml` y `Makefile`. El archivo `README.md` describe esta organización y reúne las instrucciones detalladas para el despliegue local en menos de 15 minutos, por lo que constituye el punto de entrada al código fuente disponible en el repositorio público (Gómez et al. 2026).

C. Esquema de base de datos

El esquema completo de la base de datos PostgreSQL se documenta mediante scripts SQL versionados que definen las siete entidades descritas en la Tabla 4 de la Sección 5, las restricciones de integridad referencial (claves foráneas con eliminación en cascada condicional), los índices secundarios sobre los atributos de búsqueda frecuente (fecha de creación, sector responsable y estado actual del ticket), y la generación de los identificadores de incidente. Las migraciones incrementales se gestionan mediante Alembic, lo que permite reproducir el estado de la base en cualquier punto histórico del desarrollo y revertir cambios cuando resulte necesario.

La migración inicial incluye la creación de las tablas de catálogo (`sector`, `estado`, `canal_origen`) con sus valores semilla, garantizando que un despliegue nuevo del sistema comience con los datos de referencia correctos. Las migraciones subsiguientes documentan la evolución del esquema durante las seis iteraciones de desarrollo descritas en la Sección 4.

El diagrama entidad-relación de las siete entidades del modelo se incluye como Figura 5 de la Sección 5, junto con la descripción de cada entidad en la Tabla 4; este anexo no lo reproduce nuevamente. La fuente Mermaid versionada del diagrama reside en `docs/diagrams/er_modelo_dat`
El detalle de atributos, tipos, claves primarias y foráneas, restricciones de integridad referencial e índices secundarios se conserva como scripts SQL y migraciones Alembic versionadas en el repositorio.

D. Especificación OpenAPI de la interfaz REST

La especificación completa de la interfaz REST se genera automáticamente mediante FastAPI conforme a la versión 3.1 de la especificación OpenAPI (Tiangolo, 2024). El docu-

mento resultante describe los puntos de entrada listados en la Tabla 5 de la Sección 5, los esquemas de los cuerpos de solicitud y respuesta validados mediante Pydantic v2, los códigos de estado posibles para cada método HTTP, y ejemplos representativos para cada caso de uso.

La interfaz se encuentra disponible adicionalmente en formato interactivo a través del componente Swagger UI integrado en el servidor, accesible bajo la ruta `/docs` durante la ejecución, y en formato estático como archivo `openapi.json` en el directorio `docs/` del repositorio. La sincronización entre el código y la especificación estática se verifica automáticamente en el pipeline de integración continua.

La especificación OpenAPI vigente describe 33 puntos de entrada que exponen 41 métodos HTTP y 44 esquemas de datos, agrupados por recurso: autenticación, catálogos, clasificaciones, incidentes, directorio, estadísticas, telefonía, control de costo y salud. El documento completo reside en `docs/openapi.json` del repositorio y se expone además en formato interactivo bajo la ruta `/docs`.

A título ilustrativo, los 44 esquemas de datos comprenden las estructuras de los catálogos, las entidades de dominio y los objetos de respuesta de cada recurso. El esquema `IncidenteRead`, por ejemplo, expone el identificador del ticket, la descripción pseudonimizada, la prioridad, el sector y los sectores adicionales, el estado, el canal de origen, el reportante, las marcas temporales de ingreso y persistencia, y los indicadores de latencia end-to-end y de latencia anómala, con sus tipos y campos de solo lectura. El detalle exhaustivo de atributos, tipos y ejemplos reside en `docs/openapi.json`.

E. Configuración del flujo N8N

La exportación del flujo de orquestación en formato JSON incluye la totalidad de los 32 nodos configurados, sus parámetros operativos, las credenciales referenciadas mediante identificadores opacos sin exposición de valores, y los conectores entre nodos. El archivo `n8n/workflow.json` contiene esta exportación. La importación de este artefacto en una instancia limpia de N8N 2.11.2 reproduce íntegramente el comportamiento operativo del sistema, sujeta únicamente a la provisión de las credenciales correspondientes a los servicios externos integrados (servidor IMAP de correo, cuenta Twilio y clave de API de Gemini) mediante el sistema de credenciales nativo de la plataforma.

La Figura 8 presenta una reconstrucción esquemática del flujo, derivada de la exporta-

ción JSON y agrupada por etapa funcional. Los 29 nodos operativos (más 3 notas adhesivas, 32 nodos en total) se organizan en siete etapas: entrada (triggers), con la recepción de correo por IMAP y los disparadores de telefonía, formulario web y notificación de clasificación; validación y normalización de la información de entrada, incluida la marca de canal y el sellado de ingreso; envío al backend, con la autenticación del operador y la solicitud POST del incidente; ruteo por canal de origen; revisión humana, con la notificación al operador designado y las respuestas de handoff; respuesta y auditoría del ticket; y aviso post-revisión. La parametrización completa de cada nodo reside en `n8n/workflow.json`.

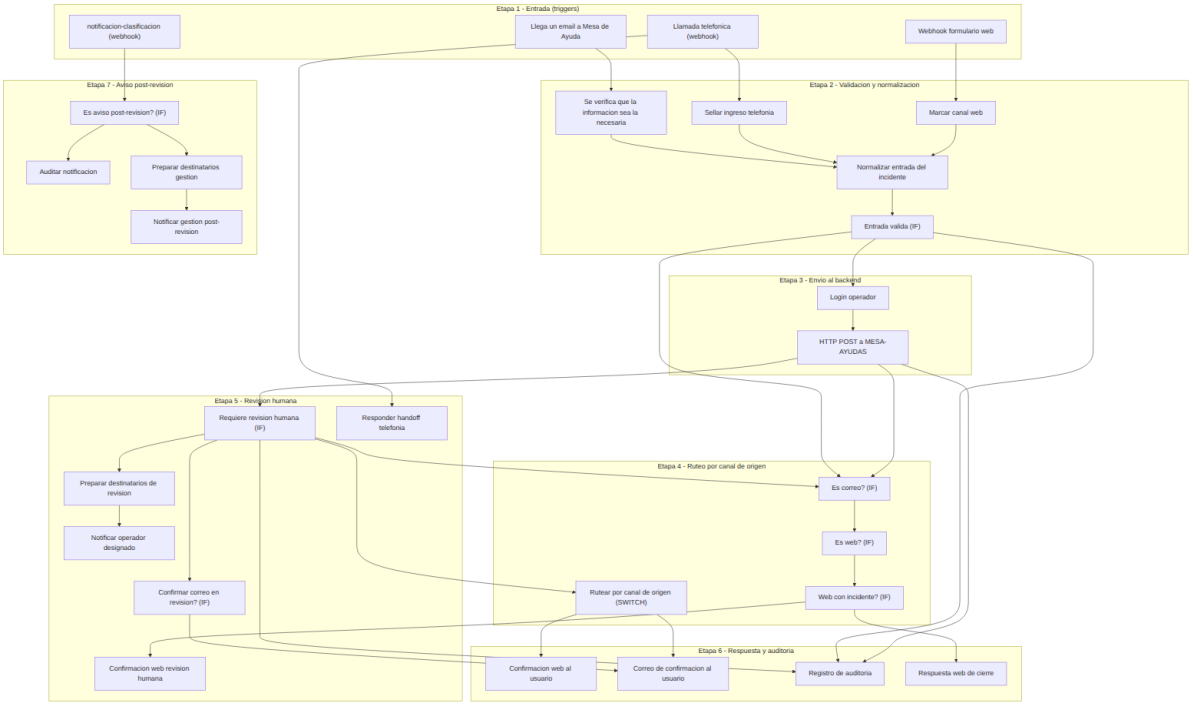


Figura 8. Reconstrucción esquemática del flujo de orquestación N8N, derivada de la exportación `n8n/workflow.json`. No es una captura de la interfaz: los 29 nodos operativos se agrupan por etapa funcional.

F. Corpus de validación

El corpus de 200 incidentes utilizado para la validación experimental reside en el repositorio en formato JSON, en su forma pseudonimizada y versionada, bajo la ruta `data/corpus_evaluacion_p`. Cada caso incluye el identificador anónimo, la descripción cruda pseudonimizada, el canal de origen, el sector asignado por consenso humano, los sectores adicionales de verdad y el tiempo de registro medido en cada flujo. La categoría predicha por el clasificador y la confianza asociada no forman parte del corpus: se conservan en `evaluation/predicciones.json`, generado por el framework de evaluación. Se incluyen además las planillas de cronometraje

del flujo manual y los registros automáticos generados por el flujo automatizado.

Cada uno de los 200 casos del corpus se compone de siete campos: `id` (identificador anónimo, por ejemplo `R001`), `descripcion` (texto pseudonimizado, con los identificadores personales reemplazados por etiquetas genéricas como `[EMAIL]`, `[TELEFONO]`, `[HOST]` y `[PERSONA]`), `canal_origen` (correo electrónico, formulario web o llamada telefónica), `sector_asignado` (sector principal de verdad, exactamente uno de los cinco sectores canónicos), `sectores_adicionales` (sectores de verdad complementarios, sin repetir el asignado), `tiempo_manual_s` (tiempo de clasificación manual, en segundos) y `tiempo_automatizado_s` (tiempo del sistema automatizado, en segundos). El corpus completo reside en `data/corpus_evaluacion_` y las predicciones del clasificador en `evaluation/predicciones.json`.

Por razones de privacidad y en cumplimiento de la Ley 25.326 (Honorable Congreso de la Nación Argentina, 2000), las descripciones textuales han sido pseudonimizadas previamente a su inclusión en el repositorio público, reemplazando nombres propios, direcciones de correo y otros identificadores personales por etiquetas genéricas.

G. Documentación operativa

La documentación operativa del sistema describe los procedimientos de despliegue inicial, configuración recurrente, respaldo y restauración, monitoreo activo y respuesta ante incidentes operativos. Se incluyen instrucciones detalladas para los entornos de desarrollo, preproducción y producción, así como una guía de resolución de incidencias frecuentes orientada al personal de operaciones de la organización adoptante. La documentación se mantiene actualizada en el repositorio del proyecto y se versiona conjuntamente con el código fuente, garantizando coherencia entre la versión desplegada y los procedimientos vigentes en cada momento del ciclo de vida del sistema.

El runbook operativo (`docs/operational-guide.md`) se organiza en una sección de prerequisitos y once secciones de procedimiento: despliegue inicial (clonado del repositorio, configuración de variables de entorno, certificados TLS, arranque del stack, verificación de salud, importación del flujo N8N y acceso al frontend), detención y reinicio de los servicios, respaldo y restauración de PostgreSQL, monitoreo de salud (endpoints, contenedores, logs estructurados y cola de revisión humana), regeneración de la especificación OpenAPI, actualización de dependencias del backend, aplicación manual de migraciones Alembic, evaluación del clasificador, arnés de ejecución en seco (*dry-run*), medición de latencia end-to-end y guarda

de costo en runtime (postura al arranque, *enforcement*, política *fail-closed* y reproducibilidad de la suite de integración).

H. Especificación del prompt de Gemini 3.6 Flash

Este anexo documenta la especificación del prompt enviado al modelo `gemini-3.6-flash` (Gemini 3.6 Flash en el texto) y los parámetros de configuración de la inferencia. El prompt se mantiene en el repositorio bajo `docs/prompt_gemini.txt` y es único para los tres canales de entrada.

H.1. Estructura y contenido del prompt

El prompt sigue una estructura fija en español organizada en cinco componentes funcionales y un conjunto de reglas de desambiguación:

1. **Instrucción de rol**: define que el modelo actúa como agente especializado en clasificación de incidentes técnicos en español rioplatense, lo que ancla el comportamiento en el dominio y el dialecto del estudio.
2. **Definición de categorías**: presenta los cinco sectores canónicos con sus *strings* exactos (sensibles a mayúsculas y sin tildes), un ámbito descriptivo por sector y un ejemplo representativo extraído del dominio real por cada uno.
3. **Formato de respuesta**: solicita un JSON válido con exactamente dos campos, `categoría` (uno de los cinco valores canónicos) y `confianza` (número entre 0,0 y 1,0), sin texto adicional, comentarios ni explicaciones.
4. **Lógica de decisión**: indica analizar la descripción y asignar una categoría única entre las cinco opciones; ante elementos de múltiples categorías, elegir la dominante.
5. **Validación sintáctica**: exige validar la corrección sintáctica del JSON antes de devolverlo y reducir la confianza cuando no sea posible clasificar con seguridad.
6. **Reglas de desambiguación de frontera**: las acciones sobre aplicaciones o sistemas operativos (acceder, iniciar sesión, abrir una aplicación) se orientan a `Soporte Tecnico Software`; los dispositivos físicos y su digitalización (escáner, impresora, periférico, error de digitalización) a `Soporte Tecnico Hardware`; y la infraestructura y los servicios de plataforma (servidor, red, SMTP, VM) a `Sistemas`. Estas reglas corrigen casos de frontera medidos sobre el corpus.

H.2. Parámetros de configuración

La invocación del modelo se realiza mediante el SDK `google-genai` (`>=2.20.0`), con los hiperparámetros que resume la Tabla 13, optimizados para equilibrar exactitud, latencia y costo de inferencia.

Tabla 13. Parámetros de configuración de la inferencia con `gemini-3.6-flash`.

Parámetro	Valor	Justificación
`temperature`	0,3	Valor bajo que reduce la variabilidad de la generación y mejora la consistencia en tareas de clasificación determinística.
`top_p`	0,9	*Nucleus sampling* que admite variantes léxicas válidas en español rioplatense sin sacrificar coherencia.
`max_tokens`	100	Límite suficiente para la respuesta JSON, con margen ante variaciones y sin truncamiento.
`candidate_count`	1	Respuesta única sin remuestreo; optimiza la latencia para el procesamiento en tiempo real.
`safety_settings`	Parcialmente desactivados	Desactiva filtros que podrían rechazar descripciones legítimas de incidentes, manteniendo los filtros contra contenido verdaderamente tóxico.
`timeout`	10 segundos	Límite máximo de latencia; al excederse, el clasificador devuelve confianza 0,0 y marca el caso para revisión humana.
SDK	`google-genai >=2.20.0`	Versión vigente al momento del estudio, documentada para reproducibilidad.

Fuente: elaboración propia a partir de la documentación del SDK `google-genai`.

H.3. Validación de la respuesta

Antes de aceptar la respuesta del modelo, el módulo de clasificación valida que el texto sea JSON parseable, que contenga exactamente las claves `categoría` y `confianza`, que la categoría sea uno de los cinco valores canónicos y que la confianza sea un número en el intervalo $[0{,}0, 1{,}0]$. Si cualquiera de estas condiciones falla, se registra la excepción, se asigna confianza 0,0, se marca el incidente para revisión humana y no se propagan estados inconsistentes a la base de datos. Este procedimiento garantiza la confiabilidad operativa frente a errores de formato o alucinaciones del modelo. El Listado 1 resume la rutina de validación.

```
1 def validar_respuesta_gemini(response_text: str) -> tuple[bool, dict]:
2     try:
3         data = json.loads(response_text)
4     except json.JSONDecodeError as e:
5         return False, {"error": f"JSON invalido: {e}"}
```

```
    if "categoria" not in data:
        return False, {"error": "Falta campo 'categoria'"}
    if data["categoria"] not in SECTORES_CANONICOS:
        return False, {"error": f"Categoria invalida: {data['categoria']}"}
    if "confianza" not in data:
        return False, {"error": "Falta campo 'confianza'"}
    if not isinstance(data["confianza"], (int, float)) \
            or not (0.0 <= data["confianza"] <= 1.0):
        return False, {"error": f"Confianza invalida: {data['confianza']}"}
    return True, data
```

Listing 1. Validación de la respuesta de Gemini (resumen).

H.4. Iteración y mejora del prompt

El prompt se validó y ajustó de forma iterativa sobre un conjunto de aproximadamente 100 incidentes representativos en entornos de preproducción, sin exposición al corpus de validación de 200 casos. Las versiones históricas del prompt se conservan en el repositorio bajo `docs/prompt_gemini_history/` para auditoría y eventual reversión.

H.5. Recomendaciones para futuras iteraciones

Ante una degradación de las métricas por cambios en el modelo, se recomienda profundizar los ejemplos por clase (de dos a cuatro o cinco), agregar contraejemplos de incidentes limítrofes, especificar umbrales de confianza explícitos, incluir un diccionario de términos clave por categoría y, una vez acumulado un corpus del orden de 5.000 casos etiquetados, migrar a un modelo ajustado específicamente al dominio.

I. Trazabilidad de las correcciones del dictamen

El registro de correcciones derivado del dictamen evaluador se conserva como artefacto versionado en el repositorio, bajo la ruta `docs/Tesis/Correcciones/dictamen-correcciones.md`. El documento compila, de forma acumulativa, cada corrección aplicada al manuscrito y las decisiones adoptadas sobre las observaciones que no admitían una resolución inmediata (por ejemplo, la convención estadística del estadístico *W*, la reformulación de la hipótesis principal y el tratamiento de los reintentos como limitación conocida). Se incluye aquí como ejemplo de trazabilidad: cada entrada del registro vincula una observación del dictamen con el archivo y

la línea corregidos, de modo que la correspondencia entre la revisión solicitada y la revisión efectuada resulte auditable de extremo a extremo.